

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN



HCMUTE

BÁO CÁO
KHOÁ LUẬN TỐT NGHIỆP
XÂY DỰNG NỀN TẢNG CUNG CẤP
KHÓA HỌC TRỰC TUYẾN

GVHD: PGS.TS Hoàng Văn Dũng

SVTH:

Phan Thị Mỹ Linh 22110172

Nguyễn Hữu Lộc 22110179

Vương Trí Hùng 22110153

Tp.Hồ Chí Minh, tháng 1 năm 2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN



HCMUTE

BÁO CÁO
KHOÁ LUẬN TỐT NGHIỆP
XÂY DỰNG NỀN TẢNG CUNG CẤP
KHÓA HỌC TRỰC TUYẾN

GVHD: PGS.TS Hoàng Văn Dũng

SVTH:

Phan Thị Mỹ Linh 22110172

Nguyễn Hữu Lộc 22110179

Vương Trí Hùng 22110153

Tp.Hồ Chí Minh, tháng 1 năm 2026

ĐỀ CƯƠNG

PHẦN MỞ ĐẦU

1. TÍNH CẤP THIẾT CỦA ĐỀ TÀI
2. MỤC ĐÍCH CỦA ĐỀ TÀI
3. CÁC TIẾP CẬN VÀ PHƯƠNG PHÁP NGHIÊN CỨU
 - 3.1. Đối tượng
 - 3.2. Phạm vi nghiên cứu
4. KẾT QUẢ DỰ KIẾN ĐẠT ĐƯỢC
 - 4.1. Về kiến thức
 - 4.2. Về đề tài
5. CÔNG NGHỆ SỬ DỤNG

PHẦN NỘI DUNG

1. CHƯƠNG 1. CƠ SỞ LÝ THUYẾT
 - 1.1. CÔNG NGHỆ PHẦN MỀM VÀ WEBSITE
 - 1.1.1. Tổng quan về Website
 - 1.1.2. Java Spring và Hệ sinh thái Cloud-Native
 - 1.1.3. Next.js
 - 1.2. GIỚI THIỆU VỀ VINA ACADEMY
 - 1.2.1. Tổng quan về Vina Academy
 - 1.2.2. Vai trò trong hệ thống
 - 1.3. GIỚI THIỆU CÁC THÀNH PHẦN TRONG ỨNG DỤNG
 - 1.3.1. Bối cảnh và Nền tảng Kế thừa
 - 1.3.2. Kiến trúc Microservices (Microservices Architecture)
 - 1.4. CÁC CÔNG NGHỆ LIÊN QUAN
 - 1.4.1. Json Web Token
 - 1.4.2. Maven
 - 1.4.3. Spring Cloud Ecosystem
 - 1.4.4. Apache Kafka
 - 1.4.5. PostgreSQL & pgvector
 - 1.4.6. LangChain
 - 1.4.7. Docker
2. CHƯƠNG 2. PHƯƠNG PHÁP NGHIÊN CỨU
 - 2.1. PHƯƠNG PHÁP TIẾP CẬN
 - 2.1.1. Nghiên cứu tài liệu
 - 2.1.2. Phân tích yêu cầu
 - 2.1.3. Lựa chọn công nghệ
 - 2.2. CÔNG CỤ VÀ MÔI TRƯỜNG PHÁT TRIỂN

- 2.2.1. Công cụ phát triển
 - 2.2.2. Công cụ kiểm thử
 - 2.2.3. Môi trường triển khai và Hạ tầng (Deployment & Infrastructure)
- 3. CHƯƠNG 3. KHẢO SÁT HIỆN TRẠNG VÀ MÔ HÌNH HÓA YÊU CẦU
 - 3.1. KHẢO SÁT HIỆN TRẠNG
 - 3.2. KHẢO SÁT MỘT SỐ WEBSITE
 - 3.2.1. Udemy (<https://www.udemy.com/>)
 - 3.2.2. Coursera (<https://www.coursera.org/>)
 - 3.2.3. edX (<https://www.edx.org/>)
 - 3.3. YÊU CẦU CHỨC NĂNG
 - 3.3.1. Người dùng chưa đăng ký (Unregistered User)
 - 3.3.2. Học viên (Student)
 - 3.3.3. Giảng viên (Instructor)
 - 3.3.4. Ban quản trị (Admin/Owner)
 - 3.3.5. Nhân viên (Staff)
 - 3.4. YÊU CẦU PHI CHỨC NĂNG (NON-FUNCTIONAL REQUIREMENTS)
 - 3.4.1. Hiệu năng và Hiệu suất (Performance Efficiency)
 - 3.4.2. Độ tin cậy và Khả năng chịu lỗi (Reliability & Resilience)
 - 3.4.3. Bảo mật và An toàn thông tin (Security)
 - 3.4.4. Khả năng mở rộng (Scalability)
 - 3.4.5. Khả năng bảo trì (Maintainability)
 - 3.4.6. Tính tương thích (Compatibility)
- 4. CHƯƠNG 4. THIẾT KẾ PHẦN MỀM
 - 4.1. LƯỢC ĐỒ CHỨC NĂNG VÀ ĐẶC TẢ CHỨC NĂNG
 - 4.1.1. Lược đồ chức năng – UseCase Diagram
 - 4.1.2. Đặc tả chức năng
 - 4.2. LƯỢC ĐỒ LỚP
 - 4.3. LƯỢC ĐỒ TUẦN TỰ CỦA MỘT SỐ CHỨC NĂNG CHÍNH
 - 4.4. THIẾT KẾ GIAO DIỆN
- 5. CHƯƠNG 5. CÀI ĐẶT VÀ KIỂM THỬ
 - 5.1. LẬP TRÌNH CHỨC NĂNG
 - 5.1.1. Tìm kiếm Ngữ nghĩa (Semantic Search Implementation)
 - 5.1.2. Chatbot Thông minh với Agentic RAG
 - 5.1.3. Cơ chế Hướng sự kiện và Xử lý Bất đồng bộ (Event-Driven & Async Implementation)
 - 5.2. TỔ CHỨC MÃ NGUỒN VÀ CHIẾN LƯỢC TRIỂN KHAI
 - 5.2.1. Chiến lược và Mô hình Triển khai
 - 5.2.2. Cấu trúc Repository

- 5.2.3. Kiến trúc Container (Container Architecture)
- 5.3. QUY TRÌNH BUILD VÀ ĐÓNG GÓI (BUILD PIPELINE)
 - 5.3.1. Quy trình tích hợp và triển khai liên tục (CI/CD pipeline)
 - 5.3.2. Tự động hóa quy trình (CI/CD Automation)
 - 5.3.3. Quy trình Tích hợp và Kiểm thử Tự động (CI Workflow)
 - 5.3.4. Quy trình Triển khai (CD Workflow)
 - 5.3.5. Công cụ Quản trị Tác vụ (Automation Scripting)
- 5.4. KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG
 - 5.4.1. Chiến lược và Phạm vi kiểm thử
 - 5.4.2. Công cụ và Môi trường
 - 5.4.3. Quy trình thực thi
 - 5.4.4. Kết quả và xử lý vấn đề

PHẦN KẾT LUẬN

- 1. KẾT QUẢ ĐẠT ĐƯỢC
 - 1.1. Về lý thuyết
 - 1.2. Về ứng dụng
- 2. ƯU ĐIỂM
- 3. NHƯỢC ĐIỂM
- 4. HƯỚNG PHÁT TRIỂN
 - 4.1. Nâng cấp Trí tuệ nhân tạo
 - 4.2. Tối ưu hóa Hạ tầng Kỹ thuật
 - 4.3. Tối ưu chi phí

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN
BẢNG CHẤM ĐIỂM KHÓA LUẬN TỐT NGHIỆP

Bộ Môn :

MaSV (1)	Họ tên (1)	MaSV (2)	Họ tên (2)	MaSV (3)	Họ tên (3)
22100172	Phan Thị Mỹ Linh	22110179	Nguyễn Kiến Lợi	22110153	Vương Tài Hưng

STT	Tiêu chí	Thang điểm	Mức độ đạt được					Điểm đạt được		
			1	2	3	4	5	(1)	(2)	(3)
1	Tính thực tiễn của đề tài, sự hiểu biết về vấn đề nghiên cứu	10						9	9	9
2	Tính đúng đắn và hợp lí của phương pháp nghiên cứu, của thiết kế, của giải pháp được nêu ra trong luận văn. Mức độ hoàn thiện của sản phẩm, mức độ hoàn thành công việc của sinh viên.	40						38	38	38
3	Chất lượng của bài thuyết trình	10						10	10	10
4	Khả năng đọc sách ngoại ngữ tham khảo	5						5	5	5
5	Khả năng tổng hợp kiến thức, viết luận văn	10						9	9	9
6	Chất lượng về hình thức của luận văn (Cấu trúc, định dạng, chính tả,...)	5						5	5	5
7	Chất lượng trả lời các câu hỏi của hội đồng	20						20	20	20
8	Điểm thưởng cho đề tài có bài báo khoa học (tối đa 10/100 điểm)	10								
Tổng điểm								96	96	96

Ghi chú : mức độ đạt được chia thành 5 thang bậc, ký hiệu 1, 2, 3, 4, 5 (Sinh viên đạt mức độ nào thì GV ghi ký hiệu của SV (VD : (1)) vào ô tương ứng)

1. Không đạt (0%) 2. Yếu (20%)
3. Trung Bình (50%) 4. Khá (70%) 5. Giỏi (100%)

Tp.HCM, ngày 16. tháng 1... năm 2025
Giảng viên chấm điểm


Mai Anh Thơ

Đề nghị :

.....

.....

.....

.....

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN
BẢNG CHẤM ĐIỂM KHÓA LUẬN TỐT NGHIỆP

Bộ Môn :

MaSV (1)	Họ tên (1)	MaSV (2)	Họ tên (2)	MaSV (3)	Họ tên (3)
22110172	Phạm Thị Mỹ Linh	22110159	Vương Trúc Hương	22110179	Nguyễn Huỳnh Lê

STT	Tiêu chí	Thang điểm	Mức độ đạt được					Điểm đạt được		
			1	2	3	4	5	(1)	(2)	(3)
1	Tính thực tiễn của đề tài, sự hiểu biết về vấn đề nghiên cứu	10				x		8	8	8
2	Tính đúng đắn và hợp lí của phương pháp nghiên cứu, của thiết kế, của giải pháp được nêu ra trong luận văn. Mức độ hoàn thiện của sản phẩm, mức độ hoàn thành công việc của sinh viên.	40						35	35	36
3	Chất lượng của bài thuyết trình	10						10	10	10
4	Khả năng đọc sách ngoại ngữ tham khảo	5						4	4	4
5	Khả năng tổng hợp kiến thức, viết luận văn	10						10	10	10
6	Chất lượng về hình thức của luận văn (Cấu trúc, định dạng, chính tả,...)	5						4	4	4
7	Chất lượng trả lời các câu hỏi của hội đồng	20						18	18	18
8	Điểm thưởng cho đề tài có bài báo khoa học (tối đa 10/100 điểm)	10								
Tổng điểm								90	90	90

Ghi chú : mức độ đạt được chia thành 5 thang bậc, ký hiệu 1, 2, 3, 4, 5 (Sinh viên đạt mức độ nào thì GV ghi ký hiệu của SV (VD : (1)) vào ô tương ứng)

1. Không đạt (0%) 2. Yếu (20%)
3. Trung Bình (50%) 4. Khá (70%) 5. Giỏi (100%)

Tp.HCM, ngày 16 tháng 01 năm 2026
Giảng viên chấm điểm


Bùi Mạnh Quân

Đề nghị :

.....
.....
.....
.....

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN
BẢNG CHẤM ĐIỂM KHÓA LUẬN TỐT NGHIỆP

Bộ Môn :

MaSV (1)	Họ tên (1)	MaSV (2)	Họ tên (2)	MaSV (3)	Họ tên (3)
22110172	Phan T. Mỹ Linh	22110153	Vương Trí Hùng	22110179	Nguyễn Hữu Lộc

STT	Tiêu chí	Thang điểm	Mức độ đạt được					Điểm đạt được		
			1	2	3	4	5	(1)	(2)	(3)
1	Tính thực tiễn của đề tài, sự hiểu biết về vấn đề nghiên cứu	10						9	9	9
2	Tính đúng đắn và hợp lí của phương pháp nghiên cứu, của thiết kế, của giải pháp được nêu ra trong luận văn. Mức độ hoàn thiện của sản phẩm, mức độ hoàn thành công việc của sinh viên.	40						36	36	36
3	Chất lượng của bài thuyết trình	10						9	9	9
4	Khả năng đọc sách ngoại ngữ tham khảo	5						5	5	5
5	Khả năng tổng hợp kiến thức, viết luận văn	10						8	8	8
6	Chất lượng về hình thức của luận văn (Cấu trúc, định dạng, chính tả,...)	5						4	4	4
7	Chất lượng trả lời các câu hỏi của hội đồng	20						18	18	18
8	Điểm thưởng cho đề tài có bài báo khoa học (tối đa 10/100 điểm)	10								
Tổng điểm								89	89	89

Ghi chú : mức độ đạt được chia thành 5 thang bậc, ký hiệu 1, 2, 3, 4, 5 (Sinh viên đạt mức độ nào thì GV ghi kí hiệu của SV (VD : (1)) vào ô tương ứng)

1. Không đạt (0%) 2. Yếu (20%)
3. Trung Bình (50%) 4. Khá (70%) 5. Giỏi (100%)

Tp.HCM, ngày 16 tháng 1 năm 2024
Giảng viên chấm điểm

Vacc
N.T. Văn

Đề nghị :

.....

.....

.....

.....

PHIẾU NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN

Họ và tên Sinh viên:

- | | |
|---------------------|----------------|
| 1. Phan Thị Mỹ Linh | MSSV: 22110172 |
| 2. Vương Trí Hùng | MSSV: 22110153 |
| 3. Nguyễn Hữu Lộc | MSSV: 22110179 |

Ngành: Công nghệ Thông tin

Tên đề tài: **XÂY DỰNG NỀN TẢNG CUNG CẤP KHÓA HỌC TRỰC TUYẾN.**

Họ và tên Giảng viên phản biện: ThS. Nguyễn Trần Thị Văn

NHẬN XÉT

- Về nội dung đề tài & khối lượng thực hiện:
 - Nội dung và khối lượng đề tài phù hợp với quy mô của môn khóa luận tốt nghiệp dành cho 3 sinh viên.
- Ưu điểm:
 - Công nghệ sử dụng tiên tiến.
 - Nền tảng xây dựng cung cấp khá đa dạng các tính năng hỗ trợ.
 - Giao diện thân thiện.
 - Tiếp cận xây dựng dựa trên kiến trúc mở, cho phép nâng cấp mở rộng về sau.
- Khuyết điểm:
 - Chưa xem xét khía cạnh vận hành hệ thống trên thực tế.
 - Chưa tối ưu hóa mô hình AI áp dụng trong hệ thống.
- Đề nghị cho bảo vệ hay không? Đồng ý cho bảo vệ.
Đề xuất điều chỉnh:
 - Bổ sung đề cương của khóa luận.
 - Điều chỉnh phần kết luận theo hướng liệt kê các nội dung.
- Đánh giá loại: Giỏi.
- Điểm:

Tp. Hồ Chí Minh, ngày 14 tháng 1 năm 2026

Giảng viên phản biện

(Ký & ghi rõ họ tên)



Nguyễn Trần Thị Văn



PHIẾU NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

Họ và tên sinh viên 1: Phan Thị Mỹ Linh

MSSV 1: 22110172

Họ và tên sinh viên 2: Nguyễn Hữu Lộc

MSSV 2: 22110179

Họ và tên sinh viên 3: Vương Trí Hùng

MSSV 3: 22110153

Ngành: Công nghệ Thông tin

Tên đề tài: XÂY DỰNG NỀN TẢNG CUNG CẤP KHÓA HỌC TRỰC TUYẾN

Họ và tên giảng viên hướng dẫn: PGS.TS Hoàng Văn Dũng

NHẬN XÉT

1. Về nội dung đề tài & khối lượng thực hiện:

Khóa luận đã trình bày được những kiến thức cơ sở, nền tảng về lập trình web, quản trị cơ sở dữ liệu để thực hiện đề tài

Sinh viên đã thực hiện khảo sát hiện trạng, phân tích đánh giá được những phần mềm liên quan nhằm làm cơ sở thiết kế giải pháp, phần mềm cung cấp khóa học trực tuyến.

Thực hiện phân tích, thiết kế và cài đặt phần mềm web để thực hiện chức năng quản lý hệ thống LMS và học tập trực tuyến như quản lý khóa học, đăng ký học, quản lý giảng viên và học viên, hệ thống thanh toán, đánh giá khóa học và tích hợp AI trong chatbot tư vấn.

Sử dụng các công nghệ, kỹ thuật, thư viện lập trình có tính cập nhật.

Demo và đánh giá phần mềm.

Kết quả thực hiện của đề tài được trình bày khá rõ ràng, phù hợp.

Nội dung thực hiện và kết quả đạt được của đề tài đáp ứng tốt yêu cầu của khóa luận tốt nghiệp trình độ đại học.

2. Ưu điểm:

Sinh viên tham gia thực hiện đề tài có thái độ tốt, tích cực.

Sinh viên có kiến thức tốt, đã thực hiện các bước theo quy trình phát triển phần mềm theo hướng ứng dụng web và tiếp cận riêng biệt về ứng dụng quản lý dạy học.

Một số tính năng cao với sự hỗ trợ của AI là xu hướng tốt trong ứng dụng tư vấn hỏi đáp.

3. Khuyết điểm

Phần mềm còn tốn nhiều tài nguyên dẫn đến hệ thống web chạy hơi chậm, đặc biệt là các công cụ tìm kiếm AI. Một số chức năng nâng cao, chuyên sâu theo hướng sư phạm chưa nhiều.

4. Đề nghị cho bảo vệ hay không? Đồng ý cho bảo vệ.

5. Đánh giá loại: Giỏi.

6. Điểm: 9.0.

TP. Hồ Chí Minh, ngày 14 tháng 01 năm 2026

Giảng viên hướng dẫn

(Ký & ghi rõ họ tên)



PGS.TS. Hoàng Văn Dũng

LỜI CẢM ƠN

Lời nói đầu tiên chúng em xin gửi lời cảm ơn chân thành nhất đến Khoa Công nghệ Thông tin, Trường Đại học Công nghệ Kỹ Thuật TP. Hồ Chí Minh, đã tạo điều kiện học tập và nghiên cứu, đồng thời cung cấp môi trường lý tưởng để chúng em phát triển kiến thức và kỹ năng trong lĩnh vực Công nghệ Thông tin.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn sâu sắc đến thầy Hoàng Văn Dũng, giảng viên hướng dẫn chúng em hoàn thành Khoá luận tốt nghiệp, người đã tận tình hướng dẫn, định hướng và truyền đạt những kiến thức quý báu để chúng em hoàn thành đề tài "Xây Dựng Nền Tảng Cung Cấp Khóa Học Trực Tuyến".

Sự tận tâm và nhiệt huyết của thầy không chỉ giúp chúng em nắm bắt được các nguyên lý, kỹ thuật kiểm thử phần mềm mà còn truyền cảm hứng để chúng em nâng cao tinh thần học hỏi và sáng tạo. Những góp ý và hướng dẫn chi tiết của thầy trong suốt quá trình thực hiện đề tài đã giúp chúng em hiểu sâu sắc hơn về cách xây dựng một sản phẩm phần mềm chất lượng.

Chúng em xin chân thành cảm ơn thầy và Khoa Công nghệ Thông tin. Chúng em hy vọng sẽ tiếp tục nhận được sự hỗ trợ, định hướng từ quý thầy cô trong những chặng đường học tập và nghiên cứu tiếp theo.

Trân trọng cảm ơn!

Nhóm sinh viên thực hiện đề tài

Phan Thị Mỹ Linh

Nguyễn Hữu Lộc

Vương Trí Hùng

MỤC LỤC

DANH MỤC HÌNH ẢNH	i
DANH MỤC BẢNG	iii
DANH MỤC CÁC THUẬT NGỮ	iv
PHẦN MỞ ĐẦU	1
1. TÍNH CẤP THIẾT CỦA ĐỀ TÀI.....	1
2. MỤC ĐÍCH CỦA ĐỀ TÀI.....	3
3. CÁC TIẾP CẬN VÀ PHƯƠNG PHÁP NGHIÊN CỨU	4
3.1. Đối tượng	4
3.2. Phạm vi nghiên cứu	4
4. KẾT QUẢ DỰ KIẾN ĐẠT ĐƯỢC	4
4.1. Về kiến thức	4
4.2. Về đề tài	5
5. CÔNG NGHỆ SỬ DỤNG.....	6
PHẦN NỘI DUNG	8
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT	8
1.1. CÔNG NGHỆ PHẦN MỀM VÀ WEBSITE	8
1.1.1. Tổng quan về Website	8
1.1.2. Java Spring và Hệ sinh thái Cloud-Native.....	9
1.1.3. Next.js	10
1.2. GIỚI THIỆU VỀ VINA ACADEMY	12
1.2.1. Tổng quan về Vina Academy	12
1.2.2. Vai trò trong hệ thống.....	13
1.3. GIỚI THIỆU CÁC THÀNH PHẦN TRONG ỨNG DỤNG	14
1.3.1. Bối cảnh và Nền tảng Kế thừa	14
1.3.2. Kiến trúc Microservices (Microservices Architecture)	15
1.4. CÁC CÔNG NGHỆ LIÊN QUAN.....	19
1.4.1. Json Web Token.....	19
1.4.2. Maven	19
1.4.3. Spring Cloud Ecosystem.....	20

1.4.4.	Apache Kafka.....	20
1.4.5.	PostgreSQL & pgvector.....	20
1.4.6.	LangChain.....	20
1.4.7.	Docker.....	21
CHƯƠNG 2. PHƯƠNG PHÁP NGHIÊN CỨU.....		21
2.1.	PHƯƠNG PHÁP TIẾP CẬN.....	21
2.1.1.	Nghiên cứu tài liệu.....	21
2.1.2.	Phân tích yêu cầu.....	22
2.1.3.	Lựa chọn công nghệ.....	22
2.2.	CÔNG CỤ VÀ MÔI TRƯỜNG PHÁT TRIỂN.....	24
2.2.1.	Công cụ phát triển.....	24
2.2.2.	Công cụ kiểm thử.....	25
2.2.3.	Môi trường triển khai và Hạ tầng (Deployment & Infrastructure) ..	25
CHƯƠNG 3. KHẢO SÁT HIỆN TRẠNG VÀ MÔ HÌNH HÓA YÊU CẦU.....		27
3.1.	KHẢO SÁT HIỆN TRẠNG.....	27
3.2.	KHẢO SÁT MỘT SỐ WEBSITE.....	28
3.2.1.	Ucademy (https://www.udemy.com/).....	28
3.2.2.	Coursera (https://www.coursera.org/).....	29
3.2.3.	edX (https://www.edx.org/).....	30
3.3.	YÊU CẦU CHỨC NĂNG.....	31
3.3.1.	Người dùng chưa đăng ký (Unregistered User).....	31
3.3.2.	Học viên (Student).....	32
3.3.3.	Giảng viên (Instructor).....	32
3.3.4.	Ban quản trị (Admin/Owner).....	32
3.3.5.	Nhân viên (Staff).....	33
3.4.	YÊU CẦU PHI CHỨC NĂNG (NON-FUNCTIONAL REQUIREMENTS).....	33
3.4.1.	Hiệu năng và Hiệu suất (Performance Efficiency).....	33
3.4.2.	Độ tin cậy và Khả năng chịu lỗi (Reliability & Resilience).....	33
3.4.3.	Bảo mật và An toàn thông tin (Security).....	34

3.4.4.	Khả năng mở rộng (Scalability).....	34
3.4.5.	Khả năng bảo trì (Maintainability)	34
3.4.6.	Tính tương thích (Compatibility).....	35
CHƯƠNG 4. THIẾT KẾ PHẦN MỀM.....		36
4.1.	LƯỢC ĐỒ CHỨC NĂNG VÀ ĐẶC TẢ CHỨC NĂNG	36
4.1.1.	Lược đồ chức năng – UseCase Diagram	36
4.1.2.	Đặc tả chức năng.....	36
4.2.	LƯỢC ĐỒ LỚP.....	61
4.3.	LƯỢC ĐỒ TUẦN TỰ CỦA MỘT SỐ CHỨC NĂNG CHÍNH	65
4.3.1.	Lược đồ tuần tự luồng Tạo Quiz tự động	65
4.3.2.	Lược đồ tuần tự luồng Học viên làm và nộp bài	66
4.3.3.	Lược đồ tuần tự luồng Tạo Quiz thủ công.....	67
4.3.4.	Lược đồ tuần tự luồng Tìm kiếm truyền thống.....	67
4.3.5.	Lược đồ tuần tự luồng Tìm kiếm AI nâng cao	68
4.3.6.	Lược đồ tuần tự luồng Học viên hỏi Chatbot về nội dung bài đang học 69	
4.3.7.	Lược đồ tuần tự luồng Chatbot gợi ý Khoá học	70
4.3.8.	Lược đồ tuần tự luồng Luồng Tạo Bình Luận & Kiểm Duyệt Tự Động 71	
4.3.9.	Lược đồ tuần tự luồng Lấy danh sách bình luận.....	72
4.3.10.	Lược đồ tuần tự luồng Quản Lý Kiểm Duyệt (Admin/Staff)	73
4.3.11.	Lược đồ tuần tự luồng Xử lý đánh giá.....	74
4.3.12.	Lược đồ tuần tự luồng Xử lý thanh toán – VNPay	75
4.4.	THIẾT KẾ GIAO DIỆN.....	76
4.4.1.	Trang chủ	76
4.4.2.	Đăng ký.....	77
4.4.3.	Đăng nhập	77
4.4.4.	Đăng ký làm giảng viên	78
4.4.5.	Tìm kiếm khóa học	79
4.4.6.	Quản lý khóa học giảng dạy	79
4.4.7.	Hồ sơ	81

4.4.8.	Giỏ hàng.....	82
4.4.9.	Học	83
CHƯƠNG 5. CÀI ĐẶT VÀ KIỂM THỬ		85
5.1.	LẬP TRÌNH CHỨC NĂNG.....	85
5.1.1.	Tìm kiếm Ngữ nghĩa (Semantic Search Implementation)	85
5.1.2.	Chatbot Thông minh với Agentic RAG.....	88
5.1.3.	Cơ chế Hướng sự kiện và Xử lý Bất đồng bộ (Event-Driven & Async Implementation)	93
5.2.	TỔ CHỨC MÃ NGUỒN VÀ CHIẾN LƯỢC TRIỂN KHAI.....	95
5.2.1.	Chiến lược và Mô hình Triển khai.....	95
5.2.2.	Cấu trúc Repository	97
5.2.3.	Kiến trúc Container (Container Architecture)	98
5.3.	QUY TRÌNH BUILD VÀ ĐÓNG GÓI (BUILD PIPELINE)	99
5.3.1.	Quy trình tích hợp và triển khai liên tục (CI/CD pipeline).....	99
5.3.2.	Tự động hóa quy trình (CI/CD Automation)	102
5.3.3.	Quy trình Tích hợp và Kiểm thử Tự động (CI Workflow).....	102
5.3.4.	Quy trình Triển khai (CD Workflow).....	104
5.3.5.	Công cụ Quản trị Tác vụ (Automation Scripting)	105
5.4.	KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG.....	108
5.4.1.	Chiến lược và Phạm vi kiểm thử	108
5.4.2.	Công cụ và Môi trường	108
5.4.3.	Quy trình thực thi.....	109
5.4.4.	Kết quả và xử lý vấn đề	109
PHẦN KẾT LUẬN.....		111
1.	KẾT QUẢ ĐẠT ĐƯỢC.....	111
1.1.	Về lý thuyết.....	111
1.2.	Về ứng dụng.....	111
2.	ƯU ĐIỂM.....	111
3.	NHUỘC ĐIỂM	112
4.	HƯỚNG PHÁT TRIỂN	112

4.1.	Nâng cấp Trí tuệ nhân tạo	112
4.2.	Tối ưu hóa Hạ tầng Kỹ thuật.....	112
4.3.	Tối ưu chi phí.....	113
TÀI LIỆU THAM KHẢO		114
PHỤ LỤC		116

DANH MỤC HÌNH ẢNH

Hình 1. Kiến trúc của Spring Framework.....	9
Hình 2. Cấu tạo của thư viện NextJS.....	11
Hình 3. Mô hình 3 lớp.....	14
Hình 4. Kiến trúc MVC	15
Hình 5. Kiến trúc Microservice của hệ thống Vina Academy.....	16
Hình 6. Trang chủ website Ucademy	28
Hình 7. Trang chủ website Coursera	29
Hình 8. Trang chủ website edX	30
Hình 9. Lược đồ chức năng	36
Hình 10. Class Diagram User Domain	61
Hình 11. Class Diagram Course Domain	62
Hình 12. Class Diagram Payment Domain.....	63
Hình 13. Class Diagram Chat Domain	64
Hình 14. Lược đồ tuần tự Luồng tạo Quiz Tự động.....	65
Hình 15. Lược đồ tuần tự luồng Học viên làm và nộp bài	66
Hình 16. Lược đồ tuần tự luồng Tạo Quiz thủ công.....	67
Hình 17. Lược đồ tuần tự luồng Tìm kiếm truyền thống	67
Hình 18. Lược đồ tuần tự chức năng luồng Tìm kiếm AI nâng cao.....	68
Hình 19. Lược đồ tuần tự chức năng luồng Chatbot hỗ trợ học tập	69
Hình 20. Lược đồ tuần tự luồng Chatbot gợi ý Khoá học	70
Hình 21. Lược đồ tuần tự luồng luồng Tạo bình luận & kiểm duyệt tự động.....	71
Hình 22. Lược đồ tuần tự luồng Lấy Danh Sách Bình Luận.....	72
Hình 23. Lược đồ tuần tự luồng Quản Lý Kiểm Duyệt (Admin/Staff)	73
Hình 24. Lược đồ tuần tự luồng Xử lý đánh giá.....	74
Hình 25. Lược đồ tuần tự luồng Xử lý thanh toán - VNPay	75
Hình 26. Giao diện trang chủ.....	76
Hình 27. Giao diện đăng ký	77
Hình 28. Giao diện đăng nhập	77

Hình 29. Giao diện đăng ký làm giảng viên	78
Hình 30. Giao diện tìm kiếm khóa học	79
Hình 31. Giao diện Xem danh sách các khóa học giảng dạy	79
Hình 32. Giao diện Quản lý học viên	80
Hình 33. Giao diện Xem hồ sơ của Giảng viên	81
Hình 34. Giao diện chỉnh sửa Hồ sơ cá nhân	82
Hình 35. Giao diện giỏ hàng	82
Hình 36. Giao diện Xem nội dung khóa học	83
Hình 37. Giao diện Học - Video	83
Hình 38. Giao diện Học – Làm Trắc nghiệm	84
Hình 39. Kiến trúc Triển khai Vật lý (Physical Deployment Architecture)	96
Hình 40. Deployment Pipeline Diagram	106

DANH MỤC BẢNG

Bảng 1. Đặc tả Usecase Search Course - Tìm kiếm khóa học.....	38
Bảng 2. Đặc tả Usecase View Course Detail - Xem Chi tiết Khóa học	38
Bảng 3. Đặc tả Usecase Register - Đăng ký	39
Bảng 4. Đặc tả Usecase Verify Email - Xác thực Email	40
Bảng 5. Đặc tả Usecase Log in- Đăng nhập	41
Bảng 6. Đặc tả Usecase Verify Account - Xác nhận tài khoản	42
Bảng 7. Đặc tả Usecase Forgot Password - Quên mật khẩu.....	43
Bảng 8. Đặc tả Usecase Log out - Đăng xuất	43
Bảng 9. Đặc tả Usecase View Notifications - Xem thông báo.....	44
Bảng 10. Đặc tả Usecase View Available Course - Xem khóa học đã đăng ký ..	45
Bảng 11. Đặc tả Usecase Learn Lectures - Học	45
Bảng 12. Đặc tả Usecase Discussion - Thảo luận	47
Bảng 13. Đặc tả Usecase Rate Course - Đánh giá khóa học	48
Bảng 14. Đặc tả Usecase Do Quiz - Làm trắc nghiệm đánh giá	50
Bảng 15. Đặc tả Usecase Download Attachments - Tải xuống tài liệu học tập ...	52
Bảng 16. Đặc tả Usecase Take Note - Ghi chú.....	55
Bảng 17. Đặc tả Usecase Manage Profile - Quản lý thông tin cá nhân.....	56
Bảng 18. Đặc tả Usecase Manage Cart - Quản lý giỏ hàng.....	57
Bảng 19. Đặc tả Usecase Cart Payment - Thanh toán giỏ hàng	57
Bảng 20. Đặc tả Usecase Quản lý khóa học giảng dạy	58
Bảng 21. Đặc tả Usecase View Course Revenue - Xem doanh thu khóa học.....	59
Bảng 22. Đặc tả Usecase Review Content Courses - Kiểm duyệt khóa học	60
Bảng 23. Đặc tả Usecase Manage Payments - Quản lý giao dịch	60

DANH MỤC CÁC THUẬT NGỮ

Thuật ngữ	Viết tắt	Tiếng Anh	Định nghĩa
Trí tuệ nhân tạo tạo sinh	GenAI	Generative AI	Tạo nội dung mới từ dữ liệu huấn luyện.
Thế hệ tăng cường truy xuất	RAG	Retrieval-Augmented Generation	Kỹ thuật tối ưu LLM bằng dữ liệu ngoài (Vector DB).
Mô hình ngôn ngữ lớn	LLM	Large Language Model	Mô hình AI hiểu và sinh ngôn ngữ tự nhiên.
Cơ sở dữ liệu Vector	-	Vector Database	Lưu trữ và truy vấn vector embeddings.
Nhúng Vector	-	Vector Embeddings	Biểu diễn dữ liệu dưới dạng vector số học.
Tìm kiếm ngữ nghĩa	-	Semantic Search	Tìm kiếm dựa trên ý nghĩa/ngữ cảnh.
Kiến trúc hướng sự kiện	EDA	Event-Driven Architecture	Giao tiếp bất đồng bộ qua sự kiện.
Môi giới tin nhắn	-	Message Broker	Trung gian trao đổi tin nhắn (Kafka).
Kiến trúc Vi dịch vụ	-	Microservices	Chia nhỏ ứng dụng thành các dịch vụ độc lập.
Container hóa	-	Containerization	Đóng gói ứng dụng (Docker).
Điều phối dịch vụ	-	Orchestration	Quản lý vòng đời container (Docker Compose).
Tích hợp/Triển khai liên tục	-	Continuous Integration/Deployment	Tự động hóa quy trình phát triển phần mềm.

Lập trình hướng khía cạnh	AOP	Aspect-Oriented Programming	Kỹ thuật tách biệt các chức năng chung như logging, transaction management
Mạng phân phối nội dung	CDN	Content Delivery Network	Mạng phân phối nội dung để tăng tốc độ tải trang
Tích hợp/Triển khai liên tục	CI/CD	Continuous Integration/Continuous Deployment	Phương pháp tích hợp liên tục và triển khai liên tục
Chia nhỏ mã nguồn	-	Code Splitting	Kỹ thuật chia nhỏ mã nguồn để tối ưu tốc độ tải
Chia sẻ tài nguyên cross-origin	CORS	Cross-Origin Resource Sharing	Cơ chế cho phép tài nguyên trên một trang web được truy cập từ domain khác
Tấn công yêu cầu cross-site	CSRF	Cross-Site Request Forgery	Loại tấn công bảo mật web
Kết xuất phía client	CSR	Client-Side Rendering	Phương pháp kết xuất nội dung trang web trên trình duyệt
Nền tảng container	-	Docker	Nền tảng containerization để đóng gói ứng dụng
Framework ORM Java	-	Hibernate	Framework Object-Relational Mapping cho Java
Đảo ngược điều khiển	IoC	Inversion of Control	Nguyên lý thiết kế trong Spring Framework
Tái tạo tĩnh tăng dần	ISR	Incremental Static Regeneration	Kỹ thuật tái tạo tĩnh tăng dần trong Next.js

API lưu trữ Java	JPA	Java Persistence API	API lưu trữ Java để làm việc với cơ sở dữ liệu
Token web JSON	JWT	JSON Web Token	Tiêu chuẩn mở để truyền thông tin an toàn dưới dạng JSON
Tải lazy	-	Lazy Loading	Kỹ thuật tải dữ liệu khi cần thiết để tối ưu hiệu suất
Kiến trúc vi dịch vụ	-	Microservices	Kiến trúc chia nhỏ ứng dụng thành các dịch vụ độc lập
Mô hình kiến trúc	MVC	Model-View-Controller	Mô hình kiến trúc phần mềm chia thành 3 thành phần
Ánh xạ đối tượng-quan hệ	ORM	Object-Relational Mapping	Kỹ thuật ánh xạ đối tượng-quan hệ
API RESTful	REST	Representational State Transfer	Kiểu thiết kế API tuân thủ nguyên tắc REST
Thiết kế responsive	-	Responsive Design	Giao diện tự động điều chỉnh theo kích thước màn hình
Phân chia dữ liệu	-	Sharding	Kỹ thuật phân chia cơ sở dữ liệu theo chiều ngang
Nguyên tắc thiết kế	SOLID	SOLID Principles	Tập hợp 5 nguyên tắc thiết kế phần mềm hướng đối tượng
Framework Spring	-	Spring Boot	Framework Java để phát triển ứng dụng web nhanh chóng

Công cụ microservices	-	Spring Cloud	Bộ công cụ xây dựng microservices trong Spring
Thư viện Spring JPA	-	Spring Data JPA	Thư viện Spring đơn giản hóa truy cập dữ liệu
Framework bảo mật	-	Spring Security	Framework bảo mật cho ứng dụng Spring
Tạo trang tĩnh	SSG	Static Site Generation	Phương pháp tạo trang web tĩnh
Kết xuất phía server	SSR	Server-Side Rendering	Phương pháp kết xuất nội dung trên server
Framework CSS	-	Tailwind CSS	Framework CSS utility-first
Cổng thanh toán	-	VNPay	Cổng thanh toán trực tuyến tại Việt Nam
Giao thức WebSocket	-	WebSocket	Giao thức truyền thông hai chiều giữa client và server

PHẦN MỞ ĐẦU

1. TÍNH CẤP THIẾT CỦA ĐỀ TÀI

Trong kỷ nguyên số hiện nay, sự phát triển mạnh mẽ của khoa học – công nghệ đã làm thay đổi sâu sắc cách thức con người tiếp cận tri thức. Internet mở ra một kho tàng kiến thức khổng lồ, giúp cho việc học tập, cập nhật thông tin mới trở nên dễ dàng và nhanh chóng hơn bao giờ hết. Nhu cầu học tập suốt đời vì vậy cũng trở nên thiết yếu, không chỉ để thích nghi với sự biến đổi không ngừng của xã hội mà còn để mỗi cá nhân tự hoàn thiện bản thân trong thế giới ngày càng cạnh tranh. Tuy nhiên, bên cạnh những cơ hội rộng mở, sự bùng nổ thông tin trên không gian mạng cũng đặt ra nhiều thách thức lớn. Lượng thông tin tràn lan, thiếu kiểm chứng khiến người học dễ đi lạc hướng, tiếp thu sai lệch, dẫn đến kết quả học tập kém hiệu quả. Phương pháp học tự do nếu thiếu định hướng bài bản còn khiến học viên dễ rơi vào tình trạng mất phương hướng, tiêu tốn thời gian và công sức mà không đạt được mục tiêu như mong muốn.

Trong bối cảnh đó, việc tìm kiếm một nền tảng học tập chất lượng, phù hợp với nhu cầu cá nhân trở thành một bài toán nan giải đối với nhiều người. Học viên không chỉ cần những khóa học có nội dung đáng tin cậy, phương pháp giảng dạy hiệu quả, chi phí hợp lý, mà còn rất cần sự đồng hành, hướng dẫn từ những người có kinh nghiệm để nâng cao hiệu quả học tập. Việc thiếu các yếu tố hỗ trợ này dẫn đến tâm lý hoang mang, dễ nản chí trong quá trình học, đặc biệt là với những lĩnh vực đòi hỏi tư duy hệ thống và lộ trình phát triển dài hạn.

Hiện nay, trên thị trường quốc tế đã xuất hiện nhiều nền tảng học tập trực tuyến như Udemy, Coursera, edX hay Khan Academy. Các nền tảng này sở hữu nhiều ưu điểm như khóa học đa dạng, đội ngũ giảng viên chất lượng, nội dung được đầu tư bài bản và chứng chỉ có giá trị toàn cầu. Tuy nhiên, đối với người học tại Việt Nam, việc tiếp cận các nền tảng này còn gặp không ít trở ngại. Trước hết, chi phí học tập tại các nền tảng quốc tế, dù được đánh giá là hợp lý ở các nước phát triển, vẫn còn khá cao so với mặt bằng thu nhập của đa số người Việt. Ngoài ra, rào cản ngôn ngữ cũng là một vấn đề lớn, bởi phần lớn các khóa học được giảng dạy bằng tiếng Anh, yêu cầu người học phải có trình độ ngoại ngữ

nhất định để theo kịp. Bên cạnh đó, phương pháp giảng dạy thiên về phong cách phương Tây đôi khi không phù hợp với cách tiếp thu kiến thức và tư duy học tập của người Việt Nam. Các nền tảng này cũng chưa thực sự cá nhân hóa nội dung theo nhu cầu và bối cảnh thực tế trong nước, khiến cho việc áp dụng kiến thức vào thực tiễn Việt Nam trở nên khó khăn hơn.

Tại thị trường nội địa, một số nền tảng học trực tuyến như Topica, Kyna, Edumall đã xuất hiện nhằm đáp ứng nhu cầu học tập của người Việt. Những nền tảng này đã có những bước phát triển đáng ghi nhận, với chi phí hợp lý hơn và nội dung tiếng Việt phù hợp hơn với người học trong nước. Tuy nhiên, chất lượng khóa học còn chưa đồng đều, một số nội dung chưa được đầu tư sâu, thiếu sự kiểm soát chặt chẽ về chuyên môn. Phương pháp giảng dạy chủ yếu dưới dạng video quay sẵn, thiếu tính tương tác, khiến học viên khó duy trì động lực học tập trong thời gian dài. Thêm vào đó, các nền tảng này chưa thực sự xây dựng được hệ sinh thái học tập bài bản, có sự liên thông và định hướng rõ ràng giữa các khóa học nhằm hỗ trợ người học phát triển toàn diện.

Từ thực tế đó, có thể thấy rằng nhu cầu xây dựng một nền tảng học tập trực tuyến dành riêng cho người Việt Nam là hết sức cấp thiết. Một nền tảng như vậy cần được thiết kế với nội dung bài bản, cập nhật liên tục theo xu hướng trong nước và quốc tế, phương pháp giảng dạy linh hoạt, đề cao tính tương tác và trải nghiệm thực tiễn. Chi phí phải phù hợp với điều kiện kinh tế của đa số người học Việt Nam, đồng thời cần có sự tham gia hướng dẫn, cố vấn từ các chuyên gia trong nước để tạo nên sự tin cậy và tính ứng dụng cao cho nội dung học tập. Bên cạnh đó, nền tảng cũng cần phát triển các chương trình học cá nhân hóa, giúp mỗi học viên xây dựng lộ trình học phù hợp với mục tiêu và năng lực riêng.

Một hệ thống như vậy không chỉ giúp học viên Việt Nam tiếp cận tri thức một cách bài bản, có định hướng và hiệu quả hơn trong thời đại số, mà còn tạo ra một môi trường học tập chuyên nghiệp, nơi giảng viên có thể chia sẻ kiến thức và tạo ra thu nhập bền vững. Xa hơn, nó sẽ góp phần thúc đẩy sự phát triển của nền giáo dục trực tuyến Việt Nam, đóng góp vào sự nâng cao dân trí, phát triển

nguồn nhân lực chất lượng cao, từ đó phục vụ cho sự phát triển bền vững của đất nước trong kỷ nguyên hội nhập toàn cầu.

2. MỤC ĐÍCH CỦA ĐỀ TÀI

Đề tài được thực hiện nhằm xây dựng một nền tảng học tập trực tuyến dành riêng cho người Việt, giúp kết nối giảng viên và học viên một cách hiệu quả. Nền tảng này sẽ cung cấp các khóa học đa dạng thuộc nhiều lĩnh vực như khoa học, kinh tế, xã hội, công nghệ, ... với nội dung giảng dạy phù hợp với tư duy tiếp cận của người Việt, đồng thời có chi phí hợp lý để đáp ứng nhu cầu của đa số học viên trong nước.

Ngoài ra, nền tảng đóng vai trò là cầu nối giữa giảng viên và học viên, cho phép giảng viên đăng tải khóa học và tạo ra nguồn thu nhập bền vững, trong khi học viên có thể dễ dàng tìm kiếm, đăng ký và tham gia học một cách tiện lợi. Hệ thống sẽ tích hợp tính năng đánh giá và phản hồi, giúp học viên lựa chọn khóa học chất lượng và hỗ trợ giảng viên trong việc cải thiện nội dung giảng dạy.

Bên cạnh đó, để đảm bảo chất lượng và tính định hướng trong học tập, nền tảng sẽ có cơ chế kiểm duyệt nội dung chặt chẽ, cung cấp các tài liệu hỗ trợ và hỗ trợ học viên qua các công cụ hướng dẫn cũng như tư vấn từ chuyên gia. Đồng thời, hệ thống sẽ gợi ý lộ trình học tập phù hợp với từng học viên nhằm tối ưu hóa quá trình tiếp thu kiến thức.

Không chỉ dừng lại ở việc kết nối giảng viên và học viên, nền tảng này còn ứng dụng công nghệ để tối ưu trải nghiệm học tập, bao gồm giao diện thân thiện, video tương tác, hệ thống bài tập và diễn đàn trao đổi. Học viên cũng có thể theo dõi tiến độ học tập của mình, từ đó duy trì động lực và nâng cao hiệu quả học tập.

Với những mục tiêu này, đề tài hướng đến việc phát triển một mô hình giáo dục trực tuyến hiện đại, góp phần nâng cao chất lượng đào tạo và giúp người Việt tiếp cận tri thức một cách bài bản, có định hướng, từ đó phát triển bản thân và nâng cao năng lực cạnh tranh trong thời đại số.

3. CÁC TIẾP CẬN VÀ PHƯƠNG PHÁP NGHIÊN CỨU

3.1. Đối tượng

Đối tượng nghiên cứu của đề tài bao gồm những cá nhân có nhu cầu học tập thông qua Internet, học linh hoạt, mọi lúc mọi nơi, những người có nhu cầu giảng dạy, truyền đạt kiến thức nhằm mục đích chia sẻ, thiết lập thu nhập bền vững.

3.2. Phạm vi nghiên cứu

Phạm vi nghiên cứu tập trung vào việc xây dựng nền tảng MOOC với bốn vai trò người dùng: học viên, giảng viên, nhân viên và chủ nền tảng. Hệ thống cung cấp các chức năng tìm kiếm, đăng ký, quản lý khóa học, thanh toán và phản hồi. Cho chủ nền tảng, hệ thống hỗ trợ quản lý danh mục, người dùng, đơn hàng, khuyến mãi và báo cáo doanh thu. Nhân viên hỗ trợ giải đáp thắc mắc, hỗ trợ kỹ thuật, quản lý nội dung và thu thập phản hồi. Website có giao diện hiện đại, trực quan và tối giản.

4. KẾT QUẢ DỰ KIẾN ĐẠT ĐƯỢC

4.1. Về kiến thức

- Nắm vững kiến trúc Microservices: Hiểu rõ và áp dụng thành công mô hình Microservices Architecture. Nắm vững cách các service tương tác với nhau thông qua API Gateway (Spring Cloud Gateway) và cơ chế khám phá dịch vụ (Eureka Server).
- Chiến lược hiện đại hóa hệ thống: Vận dụng được kỹ thuật đóng gói (package) một nền tảng Legacy (MVC/Monolith) thành một service độc lập, cho phép hệ thống cũ và các service mới cùng tồn tại và giao tiếp với nhau.
- Kiến trúc hướng sự kiện (EDA): Hiểu và triển khai hệ thống hàng đợi tin nhắn (Apache Kafka) để giao tiếp bất đồng bộ (asynchronous) giữa các service, tăng khả năng chịu lỗi và tách rời nghiệp vụ (decoupling).
- Giao tiếp đa phương thức: Xây dựng và tối ưu API bằng nhiều phương thức: RESTful (cho Frontend), gRPC (cho giao tiếp nội bộ service-to-service hiệu năng cao), và WebSocket (cho chat/notification thời gian thực).

- Ứng dụng AI/NLP chuyên sâu: Nghiên cứu và xây dựng thành công dịch vụ AI (Chatbot) dựa trên mô hình RAG (Retrieval-Augmented Generation), sử dụng Python (FastAPI), LangChain, và cơ sở dữ liệu vector pgvector.
- Vận hành & Triển khai (DevOps): Thành thạo việc Docker hóa (Containerize) toàn bộ các thành phần (Java, Python, Kafka, Redis, PostgreSQL) và quản lý triển khai đồng bộ qua Docker Compose.
- Công nghệ Frontend hiện đại: Hiểu rõ Framework Next.js (React) để xây dựng giao diện tối ưu (SSR/ISR), quản lý state phức tạp và tích hợp các kết nối thời gian thực

4.2. Về đề tài

Xây dựng và triển khai thành công một hệ sinh thái học tập trực tuyến ViNA Academy hoàn chỉnh, bao gồm các dịch vụ độc lập có khả năng mở rộng:

- Đối tượng chính: Học viên, Giảng viên, Nhân viên hỗ trợ, Quản trị viên.
- Thiết kế cơ sở dữ liệu lưu trữ thông tin khóa học, người dùng, nội dung học tập.
- Chức năng chính:
 - Học viên: Đăng ký tài khoản, tìm kiếm khóa học, xem video, làm bài tập, nhận chứng chỉ.
 - Giảng viên: Tạo hồ sơ, tạo khóa học, quản lý học viên, theo dõi doanh thu.
 - Nhân viên: Hỗ trợ người dùng, kiểm duyệt nội dung, xử lý hoàn trả.
 - Quản trị viên: Quản lý người dùng, phê duyệt khóa học, thiết lập chính sách giá, quản lý thanh toán.
- Hệ thống điều phối: API Gateway và Eureka Server quản lý luồng truy cập và khám phá dịch vụ.
- Core Learning Service: Xử lý các chức năng cốt lõi: Quản lý khóa học, người dùng, giảng viên, thanh toán (VNPay), và bài kiểm tra .
- Service Chatbot (AI Service): Cung cấp chatbot thông minh (dùng LangChain & pgvector) để giải đáp thắc mắc của học viên dựa trên nội dung khóa học.
- Notification Service: Gửi thông báo đa kênh (Email, WebSocket) thông qua hàng đợi Kafka.

- Chat Service: Cung cấp tính năng chat thời gian thực giữa học viên và giảng viên qua WebSocket/STOMP.
- Text-to-Speech: Tích hợp Azure Speech để chuyển đổi văn bản trong tài liệu học tập thành giọng nói.
- Hạ tầng dữ liệu: Đảm bảo hiệu suất cao thông qua Redis Caching và lưu trữ file/video qua MinIO (S3).

5. CÔNG NGHỆ SỬ DỤNG

- Nền tảng Giao diện (Frontend Ecosystem)
 - Next.js 14+ (React Framework): Sử dụng kiến trúc App Router, hỗ trợ Server-Side Rendering (SSR) để tối ưu SEO và Client-Side Rendering (CSR) cho trải nghiệm người dùng mượt mà.
 - TypeScript: Ngôn ngữ chính giúp kiểm soát chặt chẽ kiểu dữ liệu, giảm thiểu lỗi runtime.
 - Tailwind CSS & Shadcn/ui: Hệ thống Design System hiện đại giúp xây dựng giao diện nhanh chóng, nhất quán và tối ưu hiệu suất (Utility-first CSS).
- Nền tảng Dịch vụ & Microservices (Backend Ecosystem) Hệ thống áp dụng chiến lược Polyglot Programming (Đa ngôn ngữ) để tối ưu hóa cho từng loại tác vụ:
 - Java 17 & Spring Boot 3: Nền tảng chính cho các dịch vụ nghiệp vụ cốt lõi (Core, User, Payment) nhờ tính ổn định và bảo mật cao.
 - Spring Cloud: Bộ công cụ xây dựng kiến trúc phân tán:
 - Spring Cloud Gateway: API Gateway quản lý định tuyến và bảo mật.
 - Spring Cloud Netflix Eureka: Service Discovery.
 - Python (FastAPI): Sử dụng cho các dịch vụ AI nhờ khả năng xử lý tính toán và hệ sinh thái thư viện AI phong phú.
 - Go (Golang): Sử dụng cho các dịch vụ tiện ích yêu cầu hiệu năng cao và độ trễ thấp (như xử lý ngôn ngữ).
- Trí tuệ nhân tạo & Xử lý dữ liệu (AI & Data Engineering)

- LangChain: Framework điều phối các mô hình ngôn ngữ lớn (LLM), đóng vai trò xương sống cho tính năng Chatbot RAG.
- pgvector: Tiện ích mở rộng của PostgreSQL, biến CSDL quan hệ thành Vector Database để thực hiện tìm kiếm ngữ nghĩa (Semantic Search).
- Azure Cognitive Services: API chuyển đổi văn bản thành giọng nói (Text-to-Speech) chất lượng cao.
- Hạ tầng Dữ liệu & Sự kiện (Data & Event Bus)
 - PostgreSQL: Hệ quản trị CSDL quan hệ chính, đảm bảo tính toàn vẹn dữ liệu (ACID).
 - Redis: Bộ nhớ đệm (In-memory Data Store) dùng để caching dữ liệu nóng và quản lý phiên đăng nhập phân tán.
 - Apache Kafka & Zookeeper: Hệ thống Message Broker hiệu năng cao, đóng vai trò trục xương sống cho kiến trúc hướng sự kiện (EDA), xử lý giao tiếp bất đồng bộ giữa Chat và Notification.
 - MinIO (S3 Compatible): Hệ thống lưu trữ đối tượng (Object Storage) tự host để lưu trữ video bài giảng và tài liệu.
- Vận hành & Tự động hóa (DevOps & Infrastructure)
 - Docker & Docker Compose: Nền tảng Container hóa giúp đóng gói và điều phối toàn bộ hệ sinh thái dịch vụ.
 - GitHub Actions: Công cụ CI (Continuous Integration) để tự động hóa quy trình kiểm thử và đóng gói.
 - PowerShell Scripting: Các kịch bản tự động hóa (Automation Scripts) phục vụ quy trình triển khai (CD) nhanh chóng lên máy chủ.

PHẦN NỘI DUNG

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

1.1. CÔNG NGHỆ PHẦN MỀM VÀ WEBSITE

1.1.1. Tổng quan về Website

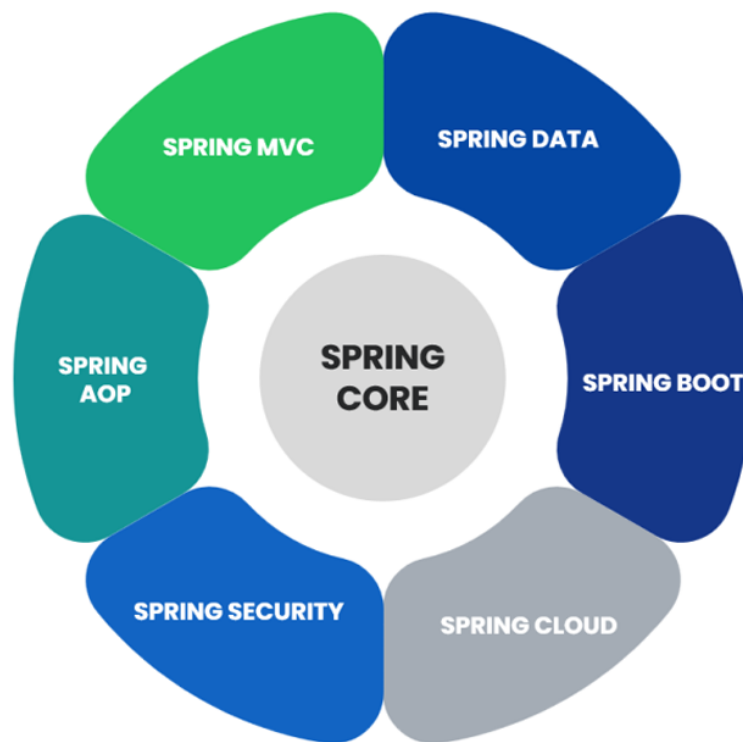
Website được định nghĩa là tập hợp các trang thông tin điện tử chứa nội dung đa phương tiện, được lưu trữ trên máy chủ web và truy cập thông qua giao thức HTTP/HTTPS bằng trình duyệt. Ưu điểm cốt lõi của nền tảng web so với ứng dụng di động (Native App) nằm ở khả năng tương thích đa nền tảng (Cross-platform capability), cho phép vận hành trên mọi thiết bị từ máy tính cá nhân đến thiết bị di động mà không yêu cầu quy trình cài đặt phức tạp.

Dựa trên kiến trúc kỹ thuật và mức độ tương tác người dùng, sự phát triển của công nghệ web có thể được phân loại thành ba giai đoạn chính. Đầu tiên là Website tĩnh (Static Website), loại hình cơ bản nhất được xây dựng thuần túy bằng HTML, CSS và JavaScript. Đặc trưng của mô hình này là nội dung cố định, không có sự giao tiếp với cơ sở dữ liệu, thường chỉ phục vụ mục đích cung cấp thông tin một chiều như trang giới thiệu doanh nghiệp (Landing Page). Tiếp theo là sự ra đời của Website động (Dynamic Website), đánh dấu bước chuyển mình quan trọng với khả năng xử lý logic phía máy chủ (Server-side) và truy xuất dữ liệu động. Sử dụng các ngôn ngữ như Java, PHP hay Python, website động cho phép cá nhân hóa nội dung và hỗ trợ các tương tác phức tạp hơn như thương mại điện tử hay mạng xã hội.

Ở cấp độ phát triển cao nhất hiện nay là Ứng dụng Web (Web Application), không chỉ hiển thị thông tin mà còn hoạt động như một phần mềm hoàn chỉnh trên trình duyệt, cung cấp trải nghiệm người dùng (UX) mượt mà tiệm cận với ứng dụng cài đặt (Desktop/Mobile App). Bằng việc ứng dụng các Framework hiện đại như ReactJS hay Next.js kết hợp với kiến trúc Backend mạnh mẽ (Microservices), Web App có khả năng xử lý các tác vụ nghiệp vụ phức tạp, cập nhật dữ liệu thời gian thực (Real-time) và duy trì trạng thái phiên làm việc (State Management) ổn định.

1.1.2. Java Spring và Hệ sinh thái Cloud-Native

Trong bối cảnh phát triển phần mềm hiện đại, đặc biệt là các hệ thống phân tán quy mô lớn, Spring Framework (cụ thể là hệ sinh thái Spring Boot và Spring Cloud) được lựa chọn làm nền tảng công nghệ chủ đạo cho các dịch vụ nghiệp vụ cốt lõi (Core Services). Không chỉ đơn thuần là một công cụ lập trình, Spring cung cấp một kiến trúc toàn diện cho việc xây dựng, triển khai và vận hành các ứng dụng Cloud-Native.



Hình 1. Kiến trúc của Spring Framework

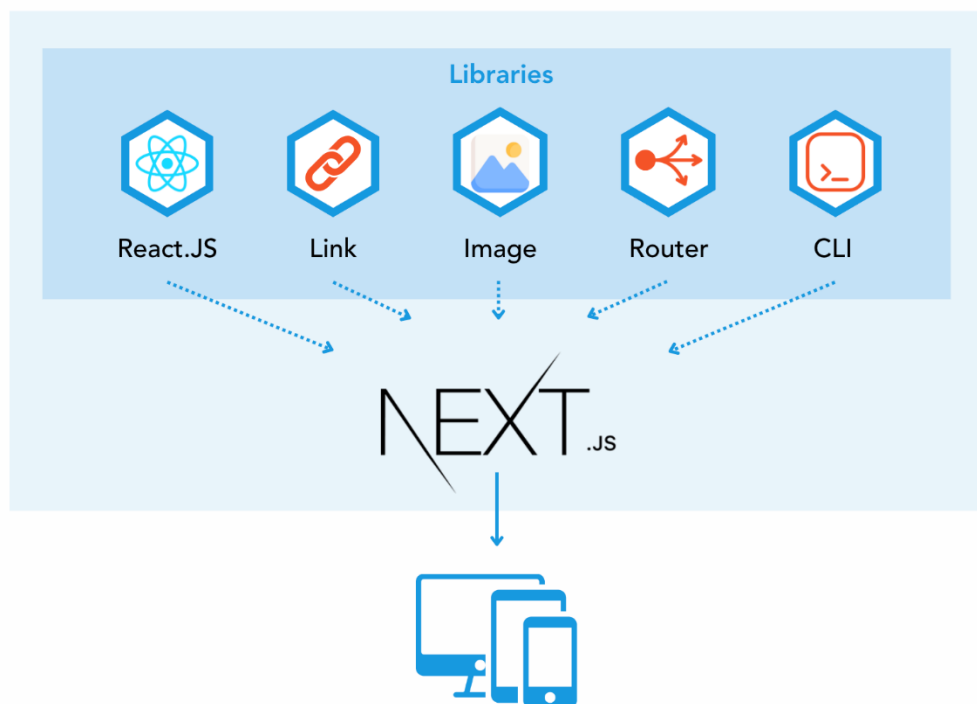
Để giải quyết bài toán vận hành các vi dịch vụ độc lập, hệ thống sử dụng Spring Boot 3.x như một tiêu chuẩn phát triển. Với cơ chế "Convention over Configuration" (Quy ước hơn cấu hình) và máy chủ nhúng (Embedded Server) như Tomcat, Spring Boot cho phép đóng gói mỗi dịch vụ thành một đơn vị thực thi độc lập, tối ưu hóa quy trình Container hóa trên nền tảng Docker. Bên cạnh đó, các công cụ giám sát tích hợp sẵn như Spring Boot Actuator đóng vai trò quan trọng trong việc cung cấp các chỉ số sức khỏe hệ thống thời gian thực, tạo tiền đề cho quy trình vận hành DevOps tự động hóa.

Đối với thách thức về quản lý giao tiếp trong môi trường phân tán, bộ công cụ Spring Cloud đóng vai trò là lớp điều phối hạ tầng. Thông qua việc tích hợp Spring Cloud Netflix Eureka, các dịch vụ có thể tự động định vị và kết nối với nhau trong mạng lưới động mà không cần cấu hình địa chỉ IP cứng. Đồng thời, Spring Cloud Gateway được triển khai như một cổng vào duy nhất (Entry Point) giúp kiểm soát lưu lượng, cân bằng tải và bảo vệ hệ thống khỏi các đợt truy cập đột biến nhờ cơ chế ngắt mạch (Circuit Breaker) của Resilience4j, đảm bảo tính sẵn sàng cao cho toàn bộ nền tảng.

Về khía cạnh an toàn thông tin và quản lý dữ liệu, hệ thống thiết lập cơ chế bảo mật chiều sâu thông qua Spring Security. Framework này hoạt động như một lớp lá chắn phi trạng thái (Stateless), xác thực danh tính người dùng qua chuẩn JSON Web Token (JWT) và phân quyền truy cập chi tiết đến từng phương thức xử lý. Song song đó, sự kết hợp giữa Spring Data JPA và Spring for Apache Kafka tạo nên lớp kết nối dữ liệu mạnh mẽ, không chỉ đơn giản hóa các thao tác với cơ sở dữ liệu quan hệ mà còn hỗ trợ đặc lực cho mô hình kiến trúc hướng sự kiện (Event-Driven Architecture), đảm bảo tính toàn vẹn và nhất quán của luồng dữ liệu nghiệp vụ.

1.1.3. Next.js

Next.js là một framework React mạnh mẽ, giúp phát triển ứng dụng web hiện đại với hiệu suất cao. Framework này hỗ trợ cả Server-Side Rendering (SSR) và Static Site Generation (SSG), giúp tối ưu tốc độ tải trang và trải nghiệm người dùng. Next.js được phát triển bởi Vercel và ngày càng trở nên phổ biến trong việc xây dựng các website thương mại điện tử, blog, dashboard quản trị, và các hệ thống web phức tạp.



Hình 2. Cấu tạo của thư viện NextJS

Một trong những ưu điểm lớn nhất của Next.js là khả năng render linh hoạt. Framework này hỗ trợ SSR để render nội dung trên server trước khi gửi đến client, giúp cải thiện tốc độ tải trang và tối ưu SEO. Ngoài ra, SSG cho phép tạo sẵn các trang tĩnh trong quá trình build, giúp tăng hiệu suất và giảm tải cho server. Đặc biệt, Incremental Static Regeneration (ISR) giúp cập nhật nội dung trang tĩnh mà không cần build lại toàn bộ ứng dụng.

Bên cạnh đó, hệ thống Routing của Next.js cũng rất đơn giản và mạnh mẽ. Next.js sử dụng File-based Routing, giúp tổ chức cấu trúc URL dễ dàng mà không cần định nghĩa routes thủ công. Framework này cũng hỗ trợ API Routes, cho phép xây dựng backend đơn giản trong cùng một dự án mà không cần backend riêng biệt.

Hiệu suất là một trong những yếu tố quan trọng khi phát triển web, và Next.js có nhiều tính năng tối ưu hóa. Framework này tích hợp Automatic Code Splitting & Tree Shaking, giúp giảm dung lượng file tải xuống. Ngoài ra, Lazy Loading giúp tải hình ảnh, video và component chỉ khi cần thiết, tiết kiệm tài

nguyên. Đặc biệt, Next.js Image Component tối ưu kích thước và định dạng hình ảnh tự động, giúp website hoạt động mượt mà hơn.

Next.js mang lại hiệu suất cao nhờ khả năng kết hợp giữa SSR, SSG và ISR, giúp tối ưu tốc độ tải trang và SEO. Hệ thống routing dựa trên file giúp phát triển dễ dàng mà không cần cấu hình phức tạp. Framework này cũng tích hợp tốt với TypeScript, CMS (Strapi, Sanity, WordPress) và các dịch vụ cloud như Vercel, AWS, Firebase, Netlify, giúp triển khai nhanh chóng và mở rộng dễ dàng.

Ngoài ra, các tính năng tối ưu hóa như code splitting, lazy loading và image optimization giúp cải thiện trải nghiệm người dùng đáng kể. Hơn nữa, Fast Refresh giúp cập nhật giao diện ngay lập tức khi code thay đổi, tăng hiệu suất phát triển đáng kể.

Next.js được ứng dụng rộng rãi trong nhiều lĩnh vực khác nhau. Một trong những lĩnh vực phổ biến nhất là xây dựng website thương mại điện tử, nơi yêu cầu tốc độ tải trang nhanh và SEO tốt. Ngoài ra, framework này còn được sử dụng để phát triển dashboard quản trị, hệ thống CMS, blog, hoặc các nền tảng học trực tuyến như Udemy, Coursera.

Bên cạnh đó, Next.js cũng được dùng trong các ứng dụng SaaS, nhờ khả năng mở rộng linh hoạt và dễ dàng tích hợp với các API bên thứ ba. Doanh nghiệp cũng có thể sử dụng Next.js để phát triển website công ty, với giao diện hiện đại, tối ưu và tốc độ tải nhanh.

Next.js là một trong những giải pháp hàng đầu cho các ứng dụng web hiện đại. Với hiệu suất vượt trội, hỗ trợ SEO tốt, khả năng tối ưu hóa mạnh mẽ và tích hợp linh hoạt, framework này đang trở thành sự lựa chọn phổ biến cho các lập trình viên và doanh nghiệp muốn xây dựng các ứng dụng web chất lượng cao.

1.2. GIỚI THIỆU VỀ VINA ACADEMY

1.2.1. Tổng quan về Vina Academy

ViNA Academy là một nền tảng học tập trực tuyến, cung cấp các khóa học đa dạng trong nhiều lĩnh vực khác nhau. Hệ thống giúp người dùng dễ dàng truy cập các tài liệu học tập, bài giảng video và thực hành trực tuyến. Với giao diện

thân thiện và nhiều tính năng hỗ trợ, ViNA Academy mang đến trải nghiệm học tập hiệu quả và tiện lợi cho người học.

Hệ thống được xây dựng nhằm hỗ trợ cả người học, giảng viên giúp quản lý khóa học, đánh giá kết quả và tối ưu hóa quá trình học tập. ViNA Academy không chỉ là một nền tảng cung cấp khóa học mà còn đóng vai trò quan trọng trong việc kết nối người học với giảng viên, tạo ra môi trường học tập trực tuyến hiện đại và chuyên nghiệp.

1.2.2. Vai trò trong hệ thống

ViNA Academy được tổ chức theo mô hình quản lý với ba vai trò chính: Quản trị viên, Nhân viên và Người dùng. Mỗi vai trò có quyền hạn và trách nhiệm khác nhau để đảm bảo hệ thống hoạt động hiệu quả.

1.2.2.1. Quản trị viên (Admin)

Quản trị viên là người có quyền cao nhất trong hệ thống, chịu trách nhiệm quản lý toàn bộ nền tảng ViNA Academy. Nhiệm vụ của quản trị viên bao gồm:

- Quản lý tài khoản giảng viên, nhân viên và người dùng.
- Duyệt và phê duyệt các khóa học mới.
- Kiểm soát nội dung khóa học để đảm bảo chất lượng giảng dạy.
- Theo dõi và đánh giá hiệu suất của hệ thống.
- Quản lý thanh toán, doanh thu và các chính sách tài chính.

1.2.2.2. Nhân viên (Staff)

Nhân viên là những người hỗ trợ vận hành hệ thống và đảm bảo mọi hoạt động diễn ra trơn tru. Vai trò của nhân viên bao gồm:

- Hỗ trợ giảng viên trong quá trình đăng tải khóa học.
- Giải đáp thắc mắc của người dùng thông qua hệ thống hỗ trợ.
- Quản lý nội dung, kiểm duyệt đánh giá và phản hồi từ người học.
- Hỗ trợ các vấn đề kỹ thuật liên quan đến hệ thống học tập trực tuyến.

1.2.2.3. Người dùng (User)

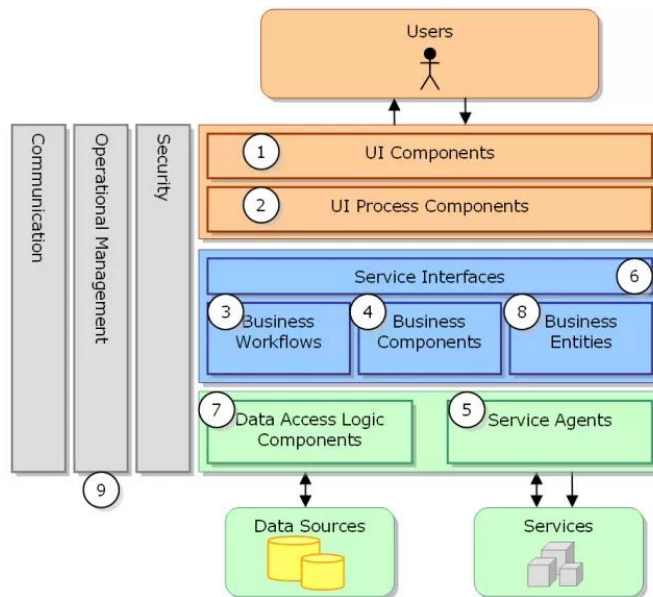
Người dùng là đối tượng chính của ViNA Academy, bao gồm học viên và giảng viên.

- a. Học viên có thể:
 - Đăng ký và tham gia các khóa học.
 - Xem bài giảng video, tài liệu và làm bài kiểm tra.
 - Đánh giá và phản hồi chất lượng khóa học.
 - Tương tác với giảng viên và các học viên khác.
- b. Giảng viên có thể:
 - Đăng tải khóa học và nội dung giảng dạy.
 - Theo dõi tiến độ học tập của học viên.
 - Trả lời câu hỏi và hỗ trợ học viên trong quá trình học.
 - Nhận thanh toán từ các khóa học đã đăng tải.

1.3. GIỚI THIỆU CÁC THÀNH PHẦN TRONG ỨNG DỤNG

1.3.1. Bối cảnh và Nền tảng Kế thừa

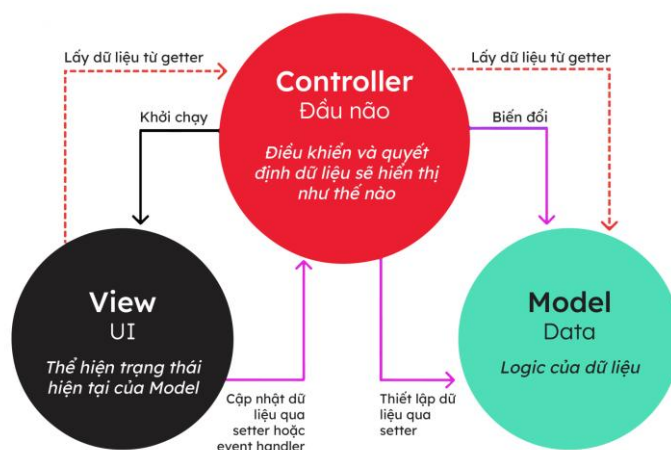
Trong giai đoạn Tiểu luận chuyên ngành, webapp Vina Academy được nhóm xây dựng dựa trên kiến trúc Nguyên khối (Monolithic) truyền thống nhằm tối ưu hóa tốc độ phát triển ban đầu. Hệ thống tuân thủ nghiêm ngặt mô hình Kiến trúc 3 lớp (Three-layer Architecture) và mẫu thiết kế MVC (Model-View-Controller), sử dụng Java Spring Boot làm nền tảng công nghệ cốt lõi.



Hình 3. Mô hình 3 lớp

Cấu trúc này đã vận hành ổn định các nghiệp vụ cơ bản như: Quản lý người dùng, Quản trị nội dung khóa học và Xử lý thanh toán. Tuy nhiên, khi tiến tới xây dựng và thực hiện đề tài Khoá luận tốt nghiệp, nhóm cần nghiên cứu mở rộng tính năng về Trí tuệ nhân tạo (AI) và Tương tác thời gian thực (Real-time), kiến trúc nguyên khối đã bộc lộ những hạn chế về khả năng mở rộng (Scalability) và tính linh hoạt trong triển khai.

Mẫu Kiến trúc MVC



Hình 4. Kiến trúc MVC

Thay vì loại bỏ hoàn toàn hệ thống và mã nguồn cũ, nhóm đã nghiên cứu và vận dụng Chiến lược “Hiện đại hoá ứng dụng” – giữ lại phần logic nghiệp vụ ổn định của hệ thống Monolith cũ, tiến hành chuẩn hoá và đóng gói thành dịch vụ trung tâm gọi là Core Learning Service. Sau đó, tái cấu trúc, chuyển đổi giao thức giao tiếp nội bộ sang chuẩn RESTful API để các dịch vụ vệ tinh được phát triển sau này có thể tích hợp vào hệ thống một cách liền mạch. Nhờ đó, nhóm tận dụng và phát triển nối tiếp theo sự ổn định của mã nguồn cũ, đồng thời mở rộng tính linh hoạt từ kiến trúc Microservices.

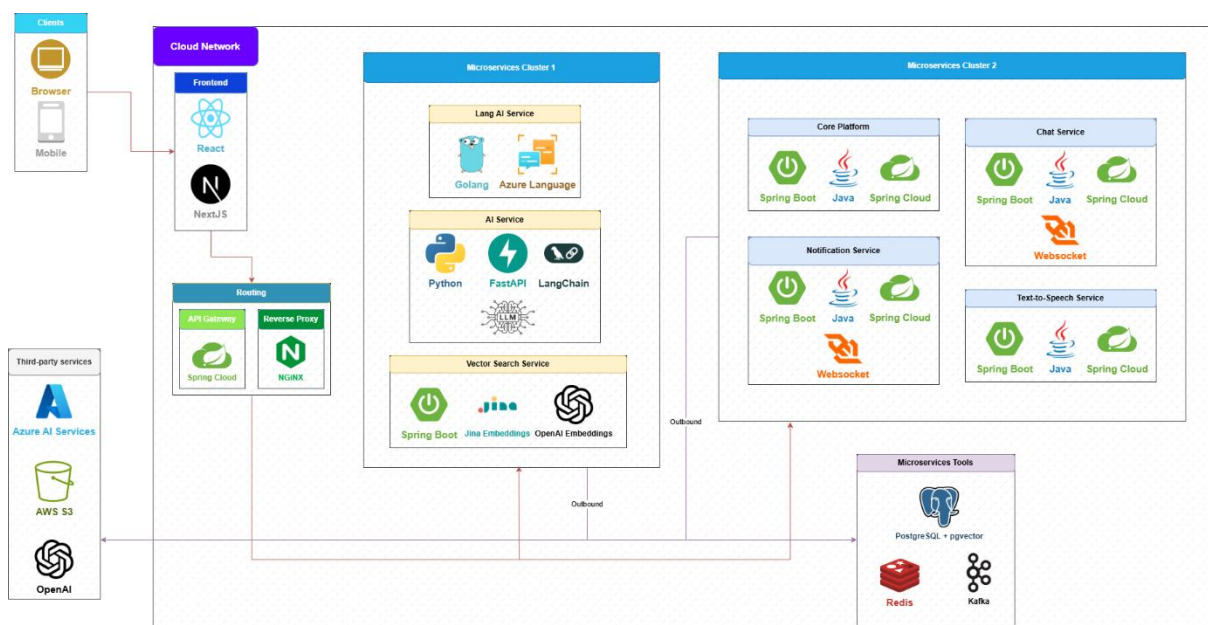
1.3.2. Kiến trúc Microservices (Microservices Architecture)

Để giải quyết các thách thức về khả năng mở rộng (scalability) và bảo trì (maintainability) của kiến trúc nguyên khối (Monolithic) truyền thống, hệ thống

được xây dựng dựa trên kiến trúc Microservices kết hợp với mô hình Hướng sự kiện (Event-Driven Architecture).

Hệ thống được phân tách thành tập hợp các dịch vụ nhỏ, hoạt động độc lập (loose coupling) và có tính liên kết cao về mặt chức năng (high cohesion). Mỗi dịch vụ quản lý một miền nghiệp vụ cụ thể, cho phép phát triển, triển khai và mở rộng tài nguyên một cách linh hoạt.

Kiến trúc hệ thống bao gồm các tầng chính sau:



Hình 5. Kiến trúc Microservice của hệ thống Vina Academy

a. Tầng Truy cập và Giao diện (Access & Presentation Layer)

Đây là tầng tương tác trực tiếp với người dùng cuối, đảm nhiệm việc hiển thị và điều hướng yêu cầu:

- End User (Client): Người dùng tương tác với hệ thống thông qua trình duyệt web đa nền tảng hoặc thiết bị di động.
- Reverse Proxy & Load Balancer (Nginx): Đóng vai trò là điểm tiếp nhận yêu cầu đầu tiên (Entry Point). Nginx chịu trách nhiệm phân tải, đảm bảo tính sẵn sàng cao (High Availability) và thiết lập lớp bảo mật sơ cấp (SSL/TLS Termination) trước khi yêu cầu đi vào mạng lưới nội bộ.
- Frontend Application (Next.js): Ứng dụng giao diện người dùng được xây dựng trên Framework Next.js (React). Thành phần này sử dụng cơ

chế Server-Side Rendering (SSR) để tối ưu hóa hiệu suất tải trang và SEO, giao tiếp với Backend thông qua chuẩn RESTful API.

b. Tầng Điều phối và Khám phá Dịch vụ (Gateway & Discovery Layer)

Tầng này đóng vai trò là "người gác cổng" và điều phối giao thông cho toàn bộ hệ thống Backend:

- API Gateway (Spring Cloud Gateway): Là cổng vào duy nhất (Single Entry Point) cho các truy cập API. Thành phần này thực hiện các chức năng cắt ngang (Cross-cutting concerns):
 - Routing: Định tuyến yêu cầu từ Client đến đúng Microservice xử lý.
 - Security & Throttling: Thực hiện xác thực (Authentication), phân quyền (Authorization) tập trung và giới hạn tốc độ truy cập (Rate Limiting) để bảo vệ hệ thống.
- Service Discovery (Eureka Server): Đóng vai trò là Máy chủ đăng ký dịch vụ (Service Registry). Các microservices khi khởi động sẽ tự động đăng ký metadata (IP, Port) với Eureka. Cơ chế này cho phép API Gateway và các dịch vụ khác định vị động các instance đang hoạt động mà không cần cấu hình cứng (Hard-coded), đảm bảo tính linh hoạt khi mở rộng hạ tầng.

c. Tầng Dịch vụ Nghiệp vụ (Microservices Layer)

Đây là trung tâm xử lý logic nghiệp vụ, bao gồm các dịch vụ độc lập được container hóa:

Core Service (Vinacademy Platform): Dịch vụ lõi quản lý các nghiệp vụ nền tảng như: Quản lý Khóa học, Người dùng, và Đăng ký học (Enrollment). Thành phần này được thiết kế lại từ hệ thống cũ theo hướng hiện đại hóa để tương thích hoàn toàn với kiến trúc Microservice.

- Interaction Services:
 - Notification Service: Quản lý việc gửi thông báo đa kênh (Email, Push Notification) đến người dùng dựa trên các sự kiện nghiệp vụ.

- Chat Service: Cung cấp khả năng nhắn tin thời gian thực (Real-time), xử lý lưu trữ lịch sử hội thoại và trạng thái online của người dùng.
- AI & Intelligence Services:
 - Text-to-Speech Service: Tích hợp Azure Speech API để chuyển đổi văn bản bài học thành giọng nói tự nhiên.
 - Chatbot Service: Trợ lý ảo thông minh sử dụng LangChain và mô hình ngôn ngữ lớn (LLM) để hỗ trợ giải đáp thắc mắc tự động cho học viên.
 - AI Search Service (Tìm kiếm thông minh): Thay thế phương pháp tìm kiếm từ khóa truyền thống bằng cơ chế Semantic Search (Tìm kiếm ngữ nghĩa) thông qua quy trình 2 giai đoạn:
 1. Retrieval (Truy xuất): Chuyển đổi truy vấn thành Vector (Embedding) và tìm kiếm các bản ghi tương đồng nhất trong cơ sở dữ liệu pgvector.
 2. Re-ranking (Tái xếp hạng): Hậu xử lý kết quả dựa trên độ chính xác ngữ nghĩa và các trọng số nghiệp vụ (Rating, số lượng học viên) để trả về kết quả tối ưu nhất.

d. Tầng Hạ tầng Dữ liệu và Giao tiếp (Data & Messaging Infrastructure)

Hệ thống áp dụng chiến lược Polyglot Persistence (Đa hình lưu trữ) và giao tiếp bất đồng bộ để đảm bảo hiệu năng:

- PostgreSQL + pgvector: Đóng vai trò là Cơ sở dữ liệu chính (Primary Database) và Cơ sở dữ liệu Vector. Tiềm năng pgvector cho phép lưu trữ Vector Embeddings, hỗ trợ truy vấn tương đồng (Similarity Search) hiệu suất cao phục vụ cho tính năng AI Search và Chatbot RAG.
- Redis: Kho lưu trữ Key-Value in-memory, được sử dụng để Caching dữ liệu truy cập thường xuyên và quản lý phiên làm việc (Session Management), giúp giảm tải đáng kể cho Database chính.
- Apache Kafka (Event Bus): Xương sống của kiến trúc hướng sự kiện. Kafka đảm bảo việc giao tiếp bất đồng bộ giữa các microservices, giúp tách rời (Decouple) sự phụ thuộc giữa các thành phần.

- Ví dụ thực tế: Khi Chat Service nhận tin nhắn mới, nó đẩy một sự kiện (Event) vào Kafka Topic. Notification Service sẽ lắng nghe (Consume) sự kiện này để gửi thông báo đẩy mà không làm gián đoạn luồng xử lý của Chat Service.

e. Môi trường Triển khai (Deployment Environment)

Toàn bộ các thành phần (Services, Database, Middleware) đều được đóng gói (Containerization) sử dụng Docker và điều phối bởi Docker Compose. Điều này đảm bảo tính nhất quán (Consistency) tuyệt đối giữa môi trường phát triển (Development) và môi trường triển khai thực tế (Production), đồng thời đơn giản hóa quy trình vận hành và nâng cấp hệ thống.

1.4. CÁC CÔNG NGHỆ LIÊN QUAN

1.4.1. Json Web Token

Json Web Token (JWT) là chuẩn mở (RFC 7519) dùng để truyền tải thông tin an toàn giữa các bên dưới dạng đối tượng JSON.

Vai trò trong Vina Academy: Trong kiến trúc Microservices, hệ thống sử dụng cơ chế xác thực phi trạng thái (Stateless Authentication). Khi người dùng đăng nhập, hệ thống cấp một chuỗi JWT. Chuỗi này được API Gateway kiểm tra tính hợp lệ trước khi điều hướng yêu cầu đến các service bên trong, giúp giảm tải việc truy vấn thông tin người dùng liên tục vào cơ sở dữ liệu.

Cấu trúc: Gồm Header, Payload và Signature. Đặc biệt, phần Signature giúp đảm bảo tính toàn vẹn dữ liệu, ngăn chặn việc giả mạo thông tin quyền hạn (Authorization) khi truyền giữa các services.

1.4.2. Maven

Apache Maven là công cụ quản lý dự án và tự động hóa quy trình xây dựng (build automation) cho hệ sinh thái Java.

Vai trò: Hệ thống sử dụng Maven để quản lý các thư viện phụ thuộc (dependencies) cho từng Microservice riêng biệt.

Quản lý đa mô-đun: Maven giúp chuẩn hóa cấu trúc thư mục và quy trình build (biên dịch, kiểm thử, đóng gói) cho toàn bộ các service Spring Boot, đảm bảo sự nhất quán khi triển khai lên môi trường Docker.

1.4.3. Spring Cloud Ecosystem

Bộ công cụ hỗ trợ xây dựng các hệ thống phân tán, giúp giải quyết các bài toán phức tạp trong kiến trúc Microservices.

Spring Cloud Gateway: Đóng vai trò là cổng vào duy nhất (API Gateway), xử lý định tuyến (routing), cân bằng tải (load balancing) và bảo mật sơ cấp.

Spring Cloud Netflix Eureka: Đóng vai trò là Service Discovery (Khám phá dịch vụ), giúp các microservices tự động tìm thấy nhau trong mạng lưới mà không cần cấu hình địa chỉ IP cứng.

1.4.4. Apache Kafka

Apache Kafka là nền tảng phân phối luồng sự kiện (Event Streaming Platform) có khả năng chịu lỗi và hiệu năng cao.

Vai trò trong hệ thống: Kafka đóng vai trò là xương sống của kiến trúc hướng sự kiện (Event-Driven Architecture). Nó đảm nhận việc truyền tải thông điệp bất đồng bộ (Asynchronous messaging) giữa các dịch vụ.

Ứng dụng thực tế: Khi Chat Service ghi nhận tin nhắn mới, nó đẩy một sự kiện vào Kafka. Notification Service sẽ lắng nghe sự kiện này để gửi thông báo cho người dùng, giúp tách rời sự phụ thuộc giữa hai dịch vụ này.

1.4.5. PostgreSQL & pgvector

Bên cạnh vai trò là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) lưu trữ dữ liệu nghiệp vụ, PostgreSQL được mở rộng với pgvector để phục vụ tính năng AI.

pgvector: Là một tiện ích mở rộng (extension) mã nguồn mở, cho phép lưu trữ và truy vấn vector embeddings ngay trong PostgreSQL.

Vai trò: Giúp hệ thống thực hiện tìm kiếm tương đồng (Similarity Search) và tìm kiếm ngữ nghĩa (Semantic Search) cho tính năng gợi ý khóa học và Chatbot, thay thế cho các giải pháp Vector Database chuyên biệt đắt đỏ.

1.4.6. LangChain

LangChain là một framework hiện đại giúp phát triển các ứng dụng dựa trên mô hình ngôn ngữ lớn (LLM).

Vai trò: LangChain được sử dụng trong Chatbot Service để quản lý luồng hội thoại, kết nối với dữ liệu khóa học (thông qua kỹ thuật RAG - Retrieval

Augmented Generation) và tương tác với các mô hình AI. Nó giúp Chatbot có khả năng "nhớ" ngữ cảnh và trả lời chính xác dựa trên dữ liệu thực tế của Vina Academy.

1.4.7. Docker

Docker là nền tảng ảo hóa mức hệ điều hành, cho phép đóng gói ứng dụng và toàn bộ thư viện phụ thuộc vào một đơn vị chuẩn gọi là Container.

Vai trò: Mỗi microservice (Core, Chat, AI...) được đóng gói thành một Docker Container riêng biệt. Điều này đảm bảo môi trường phát triển (Development) đồng nhất với môi trường triển khai (Production), đồng thời giúp việc mở rộng (scaling) hệ thống trở nên dễ dàng và linh hoạt.

CHƯƠNG 2. PHƯƠNG PHÁP NGHIÊN CỨU

2.1. PHƯƠNG PHÁP TIẾP CẬN

2.1.1. Nghiên cứu tài liệu

- Mục tiêu: Nghiên cứu các công nghệ và phương pháp phát triển phần mềm hiện đại, cũng như các mẫu ứng dụng có liên quan đến hệ thống quản lý khóa học trực tuyến cùng với các công nghệ chuyên biệt cho từng bài toán nghiệp vụ như AI và thời gian thực và các phương pháp tích hợp hệ thống.
- Nội dung nghiên cứu:
 - Kiến trúc hệ thống: Các mẫu thiết kế Microservice (API Gateway, Service Discovery).
 - Nền tảng Backend: Hệ sinh thái Spring Cloud (Eureka, Spring Cloud Gateway) và Spring Boot cho các nghiệp vụ chính .
 - Nền tảng Frontend: Công nghệ Next.js (React) hỗ trợ Server-Side Rendering (SSR).
 - Tích hợp AI/NLP: Các thư viện Python như FastAPI và LangChain để xây dựng Chatbot RAG (Retrieval-Augmented Generation).
 - Cơ sở dữ liệu: Các hệ quản trị cơ sở dữ liệu quan hệ (PostgreSQL), lưu trữ vector (pgvector), và caching (Redis).

- Giao tiếp bất đồng bộ: Kiến trúc hướng sự kiện (EDA) sử dụng Message Broker như Kafka.
- DevOps: Phương pháp container hóa (Docker) và CI/CD (GitHub Actions).
- Nguồn tài liệu: Tài liệu tham khảo từ sách, website, blog công nghệ liên quan đến lập trình web với Next.js (React) cho frontend, Spring Boot cho backend, và tích hợp hệ thống thanh toán trực tuyến (VNPay, Momo).

2.1.2. Phân tích yêu cầu

- Mục tiêu: Xây dựng các yêu cầu chức năng và phi chức năng của hệ thống, bao gồm:
 - Chức năng: Quản lý khóa học, đăng ký học, quản lý giảng viên và học viên, hệ thống thanh toán, đánh giá khóa học.
 - Phi chức năng: Tối ưu hiệu suất, bảo mật dữ liệu, hỗ trợ đa nền tảng (responsive UI), tích hợp email thông báo.
- Thực hiện: Thu thập và phân tích yêu cầu từ người dùng tiềm năng, từ đó xây dựng tài liệu phân tích hệ thống và sơ đồ luồng công việc (workflow diagram).

2.1.3. Lựa chọn công nghệ

- Mục tiêu:
 - Để tối ưu hóa hiệu năng cho từng loại tác vụ đặc thù trong hệ thống Vina Academy, nhóm áp dụng chiến lược Polyglot Programming (Đa ngôn ngữ lập trình).
 - Quyết định lựa chọn công nghệ cho từng nhóm dịch vụ dựa trên các tiêu chí về: Hiệu năng (Performance), Hệ sinh thái (Ecosystem) và Khả năng bảo trì (Maintainability).
- Thực hiện:

Nhóm Dịch vụ	Công nghệ lựa chọn	Lý do đề xuất (Justification)
Core Services	Java (Spring Boot)	Tính ổn định & Bảo mật: Java là ngôn ngữ định kiểu mạnh (Strongly typed), giúp giảm thiểu lỗi runtime.

(User, Course, Payment)		Spring Boot cung cấp hệ sinh thái bảo mật (Spring Security) và quản lý giao dịch (Transaction Management) xuất sắc, phù hợp cho các nghiệp vụ cốt lõi cần độ chính xác cao như thanh toán và quản lý dữ liệu người dùng.
AI & Data Services (Chatbot RAG, Vector Search)	Python (FastAPI)	Hệ sinh thái AI: Python là ngôn ngữ số 1 cho AI/ML. Việc sử dụng Python giúp tích hợp dễ dàng với các thư viện tiên tiến như LangChain, PyTorch và các driver kết nối Vector Database. FastAPI được chọn thay vì Django/Flask nhờ hiệu năng cao (Asynchronous) và khả năng tạo API nhanh chóng.
High Performance Services (AI Lang Service, Text Processing)	Go (Golang)	Tốc độ & Concurrency: Go có khả năng xử lý đồng thời (Concurrency) vượt trội nhờ cơ chế Goroutines và quản lý bộ nhớ hiệu quả. Service xử lý ngôn ngữ/văn bản cần độ trễ thấp (Low Latency) và chịu tải cao, do đó Go là lựa chọn tối ưu hơn Java hay Python trong trường hợp này.
Real-time Services (Chat, Notification)	Java + Kafka	Xử lý luồng sự kiện: Sự kết hợp giữa sự ổn định của Java và khả năng phân phối tin nhắn thông lượng cao (High Throughput) của Apache Kafka đảm bảo hệ thống Chat và Thông báo hoạt động mượt mà ngay cả khi có hàng nghìn người dùng đồng thời.
Frontend Application	Next.js (TypeScript)	Tối ưu SEO & UX: Next.js cung cấp cơ chế Server-Side Rendering (SSR) giúp nội dung khóa học được Google index tốt hơn (SEO). TypeScript giúp kiểm soát lỗi kiểu dữ liệu ngay lúc code, tăng độ tin cậy cho ứng dụng phía Client.

2.2. CÔNG CỤ VÀ MÔI TRƯỜNG PHÁT TRIỂN

2.2.1. Công cụ phát triển

– IDE:

- Sử dụng IntelliJ IDEA làm môi trường phát triển chính để xây dựng backend bằng Spring Boot. Đây là một trong những IDE mạnh mẽ nhất cho lập trình Java, cung cấp đầy đủ công cụ hỗ trợ như gợi ý code thông minh, quản lý thư viện với Maven/Gradle, tích hợp debug, quản lý database và kiểm tra hiệu suất ứng dụng. IntelliJ giúp tăng năng suất lập trình và giảm lỗi trong quá trình phát triển backend.
- Sử dụng Visual Studio Code để phát triển frontend với Next.js. VS Code là trình soạn thảo mã nguồn nhẹ, hỗ trợ JavaScript và TypeScript mạnh mẽ. Với hệ thống tiện ích mở rộng phong phú như ESLint, Prettier, Tailwind CSS IntelliSense, cùng với Live Server và tích hợp Git, VS Code giúp lập trình viên phát triển giao diện người dùng một cách hiệu quả và nhanh chóng.

– Công cụ quản lý mã nguồn: Git/GitKraken.

- Git: Sử dụng Git để quản lý mã nguồn của dự án. Git giúp theo dõi các thay đổi trong mã nguồn, cho phép nhiều người cùng làm việc trên một dự án mà không gặp phải xung đột. Git hỗ trợ việc version control và cung cấp khả năng phân nhánh linh hoạt để quản lý các tính năng và sửa lỗi.
- GitKraken: là một công cụ GUI (giao diện người dùng) để quản lý Git, giúp người dùng dễ dàng tương tác với Git thông qua giao diện đồ họa trực quan. GitKraken giúp dễ dàng thực hiện các thao tác như commit, merge, rebase, và quản lý các nhánh mà không cần sử dụng dòng lệnh. Đây là công cụ rất hữu ích cho những ai mới làm quen với Git hoặc muốn tiết kiệm thời gian quản lý mã nguồn.

2.2.2. Công cụ kiểm thử

API Testing: Postman sẽ được sử dụng để kiểm thử các API endpoint. Đây là công cụ mạnh mẽ giúp kiểm tra và xác thực các yêu cầu HTTP, đảm bảo rằng các API được tích hợp vào hệ thống hoạt động đúng như mong đợi.

2.2.3. Môi trường triển khai và Hạ tầng (Deployment & Infrastructure)

- Container hóa và Điều phối (Docker & Orchestration): Hệ thống sử dụng nền tảng Docker để đóng gói toàn bộ ứng dụng, đảm bảo tính nhất quán giữa môi trường phát triển (Development) và triển khai (Production), giải quyết triệt để vấn đề xung đột phụ thuộc (dependency hell).
 - Chiến lược đóng gói: Mỗi microservice (Core, Chat, Notification, AI) và Frontend đều được đóng gói thành các Docker Image riêng biệt, tối ưu hóa kích thước và bảo mật (Multi-stage build).
 - Điều phối dịch vụ (Orchestration): Sử dụng Docker Compose để định nghĩa và vận hành toàn bộ hệ sinh thái dịch vụ, bao gồm:
 - Service Layer: API Gateway, Eureka Server, các Business Services (Java/Python).
 - Infrastructure Layer: Các dịch vụ nền tảng như Database (PostgreSQL), Message Broker (Kafka, Zookeeper), Cache (Redis).
 - Điều này cho phép triển khai toàn bộ hệ thống phức tạp chỉ với một câu lệnh (docker-compose up), đồng thời tạo tiền đề dễ dàng mở rộng lên các nền tảng Container Orchestration lớn hơn như Kubernetes (K8s) trên AWS hoặc GCP.
- Hạ tầng Dữ liệu đa hình (Polyglot Persistence): Thay vì sử dụng một cơ sở dữ liệu duy nhất, hệ thống áp dụng chiến lược lưu trữ đa hình để tối ưu hóa hiệu suất cho từng loại dữ liệu đặc thù:
 - PostgreSQL (Relational Data): Đóng vai trò là kho dữ liệu chính (Source of Truth), quản lý các thực thể có cấu trúc chặt chẽ như Người dùng, Khóa học, Đơn hàng. Việc truy xuất dữ liệu được thực hiện thông qua Spring Data JPA/Hibernate để đảm bảo tính toàn vẹn và hiệu suất.

- pgvector (Vector Data): Sử dụng tiện ích mở rộng của PostgreSQL để lưu trữ và lập chỉ mục cho các vector embeddings. Đây là thành phần cốt lõi phục vụ cho tính năng AI Semantic Search và RAG Chatbot, cho phép thực hiện các truy vấn tìm kiếm tương đồng (Similarity Search) tốc độ cao ngay trong cơ sở dữ liệu quan hệ.
- Redis (In-memory Data): Được sử dụng làm lớp bộ nhớ đệm (Caching Layer) để giảm tải cho database chính và lưu trữ dữ liệu phiên làm việc (Session), đảm bảo tốc độ phản hồi nhanh cho người dùng cuối.

CHƯƠNG 3. KHẢO SÁT HIỆN TRẠNG VÀ MÔ HÌNH HÓA YÊU CẦU

3.1. KHẢO SÁT HIỆN TRẠNG

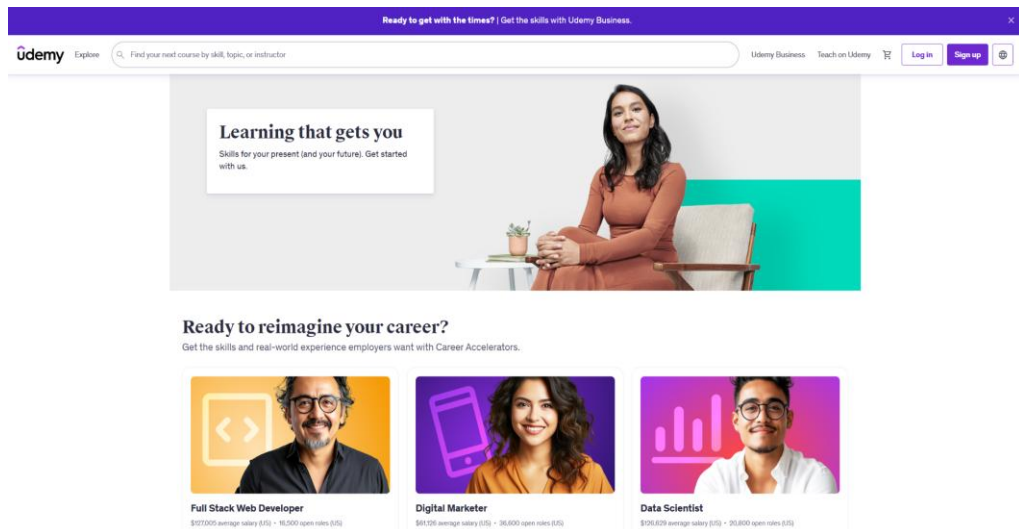
Hiện nay, thị trường học trực tuyến toàn cầu đang phát triển mạnh mẽ với các nền tảng hàng đầu như Udemy, Coursera, edX cung cấp hàng ngàn khóa học thuộc đa dạng lĩnh vực từ lập trình, kinh doanh, thiết kế đến khoa học dữ liệu. Các nền tảng này nổi bật với hệ thống bài giảng chuyên sâu, bài tập thực hành, dự án thực tế và chứng chỉ có giá trị. Tuy nhiên, một rào cản lớn đối với người học Việt Nam là phần lớn nội dung được giảng dạy bằng tiếng Anh và các ngôn ngữ quốc tế khác. Điều này tạo ra khoảng cách đáng kể đối với đa số người Việt chưa thành thạo ngoại ngữ, khiến họ khó tiếp cận được nguồn kiến thức quý giá này.

Nhận thấy nhu cầu cấp thiết về một nền tảng học trực tuyến chất lượng cao bằng tiếng Việt, Vina Academy được thành lập với sứ mệnh xóa bỏ rào cản ngôn ngữ và mang kiến thức toàn cầu đến với người Việt Nam. Nền tảng này không đơn thuần là bản dịch của các khóa học quốc tế, mà còn được thiết kế đặc biệt phù hợp với văn hóa học tập và nhu cầu thực tiễn của thị trường Việt Nam.

Vina Academy hướng đến việc xây dựng một hệ sinh thái giáo dục trực tuyến toàn diện, cung cấp các khóa học chất lượng cao trong nhiều lĩnh vực như công nghệ thông tin, kinh doanh số, marketing online, kỹ năng mềm, và phát triển cá nhân. Đặc biệt, nền tảng chú trọng vào việc cá nhân hóa lộ trình học tập, tạo môi trường tương tác giữa học viên và giảng viên, đồng thời kết nối trực tiếp với nhu cầu tuyển dụng thực tế tại Việt Nam, giúp người học không chỉ tiếp thu kiến thức mà còn nâng cao cơ hội nghề nghiệp sau khi hoàn thành các khóa học.

3.2. KHẢO SÁT MỘT SỐ WEBSITE

3.2.1. Ucademy (<https://www.udemy.com/>)



Hình 6. Trang chủ website Ucademy

a) Tính năng:

- Cung cấp hàng ngàn khóa học trực tuyến về lập trình, kinh doanh, thiết kế, marketing,...
- Cho phép giảng viên cá nhân tạo và bán khóa học.
- Thanh toán theo từng khóa học, không yêu cầu đăng ký gói thành viên.
- Hỗ trợ học tập trên cả web và ứng dụng di động.
- Cấp chứng chỉ hoàn thành khóa học.
- Hệ thống đánh giá và bình luận giúp người học tham khảo trước khi mua
- Tính năng tìm kiếm và lọc khóa học theo nhiều tiêu chí (đánh giá, mức độ, giá cả)
- Chức năng Q&A cho phép tương tác giữa học viên và giảng viên

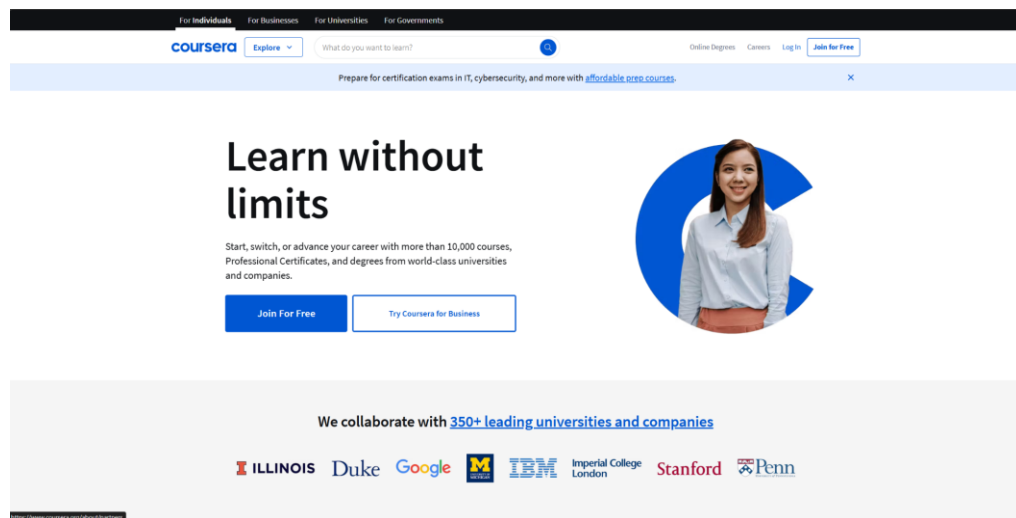
b) Ưu điểm:

- Giá cả linh hoạt, thường xuyên có khuyến mãi.
- Nội dung phong phú, đa dạng chủ đề.
- Học viên có quyền truy cập trọn đời vào khóa học đã mua.
- Có phụ đề cho một số khóa học.
- Giao diện thân thiện, dễ sử dụng cho cả người học và giảng viên
- Cơ chế đảm bảo hoàn tiền trong 30 ngày nếu không hài lòng

c) Nhược điểm:

- Chất lượng khóa học không đồng đều do giảng viên tự do đăng tải.
- Hỗ trợ khách hàng còn hạn chế.
- Chủ yếu là tiếng Anh, ít nội dung tiếng Việt.
- Thiếu lộ trình học tập rõ ràng giữa các khóa học liên quan
- Không có hướng dẫn cá nhân hóa dựa trên năng lực người học
- Không có công cụ đánh giá kỹ năng trước và sau khóa học

3.2.2. Coursera (<https://www.coursera.org/>)



Hình 7. Trang chủ website Coursera

a) Tính năng

- Cung cấp khóa học từ các trường đại học và tổ chức lớn (Stanford, Yale, Google, IBM)
- Mô hình đa dạng: khóa học miễn phí, chứng chỉ chuyên môn, bằng cấp online
- Chương trình học có cấu trúc bài bản với lộ trình rõ ràng
- Bài tập thực hành, dự án thực tế và đánh giá đồng đẳng (peer review)
- Hệ thống diễn đàn thảo luận cho từng khóa học
- Mentorship và coaching trong một số chương trình cao cấp
- Tính năng Coursera Plus cho phép truy cập không giới hạn nhiều khóa học với phí cố định

b) Ưu điểm

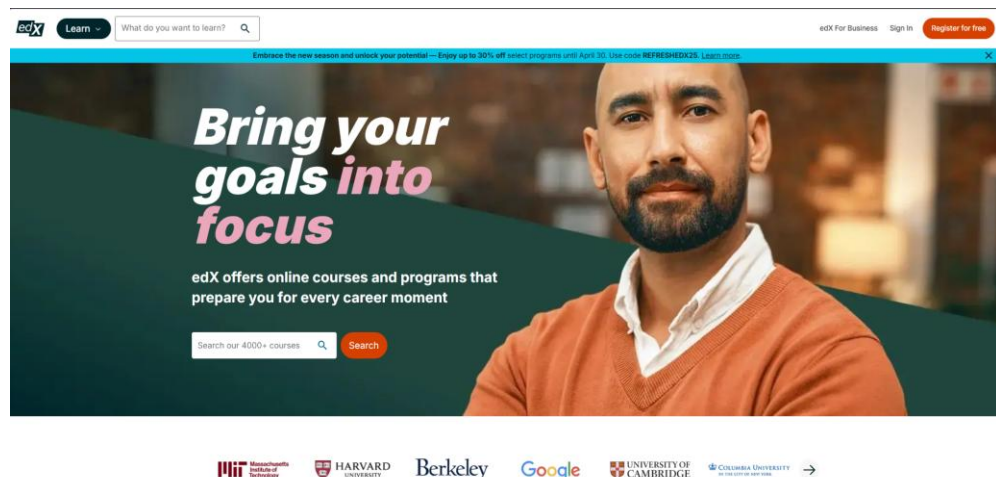
- Chất lượng khóa học cao, nội dung học thuật chuyên sâu và cập nhật

- Học từ giảng viên của các trường đại học và doanh nghiệp danh tiếng toàn cầu
- Có khóa học miễn phí với tùy chọn trả phí để nhận chứng chỉ
- Lộ trình học tập rõ ràng với các specialization và chương trình cấp bằng
- Chứng chỉ được công nhận rộng rãi bởi nhà tuyển dụng
- Hỗ trợ học bổng cho người học từ các quốc gia đang phát triển

c) Nhược điểm

- Giá khóa học khá cao so với mặt bằng chung.
- Yêu cầu thời gian học tập nghiêm túc và có lịch trình cố định.
- Nội dung chủ yếu bằng tiếng Anh, ít hỗ trợ tiếng Việt.
- Trải nghiệm mobile chưa tối ưu bằng trên web
- Hạn chế tiếp cận với giảng viên trong các khóa học lớn
- Không có nhiều khóa học về các kỹ năng đặc thù của thị trường Việt Nam

3.2.3. edX (<https://www.edx.org/>)



Hình 8. Trang chủ website edX

a) Tính năng

- Hợp tác với các trường đại học và tổ chức lớn (MIT, Harvard, Microsoft, IBM,...).
- Cung cấp khóa học trực tuyến miễn phí và trả phí.
- Chương trình học mang tính hàn lâm, có thể lấy chứng chỉ hoặc bằng cấp.
- Có các khóa học MicroMasters và Professional Certificate.

- Tính năng "audit" cho phép học miễn phí nội dung không bao gồm bài tập đánh giá
- Công cụ thảo luận và tương tác học thuật trong từng khóa học
- Hệ thống kiểm tra được giám sát qua webcam cho các khóa học chính thức

b) Ưu điểm

- Nội dung học thuật chất lượng cao, được giảng dạy bởi các trường đại học uy tín.
- Nhiều khóa học miễn phí, chỉ trả phí nếu muốn nhận chứng chỉ.
- Có chương trình cấp bằng online ở bậc đại học và sau đại học.
- Hỗ trợ tốt cho các ngành học STEM, kinh tế, và khoa học xã hội.
- Cung cấp các khóa học có tính ứng dụng cao từ các doanh nghiệp công nghệ
- Giao diện học tập chuyên nghiệp với nhiều công cụ tương tác

c) Nhược điểm

- Học phí cao nếu muốn nhận chứng chỉ hoặc bằng cấp.
- Chương trình học nặng, yêu cầu cam kết thời gian nghiêm túc.
- Nội dung chủ yếu bằng tiếng Anh, không hỗ trợ nhiều ngôn ngữ khác.
- Tỷ lệ hoàn thành khóa học thấp do yêu cầu cao về thời gian và nỗ lực
- Ít tính năng cá nhân hóa lộ trình học tập theo nhu cầu riêng
- Hạn chế trong việc kết nối với cộng đồng học viên cùng khóa học

3.3. YÊU CẦU CHỨC NĂNG

3.3.1. Người dùng chưa đăng ký (Unregistered User)

- Vào trang chủ: Xem danh sách liệt kê các khóa học, thông tin khóa học, thông tin giảng viên.
- Tìm kiếm: Tìm kiếm các khóa học theo tên, theo danh mục, chủ đề, giảng viên, theo lượt đánh giá, theo ngày đăng...
- Xem nội dung khóa học: thông tin mô tả về những gì sẽ có trong khóa học, các đánh giá, giá cả khóa học.
- Đăng ký: Người dùng mới có thể tạo tài khoản bằng cách cung cấp thông tin cá nhân cần thiết.

- Đăng nhập: Sử dụng tài khoản đã tạo để đăng nhập vào hệ thống
- Quên mật khẩu: Lấy lại mật khẩu mới khi quên mật khẩu đăng nhập

3.3.2. Học viên (Student)

- Quản lý tài khoản: Thay đổi thông tin cá nhân, mật khẩu...
- Quản lý giỏ hàng: Thêm, xóa khóa học muốn mua ở giỏ hàng
- Thanh toán: Xem tổng giá, chọn mã giảm giá, thanh toán các khóa học có trong giỏ hàng (qua VNPAY)
- Quản lý khóa học đã mua (đăng ký): Truy cập vào khóa học để xem các bài giảng, viết ghi chú về bài giảng, thực hiện bài kiểm tra kiến thức, theo dõi tiến độ học, viết đánh giá về khóa học, xem/viết thảo luận về khóa học dưới mỗi bài giảng, nhắn tin với giảng viên để hỏi đáp thắc mắc, gửi yêu cầu hoàn trả khóa học nếu cần
- Yêu cầu hỗ trợ: Gửi yêu cầu hỗ trợ kỹ thuật, thanh toán, hoặc hỗ trợ khác đến nhân viên.

3.3.3. Giảng viên (Instructor)

- Quản lý tài khoản: tương tự học viên
- Quản lý khóa học: thêm sửa xóa khóa học, thêm video bài giảng, tài liệu học tập, cũng như thêm bài kiểm tra kiến thức sau mỗi chương học, gửi khóa học để cho Ban Quản Trị kiểm duyệt,...
- Giao tiếp và quản lý học viên: Nhắn tin trao đổi với học viên.
- Theo dõi doanh thu: xem lợi nhuận, doanh thu bán khóa học
- Yêu cầu hỗ trợ: Gửi yêu cầu hỗ trợ đến nhân viên.

3.3.4. Ban quản trị (Admin/Owner)

- Quản lý tài khoản: quản lý các tài khoản học viên, giảng viên, nhân viên,...
- Quản lý khóa học: xem và phê duyệt khóa học, cập nhật trạng thái khóa học (hiển thị/ẩn).
- Quản lý giao dịch: có thể xem lịch sử giao dịch của hệ thống, thay đổi trạng thái giao dịch nếu muốn, hoàn trả tiền khóa học nếu có yêu cầu từ học viên, xem chi tiết doanh thu và tỉ lệ chia hoa hồng với giảng viên,...

3.3.5. Nhân viên (Staff)

- Kiểm duyệt khóa học
- Hỗ trợ người dùng (trả lời yêu cầu hỗ trợ)
- Kiểm duyệt yêu cầu hoàn tiền khóa học từ học viên

3.4. YÊU CẦU PHI CHỨC NĂNG (NON-FUNCTIONAL REQUIREMENTS)

Hệ thống Vina Academy được xây dựng dựa trên kiến trúc Microservices và tích hợp AI, do đó cần đáp ứng các tiêu chuẩn khắt khe về hiệu năng, tính sẵn sàng và khả năng mở rộng.

3.4.1. Hiệu năng và Hiệu suất (Performance Efficiency)

- Thời gian phản hồi (Response Time):
 - API Response Time: Trung bình < 200ms cho các tác vụ đọc (Read) và < 500ms cho các tác vụ ghi (Write/Update).
 - AI Inference Time: Phản hồi từ Chatbot hoặc Semantic Search phải hoàn tất trong vòng < 2 giây.
 - First Contentful Paint (FCP): Trang web hiển thị nội dung đầu tiên dưới 1.5 giây (nhờ cơ chế SSR của Next.js).
- Khả năng chịu tải (Throughput):
 - Hệ thống hỗ trợ tối thiểu 1.000 CCU (Concurrent Users - Người dùng đồng thời) mà không làm suy giảm hiệu năng.
 - Xử lý được lưu lượng đỉnh điểm (Peak Load) lên tới 500 request/giây tại API Gateway.
 - Truyền tải nội dung (Streaming):
 - Hỗ trợ phát video chuẩn HD/Full HD với độ trễ thấp (Low Latency).
 - Thời gian buffering khi bắt đầu video < 3 giây trong điều kiện mạng 4G tiêu chuẩn.

3.4.2. Độ tin cậy và Khả năng chịu lỗi (Reliability & Resilience)

- Tính sẵn sàng (Availability): Đảm bảo thời gian hoạt động (Uptime) đạt 99.9% trong giờ hành chính.
- Cơ chế ngắt mạch (Circuit Breaker): Hệ thống áp dụng mẫu thiết kế Circuit Breaker. Nếu một Microservice (ví dụ: Chat Service) gặp sự cố, hệ thống

phải tự động cô lập lỗi, đảm bảo các chức năng khác (Xem khóa học, Thanh toán) vẫn hoạt động bình thường.

- Phục hồi dữ liệu (Recovery):
 - RPO (Recovery Point Objective): Tối đa 15 phút (mất dữ liệu tối đa 15 phút gần nhất).
 - RTO (Recovery Time Objective): Tối đa 2 giờ để khôi phục hệ thống sau sự cố nghiêm trọng.

3.4.3. Bảo mật và An toàn thông tin (Security)

- Xác thực & Phân quyền:
 - Sử dụng cơ chế Stateless Authentication với JWT (JSON Web Token).
 - Mật khẩu người dùng phải được băm (Hash) bằng thuật toán BCrypt trước khi lưu trữ.
- Bảo mật đường truyền: 100% giao tiếp giữa Client và Server phải được mã hóa qua giao thức HTTPS (TLS 1.2+).
- Bảo vệ dữ liệu: Thông tin nhạy cảm (thanh toán, email) phải được mã hóa tại tầng cơ sở dữ liệu (Encryption at Rest).
- Chống tấn công: API Gateway phải có cơ chế Rate Limiting để ngăn chặn tấn công DDoS và Brute Force.

3.4.4. Khả năng mở rộng (Scalability)

- Mở rộng theo chiều ngang (Horizontal Scaling): Các Microservices (đặc biệt là AI Service và Core Service) phải có khả năng chạy nhiều instances (bản sao) cùng lúc và cân bằng tải tự động.
- Tách biệt tài nguyên: Service AI (sử dụng nhiều CPU/RAM cho Vector Search) phải được triển khai độc lập để không ảnh hưởng đến hiệu năng của các tác vụ nghiệp vụ thông thường.

3.4.5. Khả năng bảo trì (Maintainability)

- Containerization: Toàn bộ dịch vụ được đóng gói bằng Docker, đảm bảo môi trường đồng nhất từ Dev đến Prod.

- Logging & Monitoring: Hệ thống phải có cơ chế ghi Log tập trung và giám sát sức khỏe (Health Check) của từng service thông qua Spring Boot Actuator/Eureka.

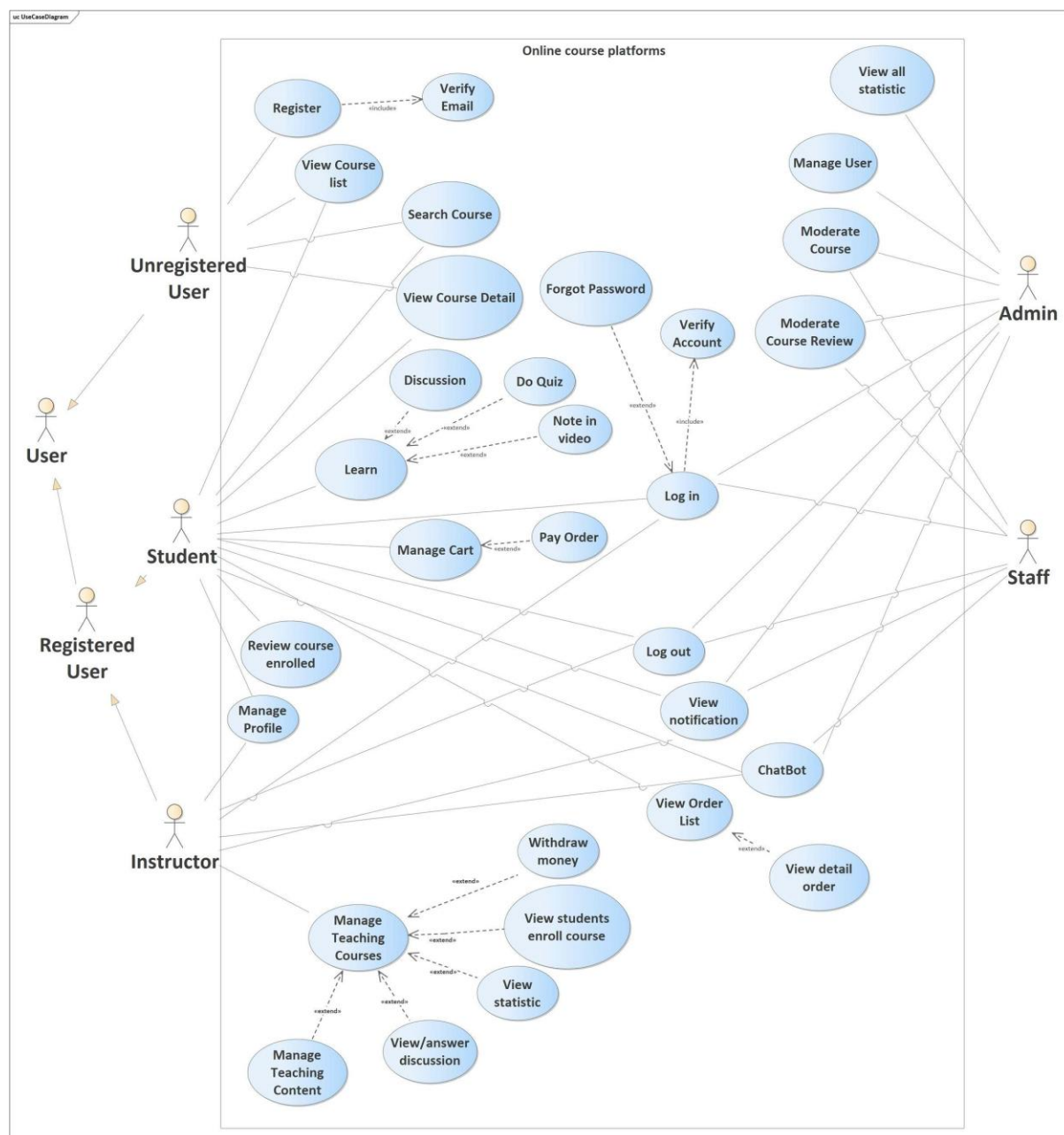
3.4.6. Tính tương thích (Compatibility)

- Đa nền tảng: Giao diện Web (Next.js) hoạt động tốt trên các trình duyệt phổ biến (Chrome, Firefox, Safari, Edge) và tự động tương thích (Responsive) với các thiết bị di động, máy tính bảng.

CHƯƠNG 4. PHÂN TÍCH VÀ THIẾT KẾ PHẦN MỀM

4.1. LƯỢC ĐỒ CHỨC NĂNG VÀ ĐẶC TẢ CHỨC NĂNG

4.1.1. Lược đồ chức năng – UseCase Diagram



Hình 9. Lược đồ chức năng

Xem chi tiết tại: [UseCase Diagram](#)

4.1.2. Đặc tả chức năng

Search Course - Tìm kiếm khóa học

[1]	Search Course
-----	---------------

Actor	Unregistered User, Student
Trigger	Actor bấm chọn “Tìm kiếm” trên giao diện
Description	Chức năng này cho phép người dùng (khách, học viên) tìm kiếm các khóa học trên nền tảng dựa trên từ khóa, tiêu đề, hoặc các bộ lọc/sắp xếp.
Pre-Conditions	Hệ thống có ít nhất một khóa học được công khai Giao diện tìm kiếm khóa học khả dụng
Post-Conditions	Thành công: Danh sách các khóa học phù hợp được hiển thị Thất bại: Thông báo "Không tìm thấy khóa học" được hiển thị
Main Flow	1. Actor truy cập vào trang tìm kiếm hoặc sử dụng thanh tìm kiếm trên header 2. Hệ thống hiển thị giao diện tìm kiếm với: • Ô nhập từ khóa tìm kiếm • Các bộ lọc (nếu có): Danh mục, Cấp độ, Giá, Đánh giá • Các tùy chọn sắp xếp: Mới nhất, Phổ biến nhất, Đánh giá cao nhất 3. Actor nhập từ khóa tìm kiếm vào ô tìm kiếm 4. Actor nhấn Enter hoặc nút "Tìm kiếm" 5. Hệ thống xử lý tìm kiếm: • Tìm trong tiêu đề khóa học • Tìm trong mô tả khóa học • Tìm trong tags/từ khóa 6. Hệ thống hiển thị danh sách các khóa học phù hợp với: • Tên khóa học • Hình ảnh thumbnail • Giá/Miễn phí • Đánh giá và số lượng học viên • Giảng viên 7. Actor xem danh sách kết quả

Alternate Flow	Actor sử dụng tùy chọn sắp xếp lọc để thu hẹp phạm vi khi tìm kiếm
Exception Flow	Nếu không có các khóa học nào thỏa tiêu chí tìm kiếm, hệ thống hiển thị Không tìm thấy sản phẩm.

Bảng 1. Đặc tả Usecase Search Course - Tìm kiếm khóa học

View Course Detail - Xem Chi tiết Khóa học

[2]	View Course Detail
Actor	Unregistered User, Student
Trigger	Actor bấm chọn “Xem chi tiết” trên khóa học đã chọn
Description	Usecase này cho phép Actor điều hướng đến trang hiển thị nội dung chi tiết mô tả về khóa học
Pre-Conditions	Khóa học tồn tại trong hệ thống Khóa học ở trạng thái công khai (published) hoặc Actor có quyền truy cập
Post-Conditions	Actor sẽ đọc được mô tả về khóa học, những kiến thức mà sẽ được học sau khi đăng ký khóa học
Main Flow	1. Actor bấm chọn “Chi tiết” trong khóa học 2. Một trang hiển thị nội dung về khóa học sẽ được hiện ra
Alternate Flow	Không có
Exception Flow	Không có

Bảng 2. Đặc tả Usecase View Course Detail - Xem Chi tiết Khóa học

Register - Đăng ký

[3]	Register
Actor	Unregistered User
Trigger	Actor chọn chức năng “Đăng ký” trên giao diện chính.
Description	Use case này cho phép actor đăng ký tài khoản bằng cách actor cung cấp các thông tin cá nhân cần thiết như họ và tên, email, mật khẩu, số điện thoại.
Pre-Conditions	Không có
Post-Conditions	Actor đăng ký thành công và có thể đăng nhập vào hệ thống.

Main Flow	1. Actor chọn nút Đăng ký trên Trang chủ. 2. Hệ thống hiển thị trang Đăng ký tài khoản. 3. Actor nhập các thông tin cá nhân: <i>họ tên, email,, username, password</i> và nhấn nút Đăng ký. 4. Hệ thống kiểm tra tính hợp lệ của thông tin. 5. Hệ thống thực hiện use case Verify Email, Actor sẽ nhận mã OTP từ email đã điền để nhập vào hộp thoại và xác thực tài khoản 6. Hệ thống tạo tài khoản cho actor và điều hướng đến trang đăng nhập.
Alternate Flow	Verify Email tài khoản đăng ký.
Exception Flow	+ 5.1. Nếu Actor nhập thiếu một trong các trường thông tin đã nêu, hệ thống hiển thị Vui lòng nhập trường này + 5.2. Nếu email, username không đúng định dạng, hệ thống hiển thị thông báo lỗi định dạng tương ứng. + 5.3. Nếu email, username đã tồn tại trong hệ thống, hệ thống hiển thị thông báo lỗi và yêu cầu actor nhập email, username khác. + 5.4. Nếu mật khẩu không thỏa điều kiện, hệ thống hiển thị yêu cầu nhập đúng điều kiện. + 5.5. Nếu mật khẩu và xác nhận mật khẩu không giống nhau, hệ thống hiển thị thông báo Mật khẩu nhập lại không khớp. + 5.6. Nếu Actor không đồng ý với Điều khoản và Chính sách của hệ thống, hệ thống hiển thị Vui lòng đồng ý với Điều khoản và Chính sách của hệ thống.

Bảng 3. Đặc tả Usecase Register - Đăng ký

Verify Email - Xác thực Email

[4]	Verify Email
Actor	System

Trigger	Actor nhấn “Xác nhận Email” ở phần Quản lý Tài khoản hoặc ngay sau khi Đăng ký.
Description	Use case này cho phép actor xác nhận email chính chủ của mình, bằng cách nhận được một email chứa mã xác minh và xác nhận đây là email chính chủ của tài khoản.
Pre-Conditions	Không có.
Post-Conditions	+ Hệ thống xác nhận email thành công.
Main Flow	1. Actor nhấn “Xác nhận Email” ở phần Quản lý Tài khoản hoặc ngay sau khi Đăng ký. 2. Hệ thống tạo Verification Code 3. Actor kiểm tra email và nhập mã xác minh. 4. Hệ thống kiểm tra mã xác minh hợp lệ.
Alternate Flow	Không có.
Exception Flow	+ 2.1. Nếu actor không nhận được email xác nhận, actor phải đợi mã xác nhận hết hạn (30s) trước khi nhấn vào “Gửi lại mã xác nhận”. + 3.1. Nếu actor nhập mã xác nhận không hợp lệ hay mã xác nhận hết hạn, hệ thống thông báo actor nhấn vào “Gửi lại mã xác nhận”.

Bảng 4. Đặc tả Usecase Verify Email - Xác thực Email

Log in- Đăng nhập

[5]	Log in
Actor	Registered User, Staff, Owner
Trigger	Actor chọn chức năng “Đăng nhập” trên giao diện chính.
Description	Use case này cho phép actor đăng nhập vào tài khoản của mình bằng cách cung cấp thông tin đăng nhập
Pre-Conditions	+ Actor đã có tài khoản trong hệ thống.
Post-Conditions	+ Actor đăng nhập thành công và có quyền truy cập vào các chức năng của hệ thống.
Main Flow	1. Actor chọn nút Đăng nhập trên Trang chủ.

	2. Hệ thống hiển thị trang Đăng nhập. 3. Actor nhập email, mật khẩu vào các trường tương ứng, sau đó nhấn nút Đăng nhập. 4. Hệ thống thực hiện use case Verify Account. 5. Hệ thống hiển thị trang giao diện theo vai trò của actor.
Alternate Flow	+ Nếu actor quên mật khẩu, actor có thể chọn chức năng “Quên mật khẩu” để nhận hướng dẫn khôi phục mật khẩu.
Exception Flow	+ 3.1. Nếu actor nhập thiếu một trong các trường thông tin đã nêu, hệ thống hiển thị thông báo nhắc nhở actor điền đầy đủ thông tin. + 3.2. Nếu email hoặc mật khẩu không chính xác, hệ thống hiển thị thông báo lỗi và yêu cầu actor nhập lại thông tin. + 4.3. Nếu email không tồn tại, hệ thống hiển thị thông báo yêu cầu actor tạo tài khoản ở trang Đăng ký.

Bảng 5. Đặc tả Usecase Log in- Đăng nhập

Verify Account - Xác nhận tài khoản

[6]	Verify Account
Actor	System
Trigger	Actor nhấn nút “Đăng nhập” ở trang Đăng nhập.
Description	Use case này cho phép hệ thống kiểm tra email và password actor cung cấp có chính xác với dữ liệu hiện có không.
Pre-Conditions	Không có.
Post-Conditions	+ Hệ thống xác nhận email và password hợp lệ.
Main Flow	1. Hệ thống kiểm tra các điều kiện cơ bản của dữ liệu đầu vào. 2. Hệ thống truy vấn cơ sở dữ liệu để tìm kiếm thông tin email mà actor đã cung cấp, sau đó tiến hành lấy password trong cơ sở dữ liệu và so sánh với password mà actor nhập. 3. Hệ thống xác nhận quá trình đăng nhập thành công.

Alternate Flow	Không có.
Exception Flow	+ 1.1. Nếu email trống hoặc password không đủ độ dài và chứa ký tự đặc biệt, hệ thống thông báo yêu cầu actor nhập lại thông tin. + 2.1. Nếu email không tồn tại, hệ thống thông báo yêu cầu actor tạo tài khoản. + 2.2. Nếu password không hợp lệ, hệ thống thông báo cho actor là “Mật khẩu không hợp lệ” để actor nhập lại mật khẩu hoặc actor có thể chọn Quên mật khẩu.

Bảng 6. Đặc tả Usecase Verify Account - Xác nhận tài khoản

Forgot Password - Quên mật khẩu

[7]	Forgot Password
Actor	Registered User
Trigger	Actor nhấn vào “Quên mật khẩu” ở phần Đăng nhập.
Description	+ Use case này cho phép hệ thống gửi một email chứa code để reset mật khẩu actor thông qua email đã được actor xác nhận. + Mật khẩu nhận được chỉ có hiệu lực trong thời gian nhất định (90s) sau khi hệ thống gửi code qua email.
Pre-Conditions	Không có.
Post-Conditions	+ Actor đã thay đổi được mật khẩu.
Main Flow	1. Actor chọn “Quên mật khẩu” ở phần Đăng nhập. 2. Hệ thống hiển thị trang Quên mật khẩu để actor nhập địa chỉ email. 3. Actor nhập địa chỉ email đã đăng ký và nhấn “Gửi mã xác nhận”. 4. Hệ thống kiểm tra sự tồn tại của email, sau đó gửi email đính kèm mã xác nhận đến actor. 5. Actor kiểm tra email và nhập mã xác nhận. 6. Actor cập nhật mật khẩu mới.

	7. Hệ thống cập nhật mật khẩu trong hệ thống và thông báo thành công.
Alternate Flow	Không có.
Exception Flow	+ 4.1. Nếu email chưa tồn tại, hệ thống thông báo actor nhập lại email. + 5.1. Nếu actor nhập mật khẩu mới sau thời gian hiệu lực (90s), hệ thống yêu cầu actor nhấn “Gửi lại mã xác nhận”. + 7.1. Nếu mật khẩu và nhập lại mật khẩu không trùng nhau, hệ thống thông báo actor nhập lại.

Bảng 7. Đặc tả Usecase Forgot Password - Quên mật khẩu

Log out - Đăng xuất

[8]	Log out
Actor	Registered User, Staff, Owner
Trigger	Actor chọn chức năng “Đăng xuất” trên giao diện chính.
Description	Usecase này cho phép actor đăng xuất khỏi website.
Pre-Conditions	Actor đã đăng nhập vào hệ thống.
Post-Conditions	+ Actor đăng xuất thành công và trở về giao diện trang chủ. + Phiên làm việc của actor sẽ kết thúc và không còn hiệu lực.
Main Flow	1. Actor chọn nút Đăng xuất trên giao diện hệ thống. 2. Hệ thống hiển thị thông báo xác nhận việc đăng xuất. 3. Actor xác nhận đăng xuất. 4. Hệ thống kết thúc phiên làm việc của actor và điều hướng actor quay lại giao diện trang chủ.
Alternate Flow	Không có.
Exception Flow	+ 3.1. Nếu actor không xác nhận đăng xuất, hệ thống giữ nguyên phiên làm việc hiện tại.

Bảng 8. Đặc tả Usecase Log out - Đăng xuất

View Notifications - Xem thông báo

[9]	View Notifications
Actor	Registered User, Staff

Trigger	Actor chọn biểu tượng “Thông báo” trên giao diện chính.
Description	+ Use case này cho phép actor xem thông báo về các sự kiện quan trọng như: - Registered User: thanh toán thành công, các chương trình khuyến mãi mới, phản hồi từ nền tảng về các yêu cầu hỗ trợ hoặc đánh giá khóa học.. - Staff: có khóa học cần kiểm duyệt, có yêu cầu hỗ trợ từ Registered User, có đánh giá cần duyệt, các thông tin hệ thống khác. - Owner: có các khóa học cần duyệt, các thông báo tài chính và phân quản lý nhân viên. + Hệ thống đánh dấu những thông báo đã đọc và chưa đọc.
Pre-Conditions	+ Actor đã đăng nhập vào hệ thống.
Post-Conditions	Không có
Main Flow	1. Actor chọn biểu tượng “Thông báo” trên giao diện chính. 2. Hệ thống hiển thị danh sách các thông báo. 3. Bấm chọn xem chi tiết hệ thống sẽ chuyển tiếp actor đến các trang tương ứng với thông báo
Alternate Flow	Không có.
Exception Flow	+ 2.1. Nếu không có thông báo, hệ thống hiển thị dòng tin “Không có thông báo”.

Bảng 9. Đặc tả Usecase View Notifications - Xem thông báo

View Available Course - Xem khóa học đã đăng ký

[10]	View Available Course
Actor	Student
Trigger	Actor chọn chức năng “Khóa học của tôi” trên giao diện chính.
Description	Usecase này cho phép Actor xem danh sách các khóa học đã đăng ký tham gia.
Pre-Conditions	Actor đã đăng nhập vào hệ thống.

Post-Conditions	Actor có thể vào xem nội dung một khóa học.
Main Flow	1. Actor điều hướng đến Khóa học của tôi trên Trang chủ. 2. Hệ thống hiển thị các khóa học Actor đã ghi danh. 3. Actor chọn một khóa học để học.
Alternate Flow	Actor sử dụng tùy chọn sắp xếp tìm kiếm/lọc để thu hẹp phạm vi.
Exception Flow	Nếu Actor chưa ghi danh vào khóa học nào, hệ thống hiển thị thông tin “Bạn chưa ghi danh vào khóa học nào”.

Bảng 10. Đặc tả Usecase View Available Course - Xem khóa học đã đăng ký

Learn Lectures - Học

[11]	Learn Lectures
Actor	Student
Trigger	Actor muốn bắt đầu hoặc tiếp tục học một khóa học.
Description	Usecase này cho phép actor truy cập và học các bài giảng trong một khóa học đã đăng ký.
Pre-Conditions	Actor đã đăng nhập vào hệ thống. Actor đã đăng ký tham gia khóa học.
Post-Conditions	Tiến độ học tập của actor được cập nhật.
Main Flow	1. Actor chọn một khóa học đã đăng ký ở trang Khóa học của tôi. 2. Hệ thống hiển thị danh sách bài giảng. 3. Actor chọn một bài giảng để học. 4. Hệ thống phát bài giảng.
Alternate Flow	Actor có thể chuyển sang bài giảng khác.
Exception Flow	Không có

Bảng 11. Đặc tả Usecase Learn Lectures - Học

Discussion - Thảo luận

[12]	Discussion
------	------------

Actor	Student
Trigger	Actor vào mục Thảo luận ở trang Bài giảng.
Description	Usecase này cho phép Actor để lại bình luận hoặc trả lời bình luận của các học viên khác để thảo luận về nội dung bài giảng hoặc chia sẻ thêm các kiến thức liên quan. Hệ thống có chức năng kiểm duyệt đánh giá để đảm bảo nội dung tuân thủ các quy định về ngôn từ và không vi phạm chính sách.
Pre-Conditions	Actor đã đăng nhập vào hệ thống. Actor đã ghi danh vào khóa học. Bài giảng đã được xuất bản và truy cập được.
Post-Conditions	Bình luận được lưu trữ trong hệ thống. Thông báo được gửi đến người dùng có liên quan (giảng viên và học viên được đề cập). Hoạt động thảo luận được ghi lại để phân tích.
Main Flow	1. Actor vào mục thảo luận của bài giảng. 2. Hệ thống hiển thị danh sách các bình luận hiện có được sắp xếp theo thứ tự mới nhất trước. 3. Học viên có thể: a. Tạo một chủ đề thảo luận bằng cách nhập nội dung vào ô bình luận và nhấp vào “Đăng”. b. Trả lời bình luận hiện có bằng cách nhấp vào “Trả lời”, nhập nội dung và nhấp vào “Đăng”. c. Thích/bình chọn cho các bình luận khác bằng cách nhấp vào nút thích/bình chọn. d. Sắp xếp bình luận theo mới nhất/cũ nhất hoặc được thích nhất. e. Lọc bình luận để chỉ hiển thị các câu hỏi hoặc thông báo. 4. Hệ thống xác thực nội dung bình luận xem có phù hợp không.

	5. Hệ thống cập nhật danh sách thảo luận và gửi thông báo cho người dùng có liên quan. 6. Học viên có thể chỉnh sửa hoặc xóa bình luận của mình trong một khoảng thời gian nhất định (30 phút) 7. Hệ thống cập nhật danh sách bình luận.
Alternate Flow	3.1. Nếu actor nhập nội dung thảo luận mà không nhấn “Gửi” mà chuyển sang thao tác khác, hệ thống sẽ gửi thông báo “Bạn chưa hoàn tất bình luận. Bạn có muốn rời khỏi trang mà chưa hoàn tất không?”, yêu cầu Actor chọn “Tiếp tục chỉnh sửa” hoặc “Hủy”. 3.2. Nếu học viên muốn định dạng văn bản, họ có thể sử dụng các tùy chọn đánh dấu hoặc văn bản có định dạng (in đậm, in nghiêng, liên kết, khối mã). 3.3. Nếu học viên muốn đưa hình ảnh hoặc tệp đính kèm vào, họ có thể nhấp vào nút tệp đính kèm và tải tệp lên (có giới hạn kích thước). 3.4. Nếu học viên muốn đề cập đến một học viên hoặc giảng viên khác, họ có thể sử dụng "@username" và hệ thống sẽ gợi ý tên người dùng khi họ nhập.
Exception Flow	4.1. Nếu nội dung bình luận chứa các từ bị cấm hoặc vi phạm chính sách của nền tảng, hệ thống sẽ hiển thị thông báo lỗi: "Bình luận của bạn chứa nội dung không phù hợp. Vui lòng sửa đổi và thử lại." 4.2. Nếu hệ thống không lưu được bình luận do sự cố kỹ thuật, hệ thống sẽ hiển thị thông báo lỗi: "Không thể đăng bình luận của bạn. Vui lòng thử lại sau." 4.3. Nếu học viên cố gắng đăng bình luận trống, hệ thống sẽ ngăn không cho gửi và hiển thị thông báo: "Vui lòng nhập nội dung bình luận trước khi đăng."

Bảng 12. Đặc tả Usecase Discussion - Thảo luận

Rate Course - Đánh giá khóa học

[13]	Rate Course
Actor	Student
Trigger	Actor đánh giá sao ở thanh xếp hạng của khóa học.
Description	Usecase này cho phép Actor đánh giá và để lại nhận xét về một khóa học đã ghi danh. Hệ thống có chức năng kiểm duyệt đánh giá để đảm bảo nội dung tuân thủ các quy định về ngôn từ và không vi phạm chính sách.
Pre-Conditions	Actor đã đăng nhập vào hệ thống. Actor đã ghi danh vào khóa học.
Post-Conditions	Đánh giá của Actor được thực hiện thành công và hiển thị trên trang sản phẩm.
Main Flow	1. Actor truy cập vào Khóa học của tôi. 2. Xếp hạng sao ở thẻ hiển thị khóa học. 3. Hệ thống mở giao diện đánh giá, cho phép Actor: - Chọn điểm sao (1 - 5 sao). - Nhập nội dung nhận xét về khóa học. - Chọn “Lưu và tiếp tục”. 4. Hệ thống kiểm tra tính hợp lệ của thông tin. 5. Hệ thống lưu đánh giá vào cơ sở dữ liệu và hiển thị trên trang khóa học.
Alternate Flow	1.1. Nếu actor đã đánh giá khóa học, thì hệ thống sẽ hiện thanh xếp hạng sao của sản phẩm đã được tô tương ứng thay vì những ngôi sao chưa được tô màu. 3.1. Nếu actor nhập thông tin nhưng không nhấn nút “Lưu và tiếp tục” mà muốn chuyển sang thao tác khác, thì hệ thống tự động lưu lại thông tin đánh giá.
Exception Flow	4.1. Nếu actor không đánh giá xếp hạng sao thì hệ thống sẽ yêu cầu actor hãy đánh giá xếp hạng sao trước khi gửi đi.

Bảng 13. Đặc tả Usecase Rate Course - Đánh giá khóa học

Do Quiz - Làm trắc nghiệm đánh giá

[14]	Do Quiz
Actor	Student
Trigger	Học viên nhấp vào liên kết bài kiểm tra (Quiz) trong phần nội dung khóa học hoặc từ tab "Bài tập & Bài kiểm tra".
Description	Use case này cho phép học viên thực hiện bài kiểm tra đánh giá kiến thức trong khóa học. Bài kiểm tra có thể bao gồm các câu hỏi trắc nghiệm, đúng/sai, hoặc tự luận. Một số bài kiểm tra có thể giới hạn thời gian, cho phép làm lại, và có cơ chế xáo trộn câu hỏi hoặc đáp án.
Pre-Conditions	- Học viên đã đăng nhập vào hệ thống. - Đã ghi danh vào khóa học chứa bài kiểm tra. - Bài kiểm tra đã được giảng viên kích hoạt. - Học viên đã hoàn thành các điều kiện tiên quyết (nếu có).
Post-Conditions	- Kết quả được lưu vào hệ thống. - Học viên có thể xem điểm và giải thích (nếu được bật). - Cập nhật tiến độ khóa học nếu học viên đạt yêu cầu. - Hệ thống xóa cache sau khi nộp bài hoặc hết thời gian.
Main Flow	1. Học viên chọn bài kiểm tra. 2. Hệ thống hiển thị mô tả bài kiểm tra (tiêu đề, mô tả, số câu, giới hạn thời gian, điểm qua, số lần làm lại, v.v.). 3. Học viên nhấn "Bắt đầu bài kiểm tra". 4. Hệ thống tạo session và bắt đầu đồng hồ (nếu có giới hạn thời gian). 5. Học viên thực hiện bài kiểm tra: a. Xem và trả lời từng câu hỏi. b. Câu trả lời được lưu tạm thời tự động sau mỗi lần chọn/nhập (qua API). c. Học viên có thể di chuyển giữa các câu hỏi (nếu được phép).

	6. Học viên nhấn "Nộp bài": - Nếu chưa hoàn thành tất cả câu hỏi, có thể có cảnh báo (future enhancement). - Hệ thống xác nhận. 7. Hệ thống chấm điểm tự động các câu lựa chọn; câu tự luận sẽ chờ giảng viên chấm. 8. Hiển thị kết quả: tổng điểm, câu đúng/sai, giải thích (nếu được bật). 9. Nếu được phép (allowRetake = true), học viên có thể nhấn “Làm lại bài” để thực hiện lại.
Alternate Flow	- AF1. Lưu & Thoát: Học viên có thể thoát giữa chừng. Hệ thống lưu cache câu trả lời tạm thời và cho phép quay lại. - AF2. Hết giờ: Nếu có giới hạn thời gian, hệ thống tự động nộp bài sau thời gian hết hạn. - AF3. Làm lại: Nếu được phép, học viên nhấn “Làm lại bài” và quay lại bước 3.
Exception Flow	Mất kết nối internet hoặc thoát bài kiểm tra: - Khi mất kết nối, hệ thống hiển thị cảnh báo và cố gắng phục hồi trạng thái khi kết nối trở lại. - Nếu không thể phục hồi toàn bộ, các câu đã lưu tạm thời trước đó sẽ được dùng để chấm bài (qua Redis cache). - Nếu học viên đóng trình duyệt hoặc rời khỏi trang, hệ thống vẫn duy trì session trong thời gian hiệu lực (qua backend), không tự hủy.

Bảng 14. Đặc tả Usecase Do Quiz - Làm trắc nghiệm đánh giá

Download Attachments - Tải xuống tài liệu học tập

[15]	Download Attachments
Actor	Student
Trigger	Học viên nhấp vào biểu tượng tài nguyên đính kèm trong bài giảng.

Description	Use case này cho phép học viên xem và tải xuống các tài nguyên bổ sung của bài học hoặc toàn bộ khóa học. Các tài nguyên có thể bao gồm tài liệu PDF, file mã nguồn, bài tập thực hành, bảng tính, hình ảnh, hoặc các tệp dữ liệu khác do giảng viên cung cấp để hỗ trợ việc học tập.
Pre-Conditions	Học viên đã đăng nhập vào hệ thống. Học viên đã ghi danh vào khóa học. Khóa học đang trong thời hạn truy cập (nếu khóa học có giới hạn thời gian). Giảng viên đã tải lên tài nguyên cho bài học hoặc khóa học.
Post-Conditions	Tài nguyên được tải xuống thành công vào thiết bị của học viên. Hệ thống ghi lại hoạt động tải xuống trong lịch sử người dùng. Tài nguyên đã tải xuống có thể được truy cập ngoại tuyến (tùy theo loại tệp).
Main Flow	- Học viên truy cập bài học cụ thể hoặc trang tổng quan khóa học. - Học viên nhấp vào tab "Tài nguyên" hoặc biểu tượng tài nguyên đính kèm của bài học. - Hệ thống hiển thị danh sách tài nguyên có sẵn, bao gồm: - Tên tài nguyên - Loại tệp (PDF, ZIP, DOCX, v.v.) - Kích thước tệp - Mô tả ngắn (nếu có) - Ngày tải lên - Học viên nhấp vào nút tải xuống bên cạnh tài nguyên mong muốn. - Hệ thống kiểm tra quyền truy cập và tính khả dụng của tài nguyên. - Hệ thống bắt đầu quá trình tải xuống: - Đối với tệp nhỏ: Tải xuống trực tiếp - Đối với tệp lớn: Hiển thị thanh tiến trình

	7. Trình duyệt lưu tệp vào vị trí đã chỉ định hoặc vị trí tải xuống mặc định. 8. Hệ thống hiển thị thông báo tải xuống thành công.
Alternate Flow	2.1. Học viên có thể nhấp vào biểu tượng "Lưu để xem ngoài tuyến" (cho tài nguyên được hỗ trợ). 4.1. Học viên có thể chọn tài nguyên tải xuống bằng cách: • Nhấn vào nút Tải xuống bên cạnh. • Hệ thống bắt đầu tải xuống tệp.. 4.2. Học viên có thể nhấp vào nút "Tải xuống tất cả". • Hệ thống tạo một tệp nén (ZIP) chứa tất cả tài nguyên của bài học hoặc khóa học. • Hệ thống bắt đầu tải xuống tệp nén. 4.3. Đối với một số loại tệp (PDF, hình ảnh), học viên có thể nhấp vào nút "Xem trước". • Hệ thống hiển thị nội dung tệp trong trình xem tích hợp. • Học viên có thể quyết định tải xuống sau khi xem trước.
Exception Flow	5.1. Nếu tệp tài nguyên không còn tồn tại, hệ thống hiển thị thông báo lỗi. Học viên được thông báo liên hệ với giảng viên hoặc bộ phận hỗ trợ. 7.1. Nếu thiết bị của học viên không có đủ dung lượng lưu trữ, quá trình tải xuống sẽ thất bại. Hệ thống thông báo cho học viên về sự thiếu hụt dung lượng và đề xuất giải phóng không gian. 7.2. Nếu kết nối mạng bị mất trong quá trình tải xuống, hệ thống hiển thị thông báo lỗi. • Hệ thống đề xuất học viên kiểm tra kết nối và thử lại. • Một số trình duyệt có thể tự động tiếp tục quá trình tải xuống khi kết nối được khôi phục.

Bảng 15. Đặc tả Usecase Download Attachments - Tải xuống tài liệu học tập

Take Note - Ghi chú

[16]	Take Note
------	-----------

Actor	Student (Học viên)
Trigger	Học viên nhấp vào biểu tượng ghi chú hoặc nút "Thêm ghi chú" trong khi xem bài giảng.
Description	Use case này cho phép học viên tạo, xem, chỉnh sửa và quản lý các ghi chú cá nhân trong khi học tập. Ghi chú có thể được liên kết với thời điểm cụ thể trong video bài giảng, một slide, hoặc một phần nội dung cụ thể, giúp học viên dễ dàng ghi lại ý tưởng, hiểu biết và câu hỏi trong quá trình học tập.
Pre-Conditions	Học viên đã đăng nhập vào hệ thống. Học viên đã ghi danh vào khóa học. Học viên đang xem một bài giảng hoặc nội dung khóa học.
Post-Conditions	Ghi chú được lưu trữ trong cơ sở dữ liệu và liên kết với bài giảng cụ thể. Học viên có thể truy cập và xem lại ghi chú trong tương lai. Nếu liên kết với thời điểm video, ghi chú được đánh dấu trên thanh tiến trình của trình phát video.
Main Flow	1. Trong khi xem bài giảng, học viên nhấp vào biểu tượng ghi chú hoặc nút "Thêm ghi chú". 2. Hệ thống hiển thị cửa sổ ghi chú với: • Trình soạn thảo văn bản • Thông tin về thời điểm hiện tại trong video (nếu đang xem video) • Thông tin về bài giảng và phần nội dung hiện tại 3. Học viên nhập nội dung ghi chú vào trình soạn thảo. 4. Học viên có thể định dạng ghi chú sử dụng các công cụ định dạng văn bản (đậm, nghiêng, gạch chân, danh sách, v.v.). 5. Học viên nhấp vào nút "Lưu". 6. Hệ thống lưu ghi chú vào cơ sở dữ liệu, liên kết với: • ID của học viên

	• ID của khóa học • ID của bài giảng • Thời điểm trong video (nếu có) 7. Hệ thống hiển thị thông báo xác nhận rằng ghi chú đã được lưu thành công. 8. Hệ thống thêm chỉ báo trên thanh tiến trình video (nếu ghi chú liên kết với video) hoặc trong giao diện bài giảng để chỉ ra vị trí của ghi chú.
Alternate Flow	1. Xem tất cả ghi chú: • Tại bất kỳ lúc nào, học viên có thể nhấp vào tab "Ghi chú của tôi" hoặc biểu tượng ghi chú trong trang khóa học. • Hệ thống hiển thị danh sách tất cả ghi chú của học viên cho khóa học hiện tại, được sắp xếp theo bài giảng và thời gian. • Học viên có thể lọc ghi chú theo bài giảng, chương, hoặc nội dung cụ thể. 2. Chỉnh sửa ghi chú: • Từ danh sách ghi chú hoặc khi xem bài giảng, học viên có thể nhấp vào một ghi chú hiện có. • Hệ thống hiển thị ghi chú trong chế độ chỉnh sửa. • Học viên thực hiện các thay đổi và nhấp vào "Cập nhật". • Hệ thống lưu phiên bản cập nhật của ghi chú. 3. Xóa ghi chú: • Học viên có thể nhấp vào biểu tượng xóa bên cạnh ghi chú. • Hệ thống hiển thị hộp thoại xác nhận. • Học viên xác nhận việc xóa. • Hệ thống xóa ghi chú khỏi cơ sở dữ liệu. 4. Nhảy đến thời điểm ghi chú trong video:

	• Khi xem danh sách ghi chú, học viên có thể nhấp vào đánh dấu thời gian bên cạnh ghi chú liên kết với video. • Hệ thống mở trình phát video của bài giảng tương ứng và di chuyển đến thời điểm chính xác được liên kết với ghi chú..
Exception Flow	1. Lỗi kết nối: • Nếu kết nối internet bị mất trong quá trình lưu ghi chú, hệ thống sẽ lưu ghi chú vào bộ nhớ cục bộ tạm thời. • Khi kết nối được khôi phục, hệ thống sẽ tự động đồng bộ hóa ghi chú với máy chủ. • Hệ thống thông báo cho học viên về tình trạng đồng bộ hóa. 2. Giới hạn độ dài: • Nếu ghi chú vượt quá giới hạn kí tự cho phép, hệ thống sẽ hiển thị cảnh báo và không cho phép lưu cho đến khi ghi chú được rút ngắn. 3. Vấn đề với video: • Nếu trình phát video gặp sự cố khi liên kết ghi chú, hệ thống sẽ vẫn lưu ghi chú nhưng đánh dấu thời điểm liên kết là không chắc chắn, cho phép học viên điều chỉnh sau.

Bảng 16. Đặc tả Usecase Take Note - Ghi chú

Manage Profile - Quản lý thông tin cá nhân

[17]	Manage Profile
Actor	Student, Instructor
Trigger	Actor bấm chọn “Thông tin cá nhân”

Description	Use case này cho phép actor quản lý và cập nhật thông tin cá nhân của mình như họ và tên, email, số điện thoại, địa chỉ và các thông tin liên hệ khác.
Pre-Conditions	Actor đã đăng nhập hệ thống
Post-Conditions	Actor sẽ được xem lại hoặc chỉnh sửa thông tin của bản thân
Main Flow	1. Actor chọn biểu tượng ảnh nền hình tròn trên thanh công cụ 2. Chọn mục “Thông tin cá nhân” để điều hướng đến trang cá nhân 3. Hệ thống hiển thị trang quản lý thông tin cá nhân của actor, ở đây actor có thể chọn chỉnh sửa thông tin cá nhân hoặc thay đổi mật khẩu. 4. Actor có thể chỉnh sửa hoặc cập nhật thông tin của bản thân 5. Thông tin cá nhân của actor được cập nhật thành công trong hệ thống.
Alternate Flow	Actor có thể thay đổi mật khẩu ở trang này khi bấm “Đổi mật khẩu”
Exception Flow	+ Nếu actor chọn “Hủy” thì hệ thống sẽ không thực hiện việc lưu. + Nếu thông tin không hợp lệ, hệ thống hiển thị thông báo lỗi và yêu cầu actor sửa lại thông tin.

Bảng 17. Đặc tả Usecase Manage Profile - Quản lý thông tin cá nhân

Manage Cart - Quản lý giỏ hàng

[18]	Manage Cart
Actor	Student
Trigger	Actor bấm chọn biểu tượng giỏ hàng
Description	Usecase này cho phép Actor chuyển hướng đến trang giỏ hàng
Pre-Conditions	Actor đã đăng nhập hệ thống
Post-Conditions	Actor xem và điều chỉnh giỏ hàng của mình

Main Flow	1. Actor bấm biểu tượng giỏ hàng 2. Trang giỏ hàng chứa các khóa học mà Actor đã chọn sẽ hiện ra 3. Actor có thể xóa khóa học ra giỏ hàng của mình 4. Actor chọn tiếp tục thanh toán để chuyển hướng đến trang xác nhận thanh toán
Alternate Flow	Không có
Exception Flow	Không có

Bảng 18. Đặc tả Usecase Manage Cart - Quản lý giỏ hàng

Cart Payment - Thanh toán giỏ hàng

[19]	Cart Payment
Actor	Student
Trigger	Actor bấm chọn “tiến hành thanh toán”
Description	Usecase cho phép Actor điều hướng đến trang xác nhận thanh toán
Pre-Conditions	Actor đã đăng nhập hệ thống, giỏ hàng không được trống
Post-Conditions	Actor hoàn thành thanh toán và có thể học các khóa học đã mua
Main Flow	1. Actor chọn phương thức thanh toán 2. Chọn tiến hành thanh toán để hoàn tất quá trình mua khóa học
Alternate Flow	+ Có thể chọn coupon để giảm giá
Exception Flow	+ Nếu thanh toán thất bại (tiền không đủ/hủy thanh toán) sẽ hiện thông báo thanh toán thất bại và điều hướng về trang chủ

Bảng 19. Đặc tả Usecase Cart Payment - Thanh toán giỏ hàng

Manage Teaching Courses - Quản lý khóa học giảng dạy

[20]	Manage Teaching Courses
Actor	Instructor
Trigger	Giảng viên chọn chức năng "Quản lý khóa học giảng dạy" trên dashboard cá nhân.
Description	Giảng viên quản lý nội dung khóa học của mình, bao gồm thêm bài giảng mới, chỉnh sửa nội dung hiện tại, hoặc xóa bài giảng.

Pre-Conditions	Giảng viên đã tạo ít nhất một khóa học và có quyền truy cập vào khóa học đó.
Post-Conditions	Nội dung khóa học được cập nhật thành công.
Main Flow	1. Giảng viên đăng nhập và chọn "Quản lý khóa học giảng dạy". 2. Hệ thống hiển thị danh sách các khóa học của giảng viên. 3. Giảng viên thêm/xóa/sửa bài giảng hoặc nội dung khóa học. 4. Hệ thống lưu thay đổi và hiển thị thông báo thành công.
Alternate Flow	3.1. Instructor có thể xem trước nội dung khóa học sau khi chỉnh sửa trước khi lưu thay đổi.
Exception Flow	4.1. Nếu giảng viên không điền đủ thông tin bắt buộc khi thêm/sửa bài giảng mới, hệ thống hiển thị thông báo lỗi.

Bảng 20. Đặc tả Usecase Quản lý khóa học giảng dạy

View Course Revenue - Xem doanh thu khóa học

[21]	View Course Revenue
Actor	Instructor
Trigger	Instructor chọn chức năng "Xem doanh thu khóa học" trên dashboard cá nhân.
Description	Giảng viên xem báo cáo doanh thu từ các khóa học của mình, bao gồm chi tiết về thu nhập và trạng thái thanh toán.
Pre-Conditions	Giảng viên đã có ít nhất một khóa học được xuất bản trên hệ thống.
Post-Conditions	Doanh thu được hiển thị đầy đủ và chính xác cho giảng viên.
Main Flow	1. Instructor đăng nhập vào tài khoản và chọn "Xem doanh thu khóa học". 2. Hệ thống truy xuất dữ liệu doanh thu từ cơ sở dữ liệu và hiển thị chi tiết bao gồm tổng thu nhập, số tiền chờ xử lý, và số tiền đã thanh toán.

	3. Instructor xem chi tiết doanh thu theo từng khóa học hoặc tổng hợp theo thời gian.
Alternate Flow	3.1. Instructor có thể lọc doanh thu theo ngày/tháng/năm hoặc theo từng khóa học cụ thể.
Exception Flow	2.1. Nếu không có dữ liệu doanh thu, hệ thống hiển thị thông báo "Hiện chưa có doanh thu nào".

Bảng 21. Đặc tả Usecase View Course Revenue - Xem doanh thu khóa học

Review Content Courses - Kiểm duyệt khóa học

[22]	Review Content Courses
Actor	Staff
Trigger	Nhân viên truy cập trang quản lý các khóa học cần duyệt từ dashboard. Hệ thống gửi thông báo cho nhân viên khi có khóa học mới cần được duyệt.
Description	Nhân viên kiểm tra và duyệt nội dung các khóa học mới hoặc các bản cập nhật khóa học trước khi được xuất bản trên nền tảng, đảm bảo rằng chúng tuân thủ các tiêu chuẩn chất lượng, chính sách của nền tảng, và không vi phạm bản quyền.
Pre-Conditions	Giảng viên đã gửi khóa học mới hoặc bản cập nhật khóa học để duyệt. Nhân viên đã đăng nhập vào hệ thống.
Post-Conditions	Khóa học được phê duyệt và xuất bản trên nền tảng hoặc bị từ chối với phản hồi chi tiết gửi lại cho giảng viên. Trạng thái duyệt khóa học được cập nhật trong hệ thống.
Main Flow	1. Nhân viên đăng nhập vào hệ thống và truy cập trang quản lý các khóa học cần duyệt. 2. Nhân viên chọn một khóa học để xem chi tiết nội dung. 3. Nhân viên kiểm tra nội dung khóa học theo các tiêu chí quy định. 4. Nhân viên đưa ra quyết định duyệt hoặc từ chối khóa học.

	5. Hệ thống cập nhật trạng thái của khóa học và thông báo cho giảng viên.
Alternate Flow	4.1. Nhân viên có thể yêu cầu giảng viên chỉnh sửa nội dung và gửi lại để duyệt.
Exception Flow	4.1. Nếu nhân viên quên cung cấp lý do từ chối khi từ chối khóa học, hệ thống sẽ yêu cầu cung cấp lý do trước khi tiếp tục.

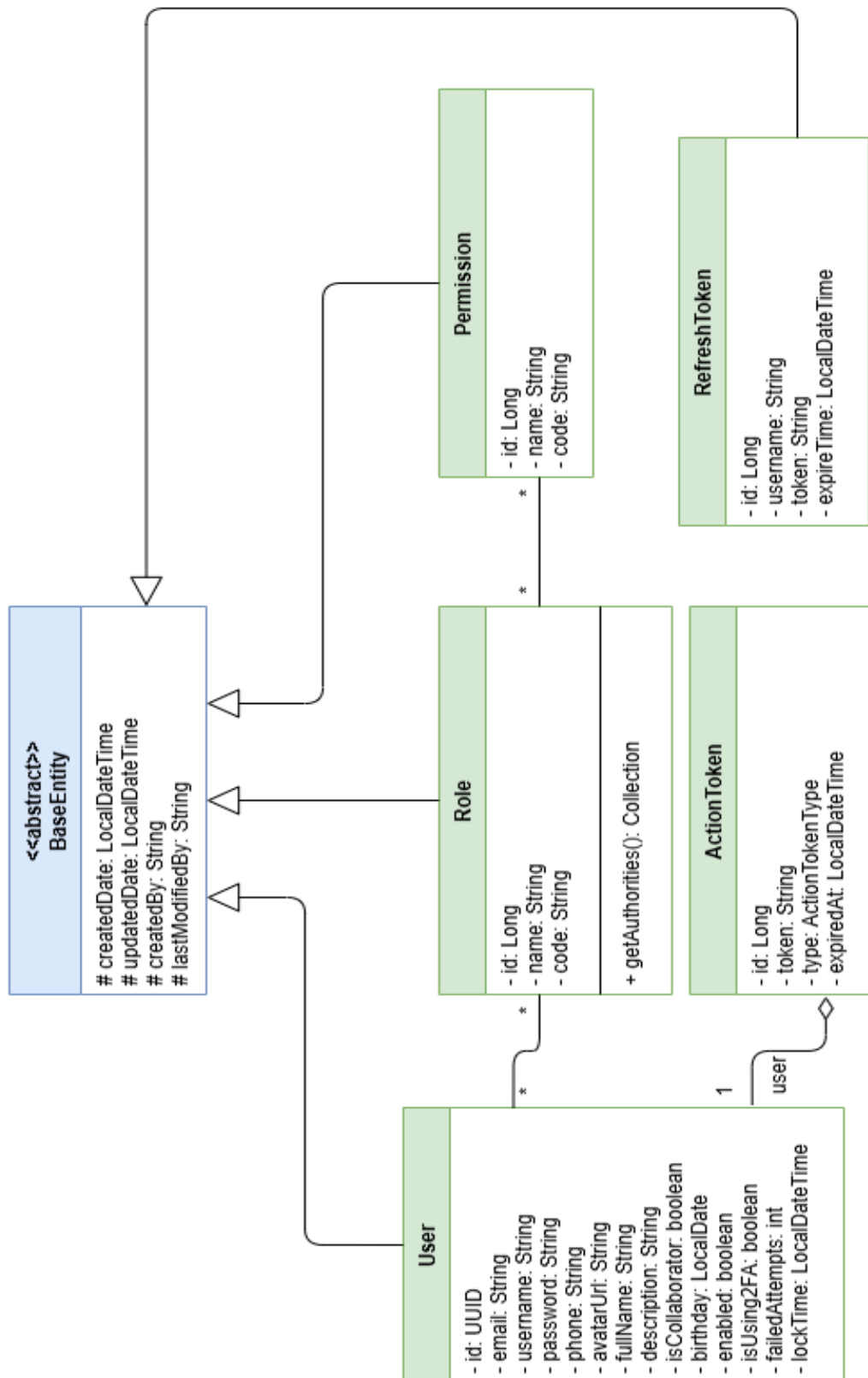
Bảng 22. Đặc tả Usecase Review Content Courses - Kiểm duyệt khóa học

Manage Payments - Quản lý giao dịch

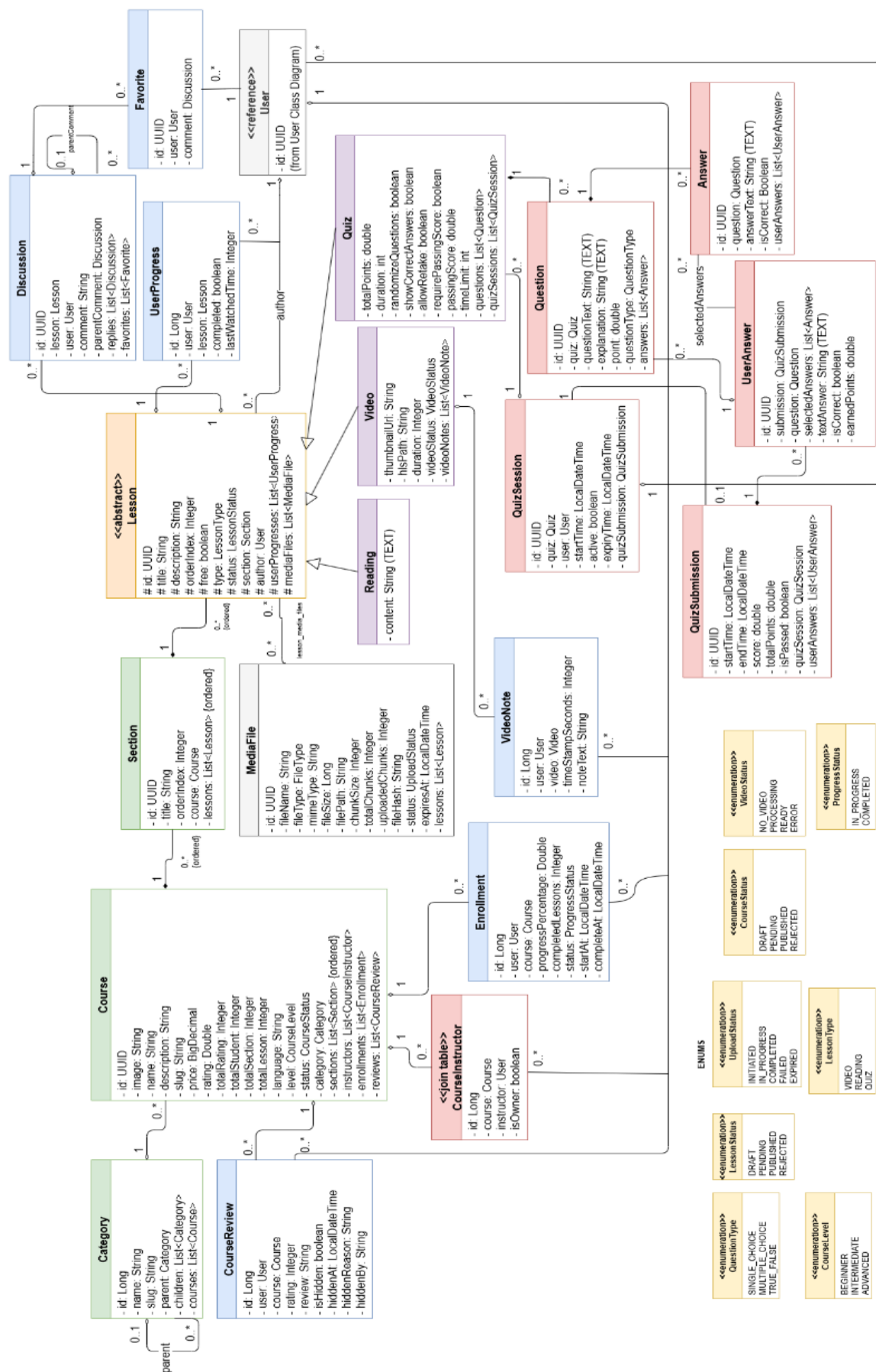
[23]	Manage Payments
Actor	Admin
Trigger	Admin chọn chức năng “Giao dịch” từ menu quản trị hệ thống.
Description	Usecase này cho phép Admin quản lý các giao dịch thanh toán của hệ thống, bao gồm xem chi tiết giao dịch, xử lý các giao dịch lỗi và cập nhật trạng thái thanh toán.
Pre-Conditions	Admin đã đăng nhập vào hệ thống quản trị.
Post-Conditions	Thông tin giao dịch được cập nhật và lưu trữ chính xác theo từng trạng thái thanh toán.
Main Flow	1. Admin truy cập vào chức năng “Manage Payments”. 2. Hệ thống hiển thị danh sách các giao dịch thanh toán. 3. Admin chọn một giao dịch để xem chi tiết. 4. Admin cập nhật trạng thái thanh toán hoặc xử lý giao dịch lỗi nếu có. 5. Hệ thống lưu lại thay đổi và hiển thị thông báo thành công.
Alternate Flow	Nếu không có giao dịch mới, hệ thống hiển thị thông báo “Không có giao dịch mới”.
Exception Flow	Nếu thông tin giao dịch không hợp lệ khi cập nhật, hệ thống hiển thị thông báo lỗi “Thông tin giao dịch không hợp lệ” và yêu cầu nhập lại.

Bảng 23. Đặc tả Usecase Manage Payments - Quản lý giao dịch

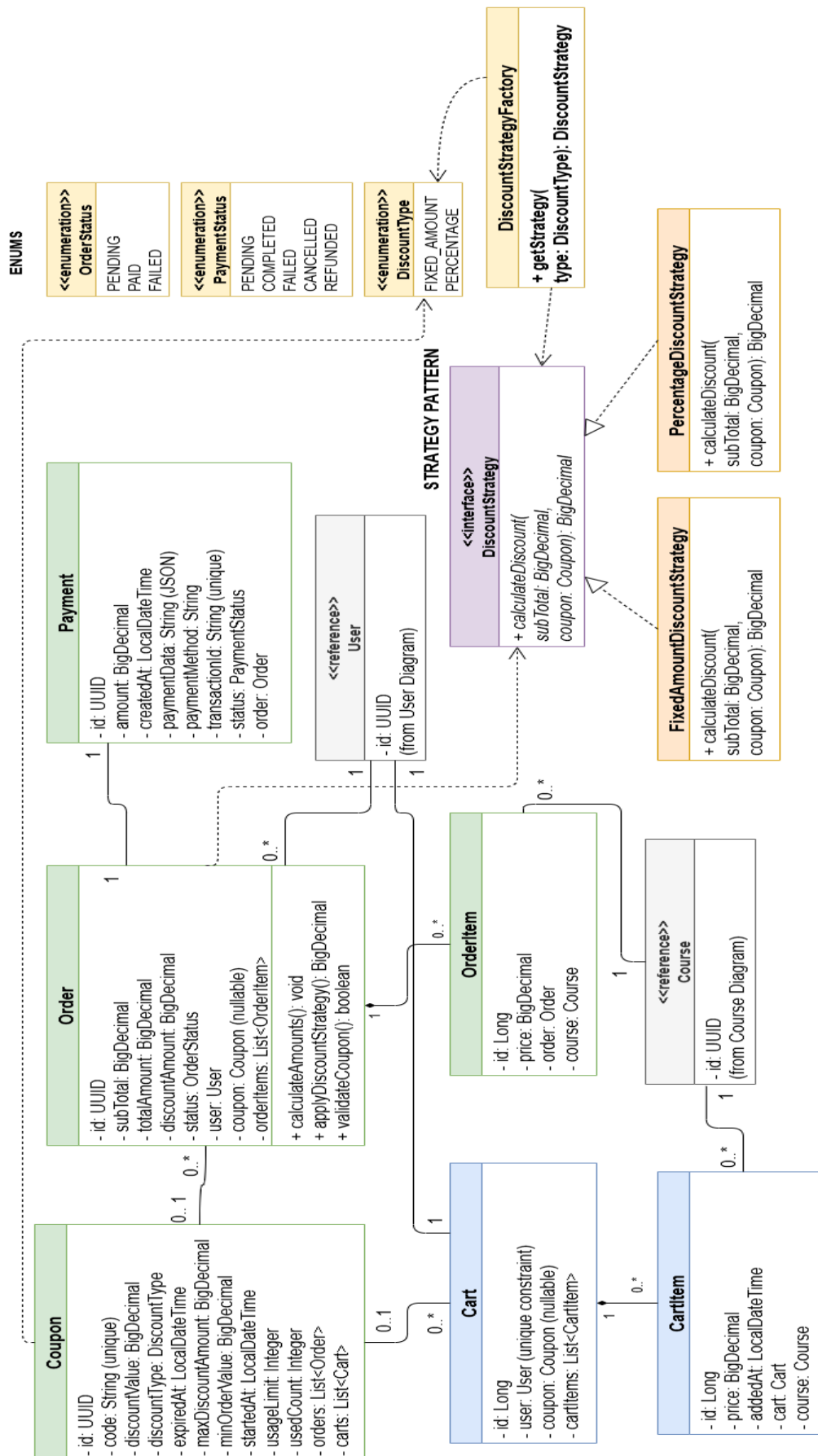
4.2. LƯỢC ĐỒ LỚP



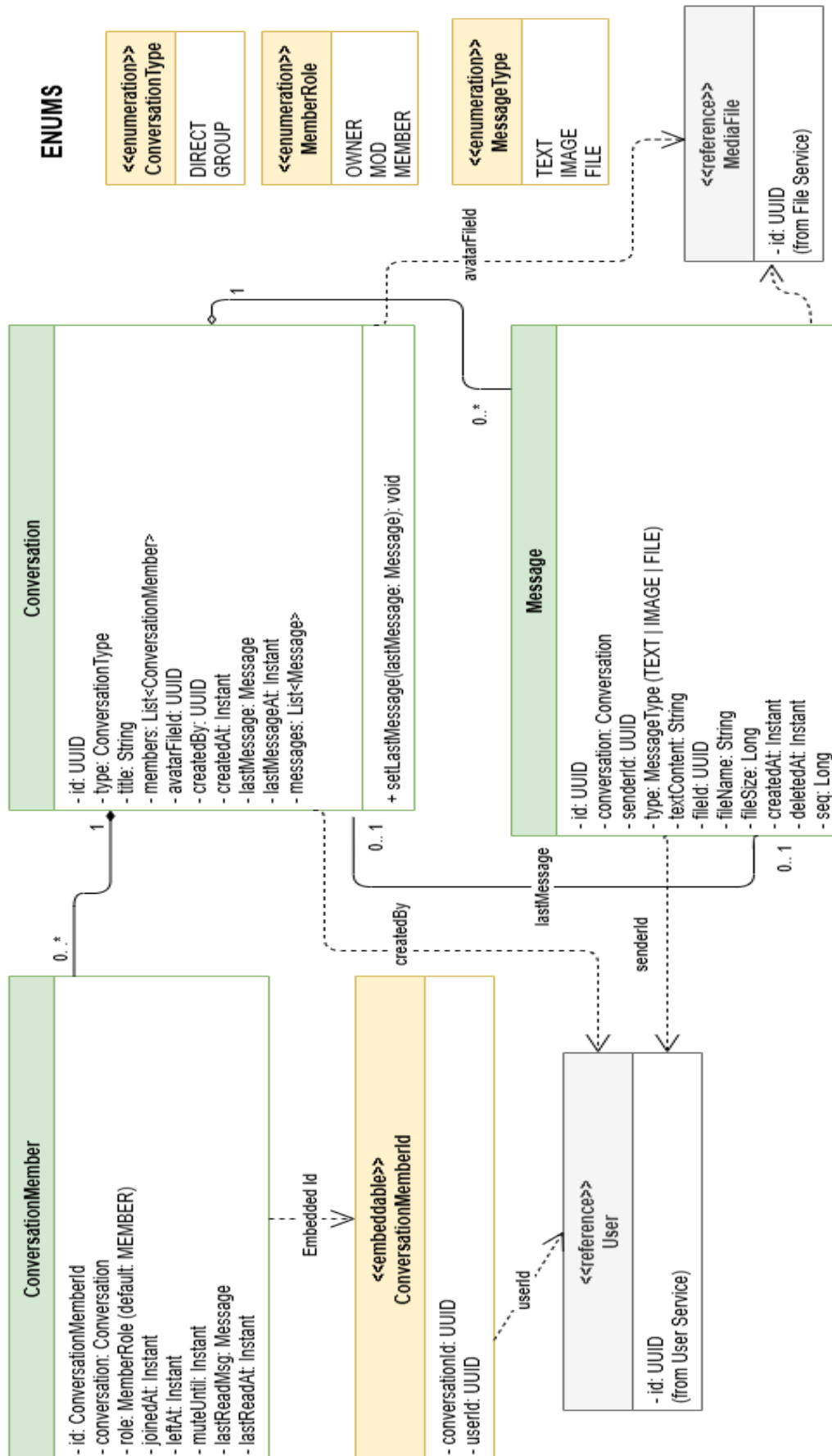
Hình 10. Class Diagram User Domain



Hình 11. Class Diagram Course Domain



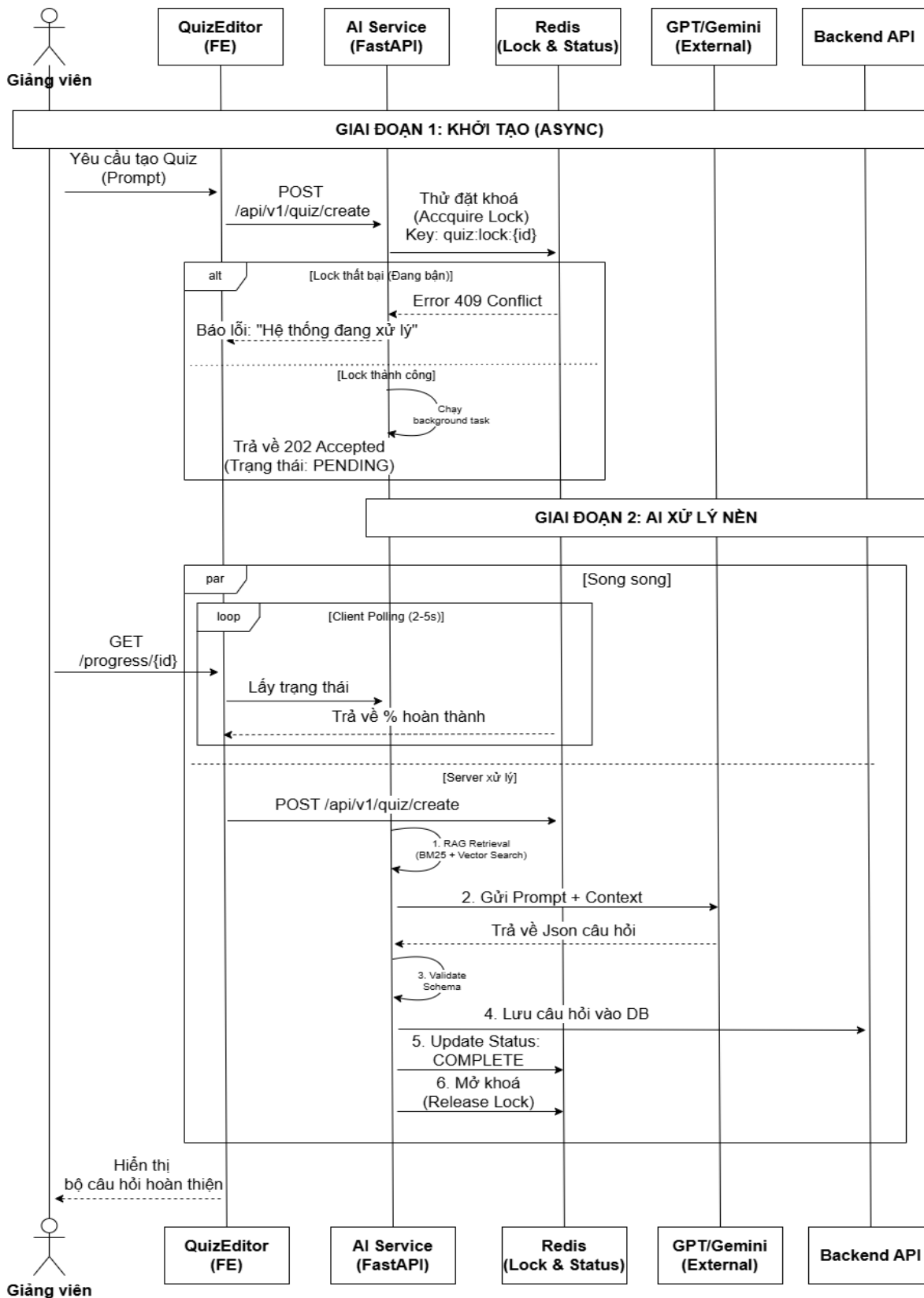
Hình 12. Class Diagram Payment Domain



Hình 13. Class Diagram Chat Domain

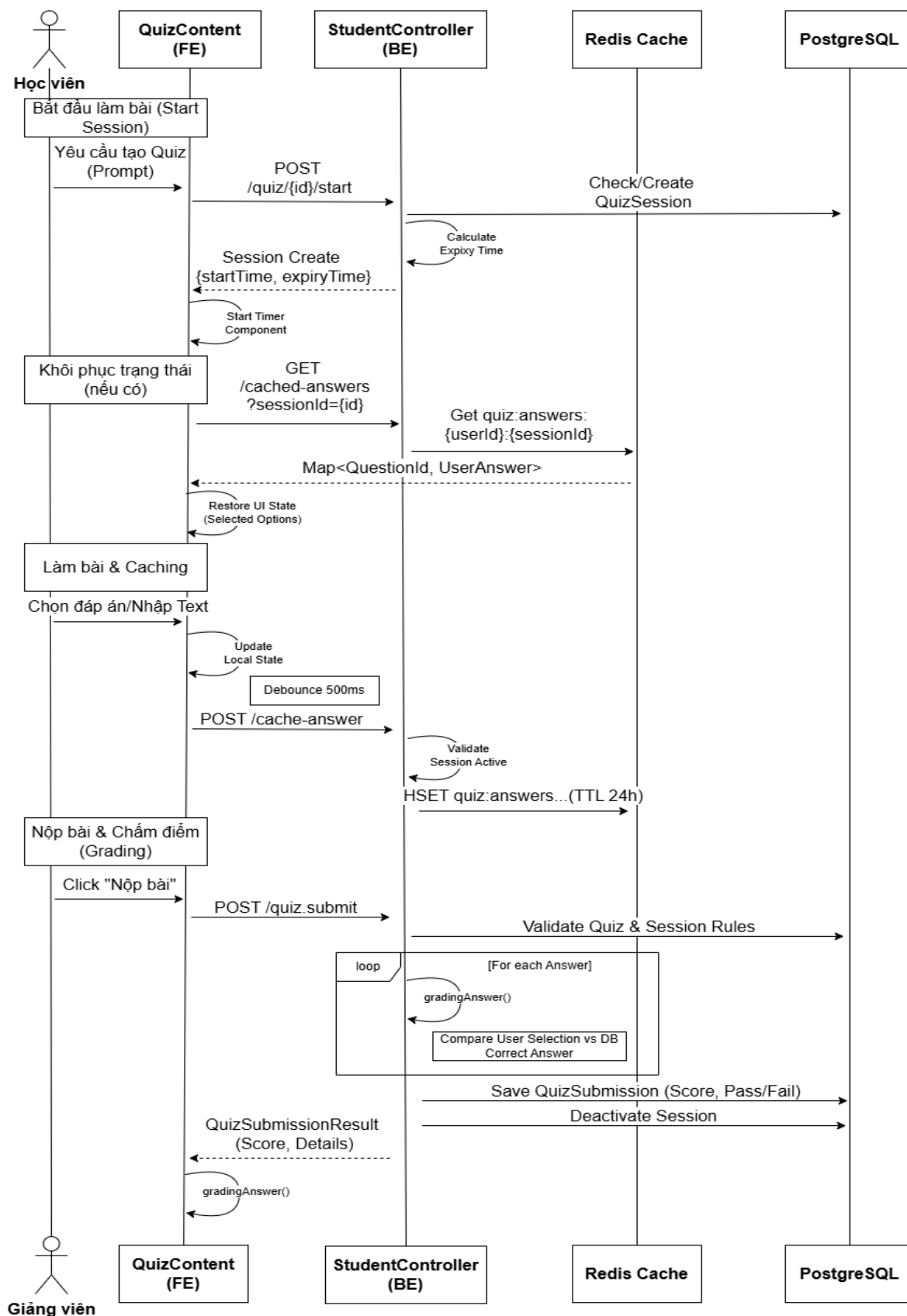
4.3. LƯỢC ĐỒ TUẦN TỰ CỦA MỘT SỐ CHỨC NĂNG CHÍNH

4.3.1. Lược đồ tuần tự luồng Tạo Quiz tự động



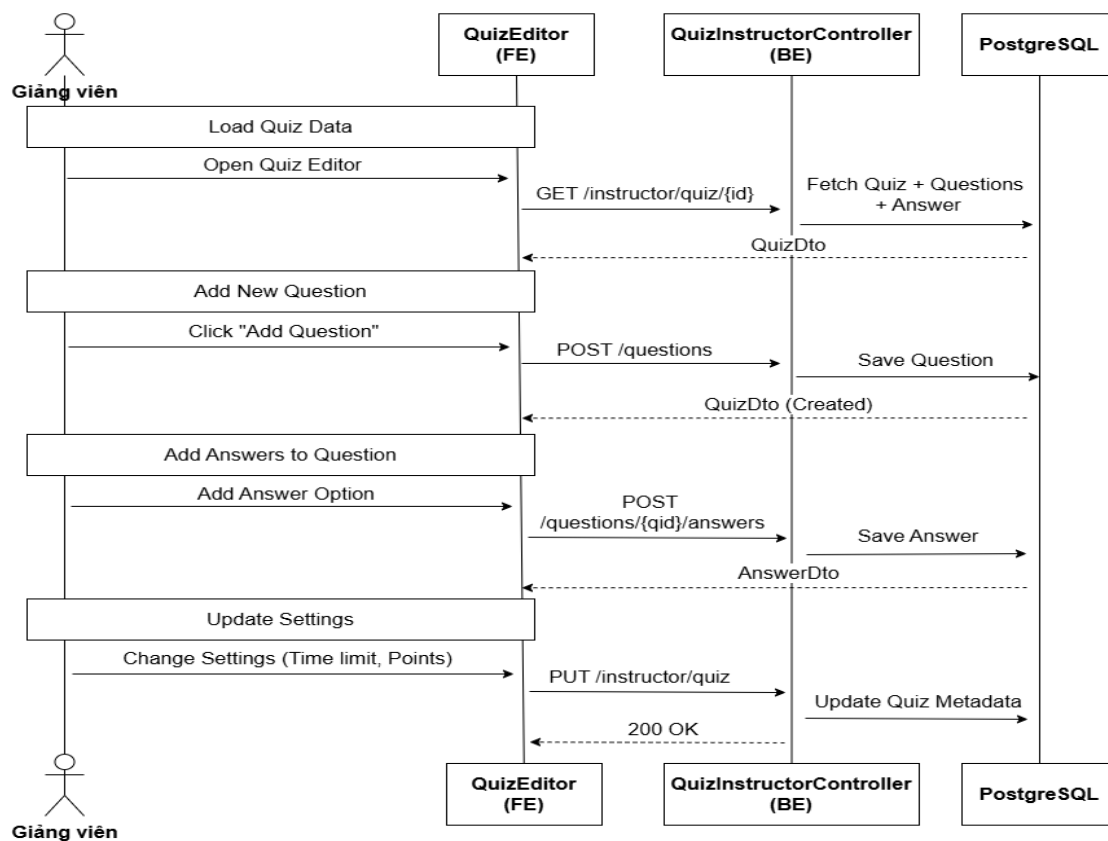
Hình 14. Lược đồ tuần tự Luồng tạo Quiz Tự động

4.3.2. Lược đồ tuần tự luồng Học viên làm và nộp bài



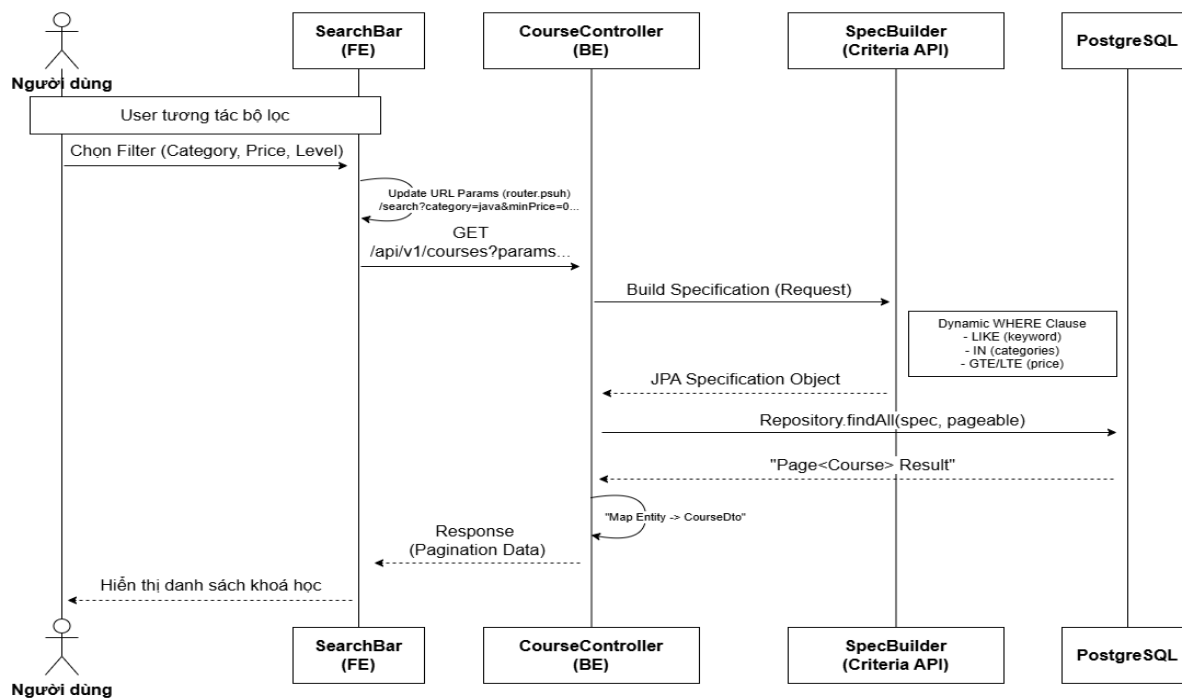
Hình 15. Lược đồ tuần tự luồng Học viên làm và nộp bài

4.3.3. Lược đồ tuần tự luồng Tạo Quiz thủ công



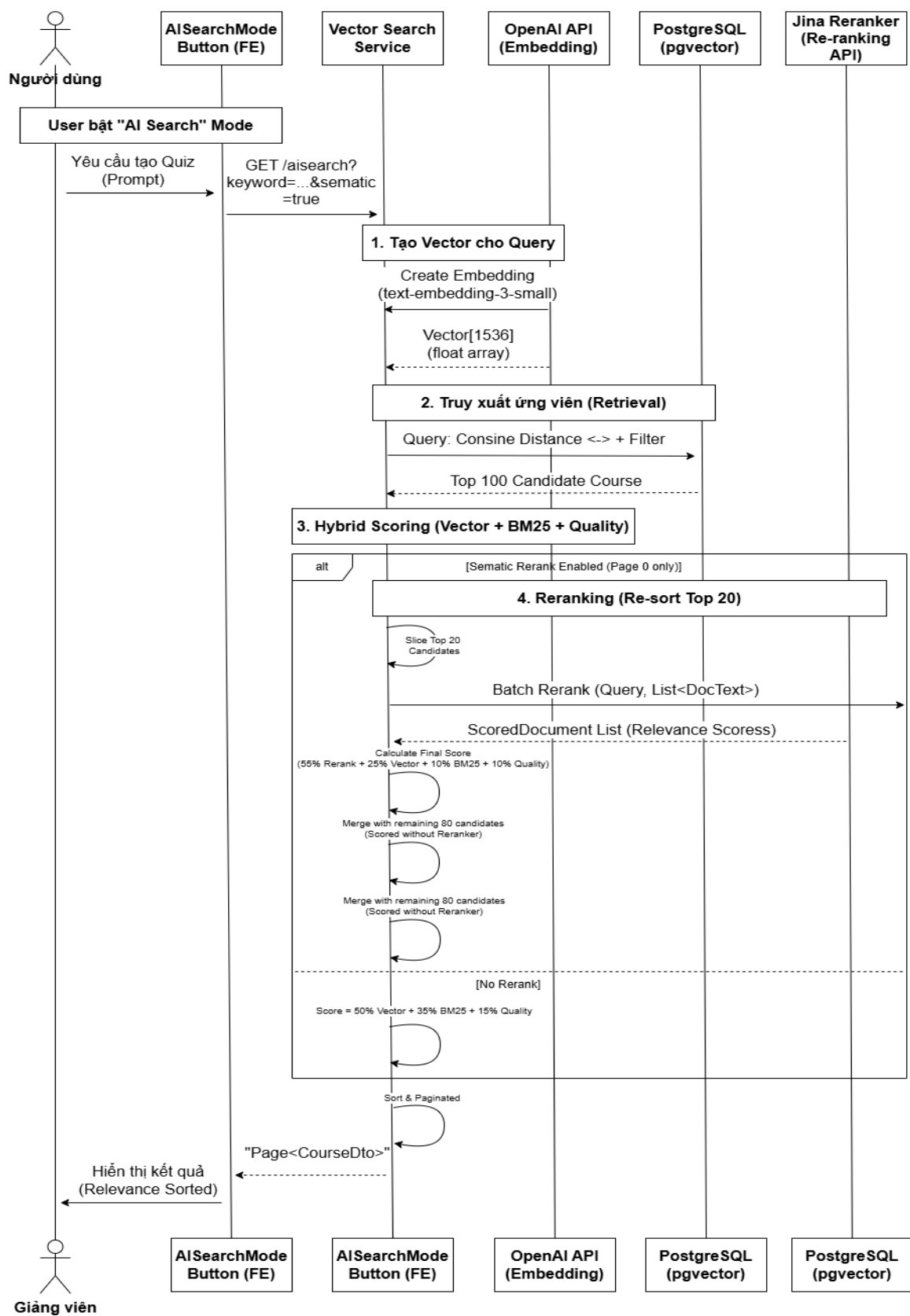
Hình 16. Lược đồ tuần tự luồng Tạo Quiz thủ công

4.3.4. Lược đồ tuần tự luồng Tìm kiếm truyền thống



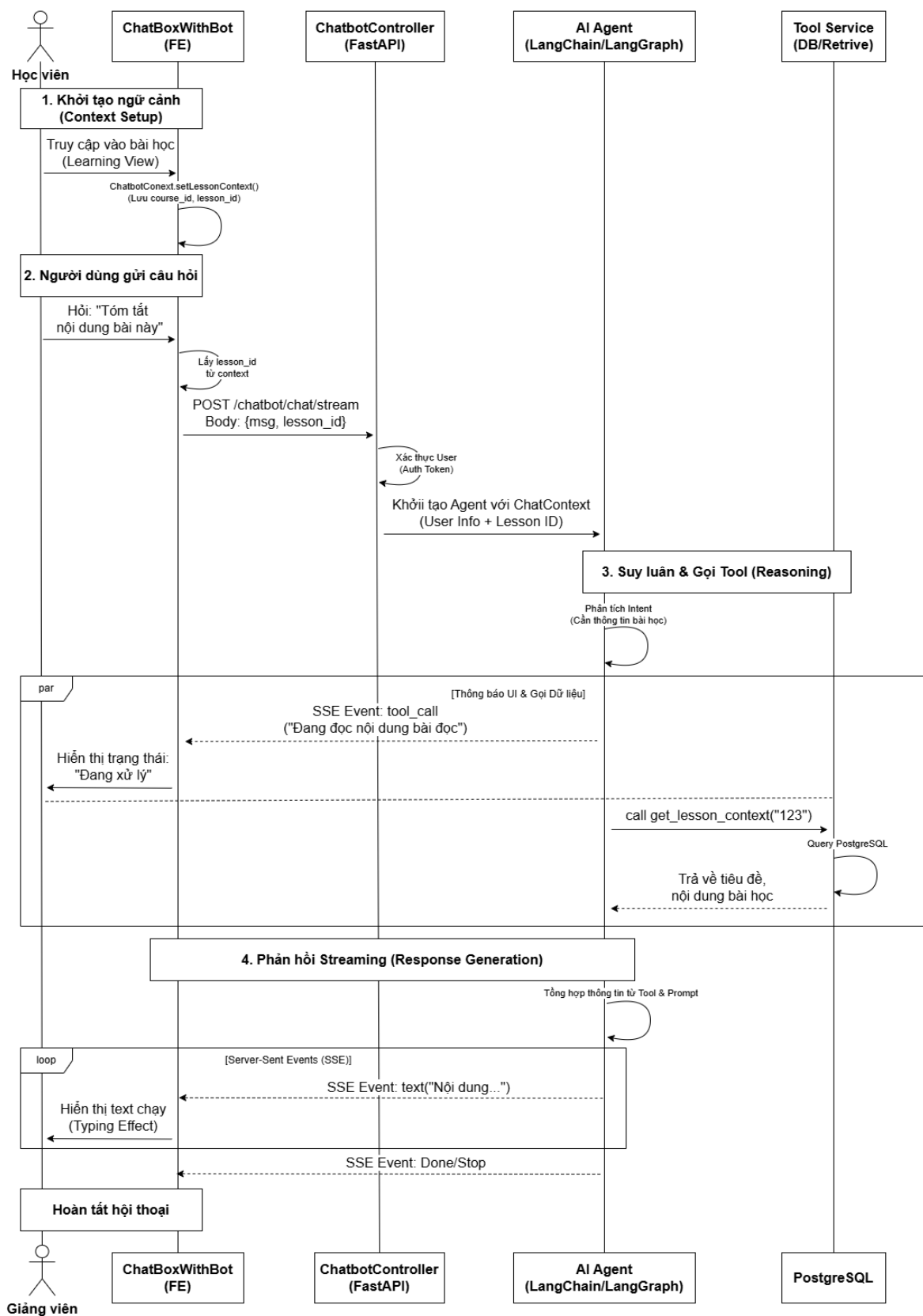
Hình 17. Lược đồ tuần tự luồng Tìm kiếm truyền thống

4.3.5. Lược đồ tuần tự luồng Tìm kiếm AI nâng cao



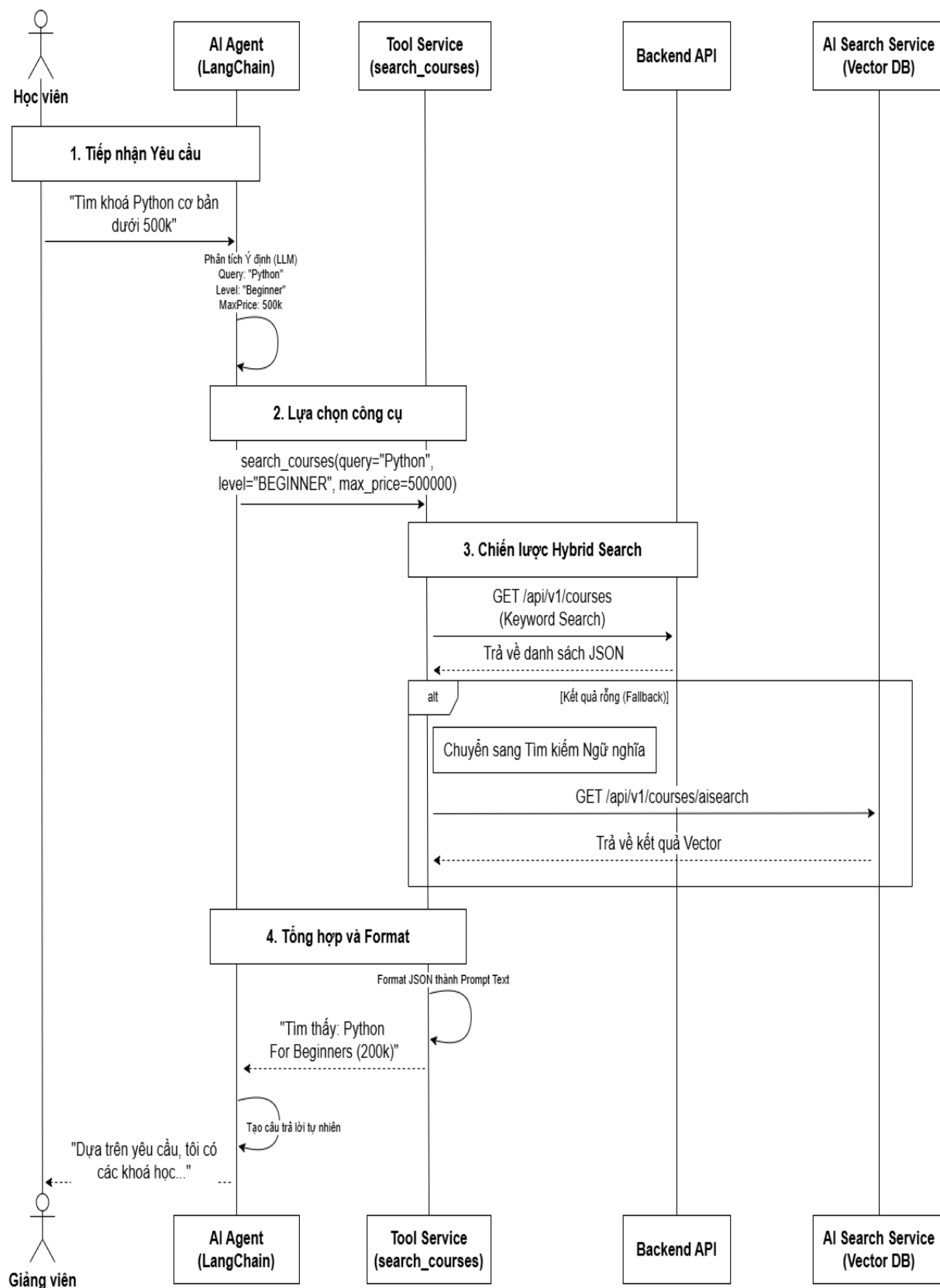
Hình 18. Lược đồ tuần tự chức năng luồng Tìm kiếm AI nâng cao

4.3.6. Lược đồ tuần tự luồng Học viên hỏi Chatbot về nội dung bài đang học



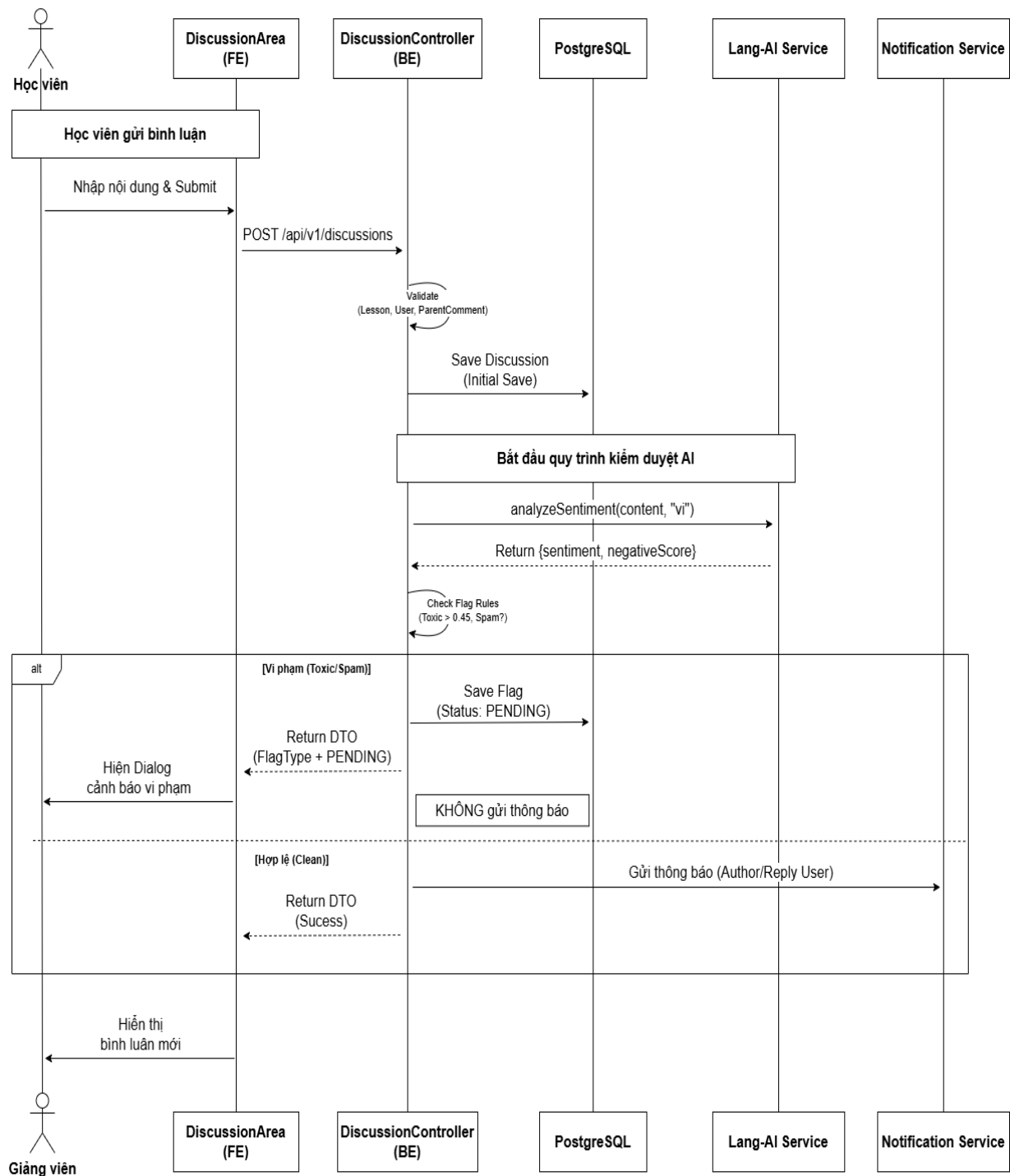
Hình 19. Lược đồ tuần tự chức năng luồng Chatbot hỗ trợ học tập

4.3.7. Lược đồ tuần tự luồng Chatbot gợi ý Khoá học



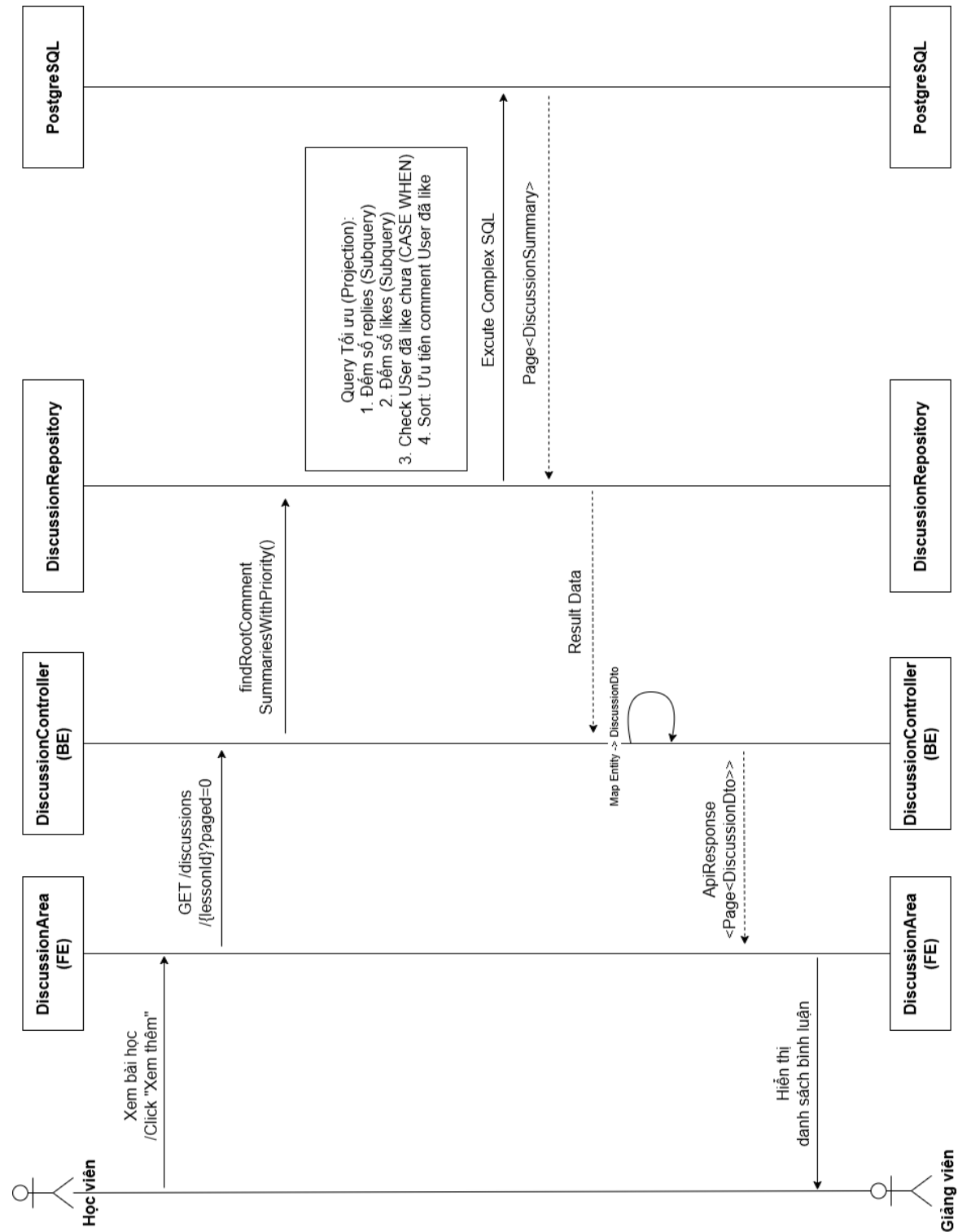
Hình 20. Lược đồ tuần tự luồng Chatbot gợi ý Khoá học

4.3.8. Lược đồ tuần tự luồng **Luồng Tạo Bình Luận & Kiểm Duyệt Tự Động**



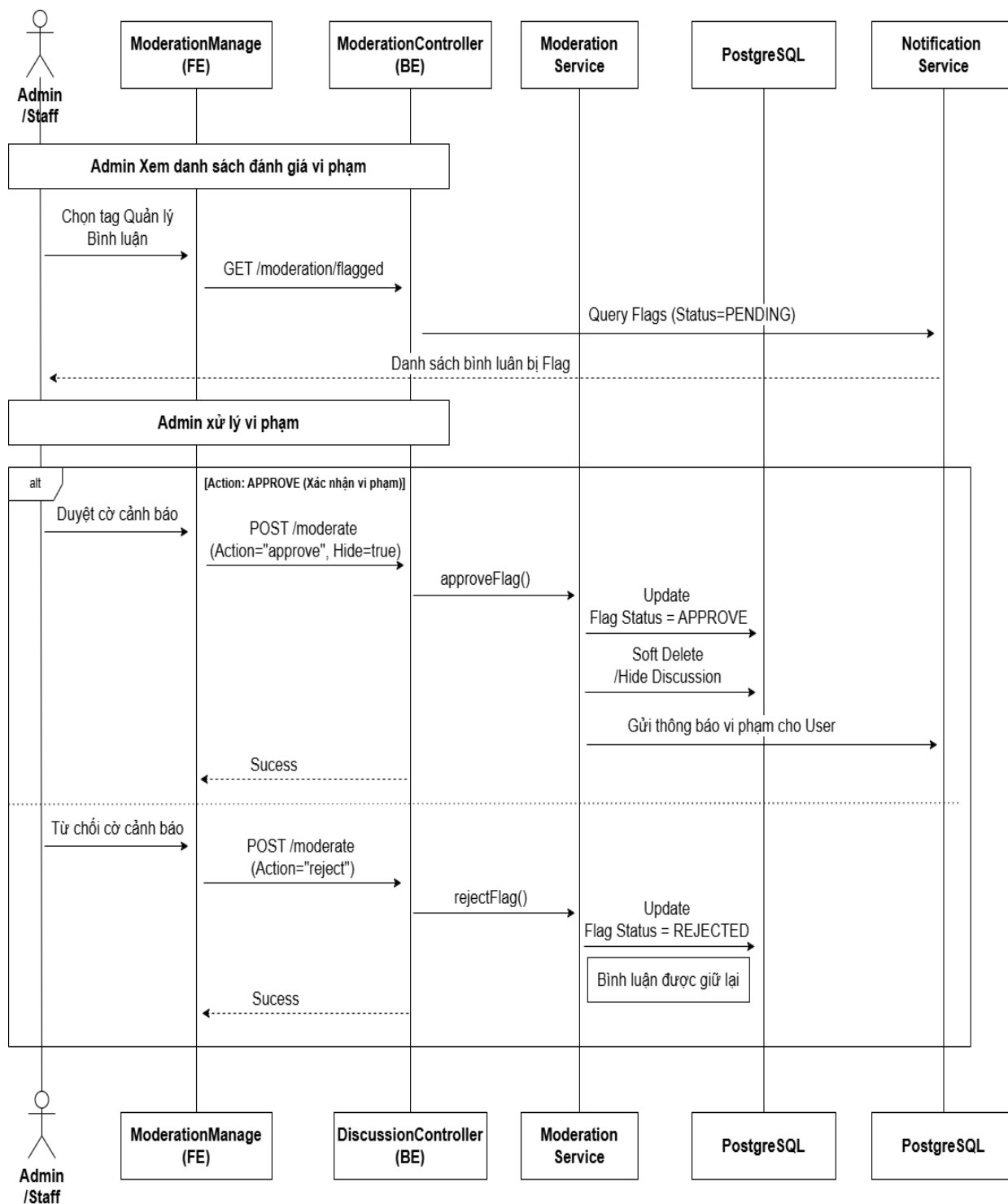
Hình 21. Lược đồ tuần tự luồng **Luồng Tạo bình luận & kiểm duyệt tự động**

4.3.9. Lược đồ tuần tự luồng Lấy danh sách bình luận



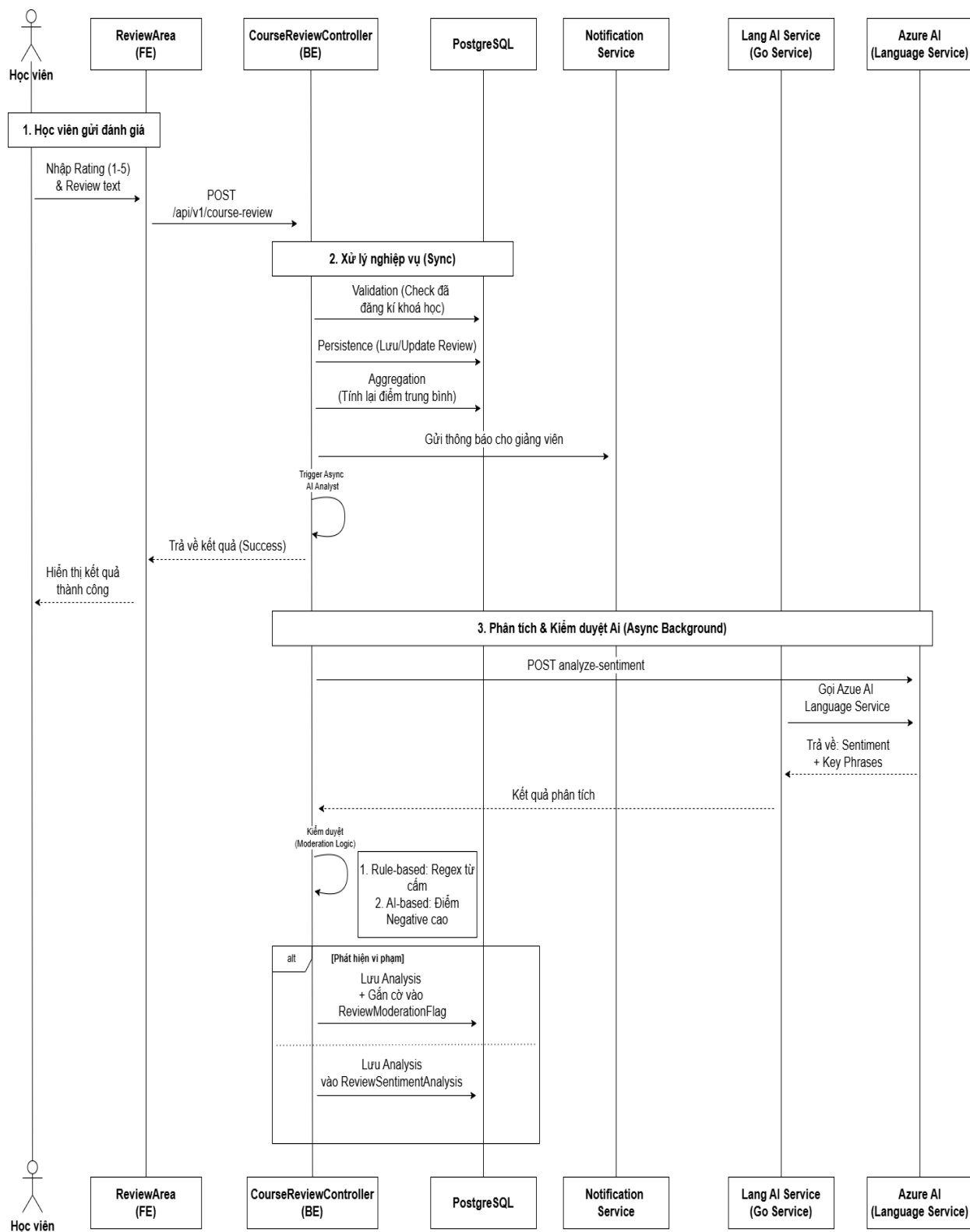
Hình 22. Lược đồ tuần tự luồng Lấy Danh Sách Bình Luận

4.3.10. Lược đồ tuần tự luồng Quản Lý Kiểm Duyệt (Admin/Staff)



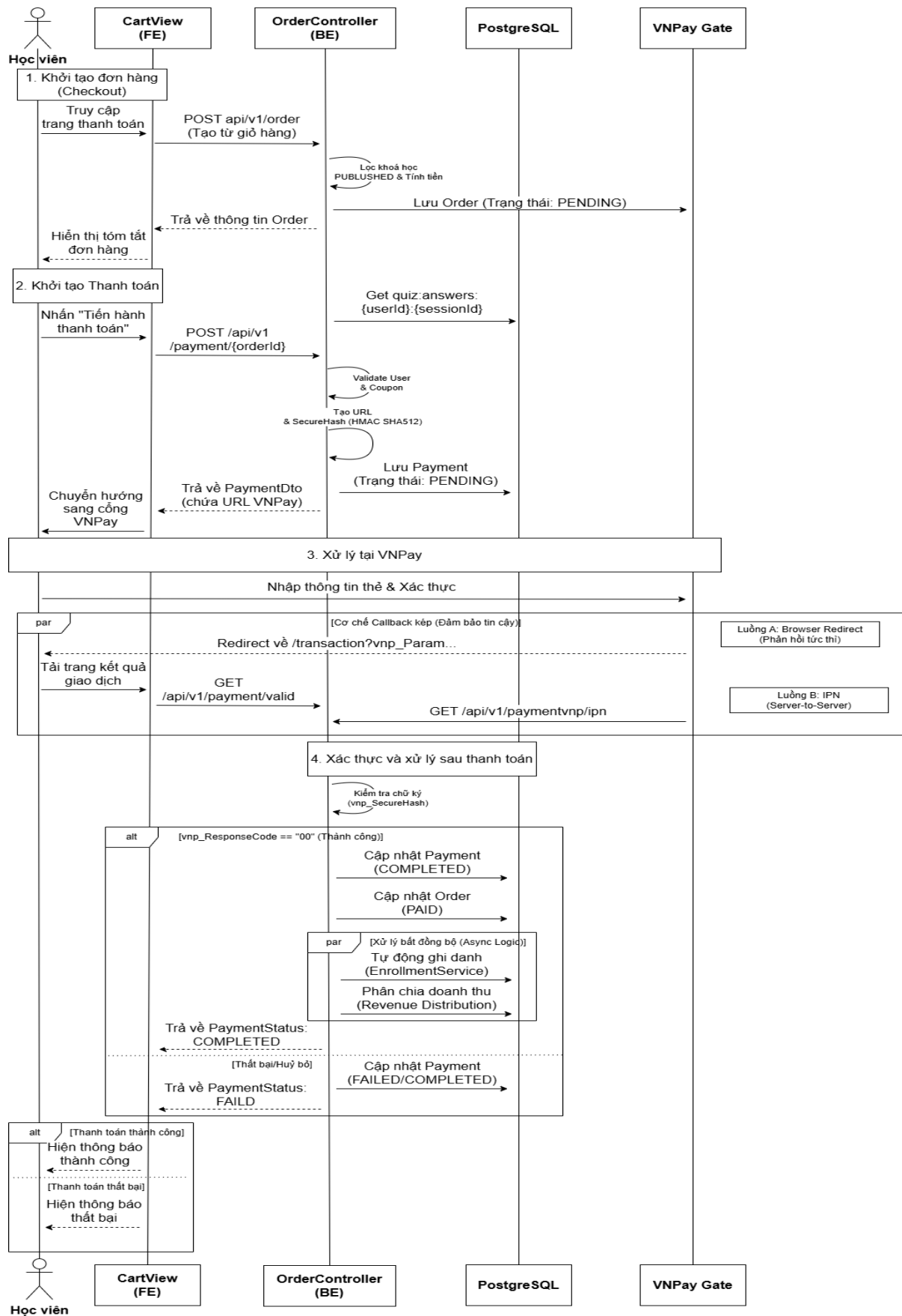
Hình 23. Lược đồ tuần tự luồng Quản Lý Kiểm Duyệt (Admin/Staff)

4.3.11. Lược đồ tuần tự luồng Xử lý đánh giá



Hình 24. Lược đồ tuần tự luồng Xử lý đánh giá

4.3.12. Lược đồ tuần tự luồng Xử lý thanh toán – VNPay

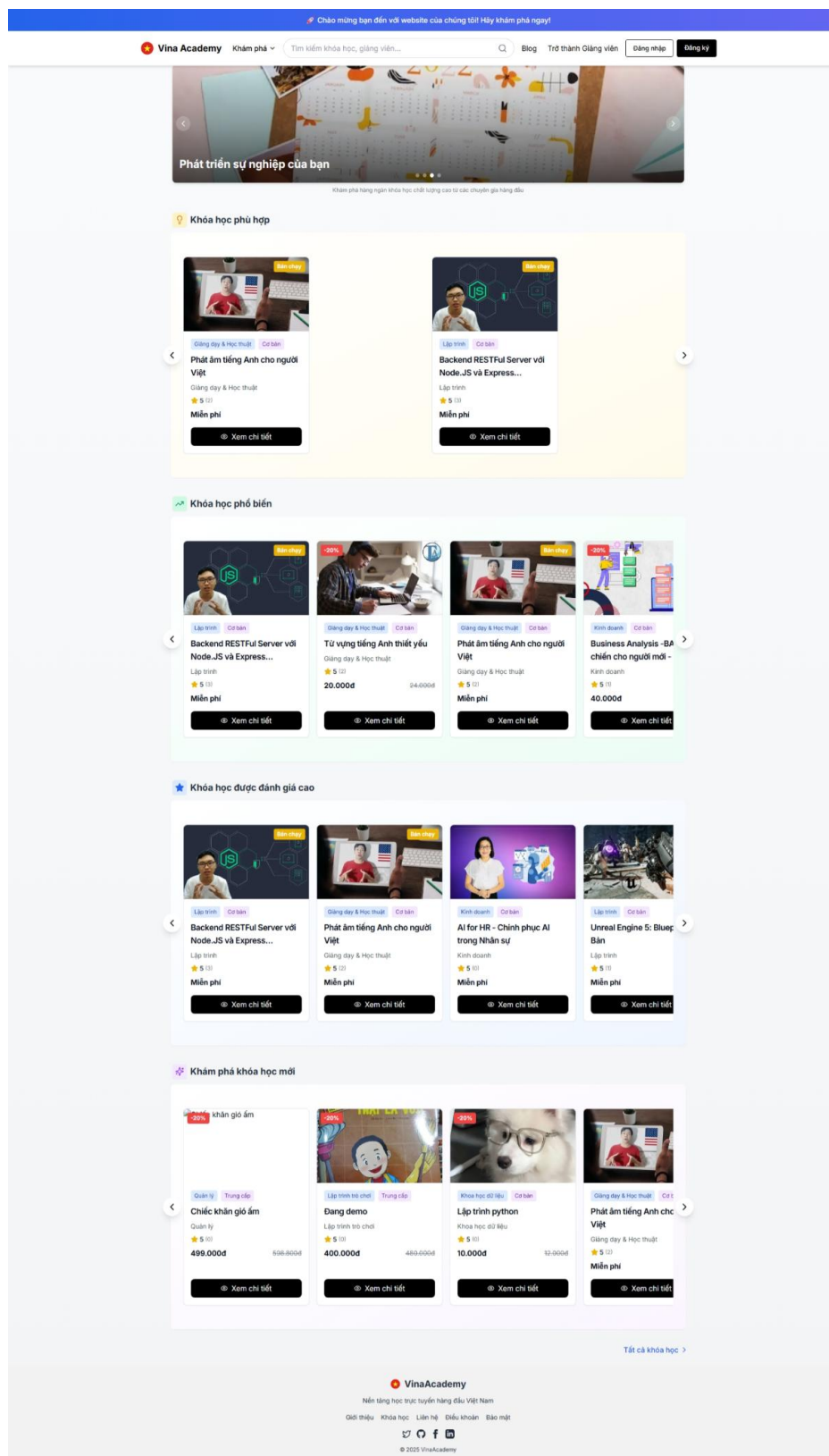


Hình 25. Lược đồ tuần tự luồng Xử lý thanh toán - VNPay

4.4. THIẾT KẾ GIAO DIỆN

Xem thêm tại: [Thiết kế giao diện](#)

4.4.1. Trang chủ



Hình 26. Giao diện trang chủ

4.4.2. Đăng ký

The screenshot shows the registration page of VINA Academy. At the top, there is a navigation bar with the VINA Academy logo, a search bar, and links for 'Blog', 'Trở thành Giảng viên', 'Đăng nhập', and 'Đăng ký'. The main content area features a large illustration of two people interacting with a large screen. Below the illustration, there is a registration form with the following fields: 'Tên đăng nhập' (Username), 'Họ và tên' (Full name), 'Email', 'Mật khẩu' (Password), and 'Xác nhận mật khẩu' (Confirm password). Each field has a placeholder text indicating what to enter. A 'Đăng ký' (Register) button is at the bottom of the form. Below the button, there is a link for 'Đã có tài khoản? Đăng nhập' (Already have an account? Log in). At the bottom of the page, there is a footer with the VINA Academy logo, a description of the platform, and social media links.

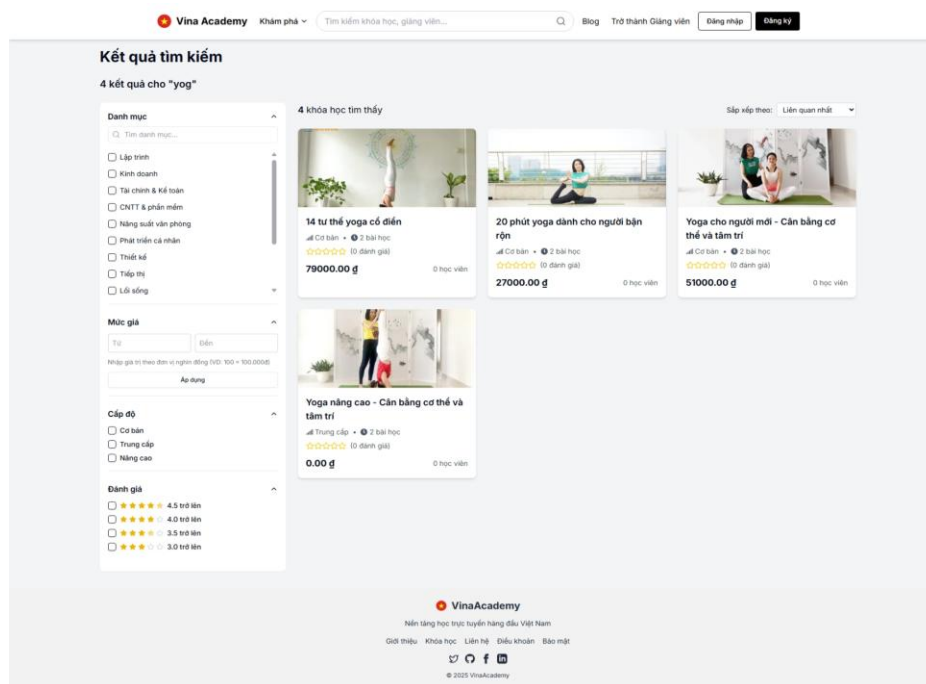
Hình 27. Giao diện đăng ký

4.4.3. Đăng nhập

The screenshot shows the login page of VINA Academy. At the top, there is a navigation bar with the VINA Academy logo, a search bar, and links for 'Blog', 'Trở thành Giảng viên', 'Đăng nhập', and 'Đăng ký'. The main content area features a large illustration of a person using a laptop. Below the illustration, there is a login form with the following fields: 'Email' and 'Mật khẩu' (Password). Each field has a placeholder text indicating what to enter. A 'Đăng nhập' (Log in) button is at the bottom of the form. Below the button, there is a link for 'Quên mật khẩu?' (Forgot password?). At the bottom of the page, there is a footer with the VINA Academy logo, a description of the platform, and social media links.

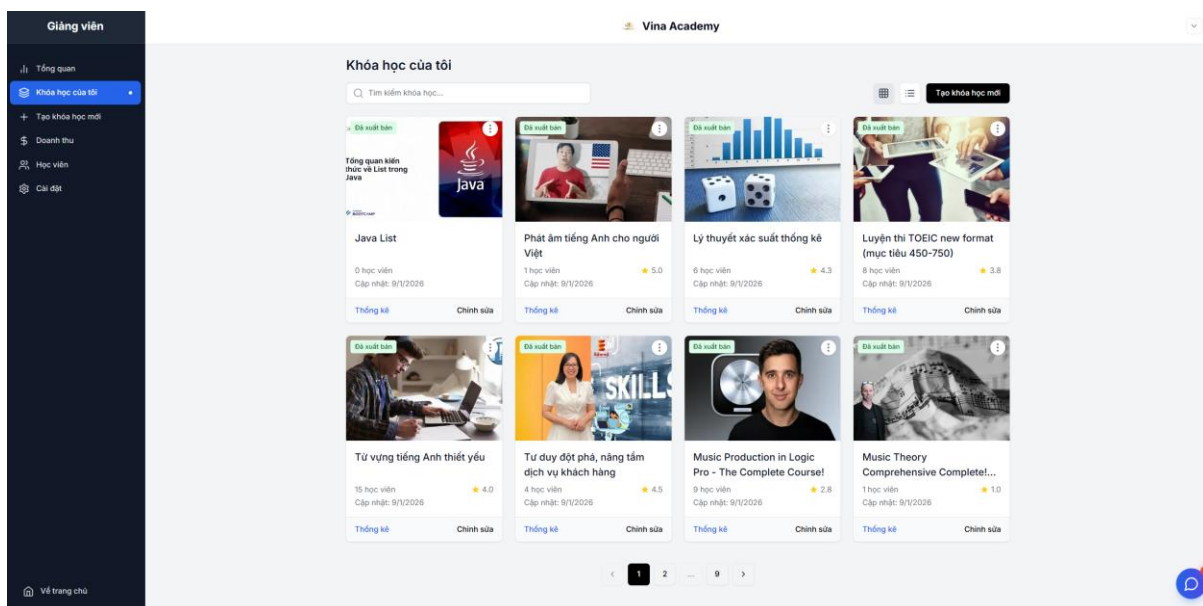
Hình 28. Giao diện đăng nhập

4.4.5. Tìm kiếm khóa học



Hình 30. Giao diện tìm kiếm khóa học

4.4.6. Quản lý khóa học giảng dạy

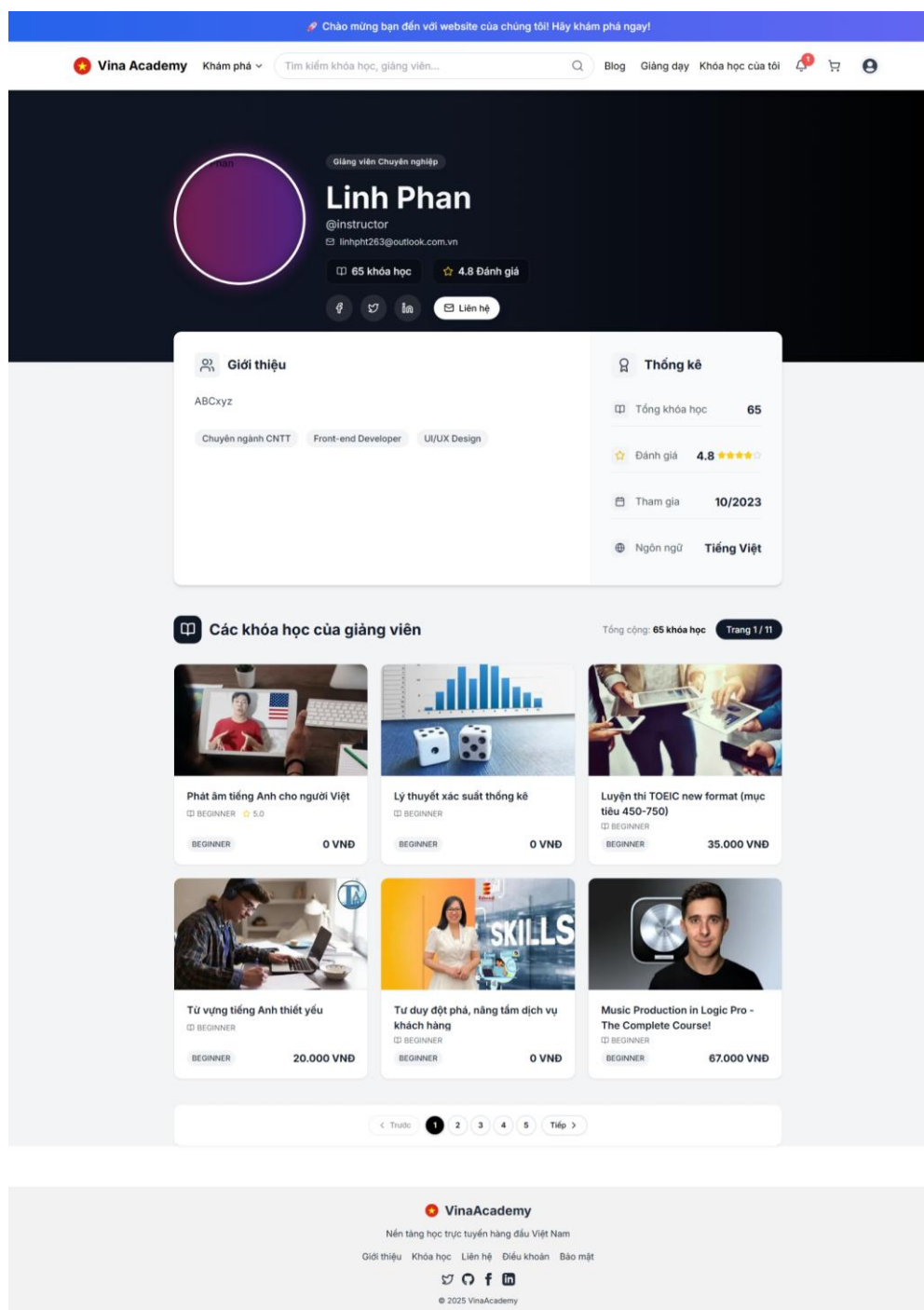


Hình 31. Giao diện Xem danh sách các khóa học giảng dạy



Hình 32. Giao diện Quản lý học viên

4.4.7. Hồ sơ



Hình 33. Giao diện Xem hồ sơ của Giảng viên

Vina Academy Khám phá

Blog Giảng dạy Khóa học của tôi

Chào mừng trở lại, **Linh Phan**

Thông tin cá nhân
Lịch sử thanh toán
Thông báo
Đăng xuất

Chỉnh sửa hồ sơ cá nhân

Cập nhật thông tin của bạn

Họ và tên:

Email:
Email không thể thay đổi

Số điện thoại:

Mô tả:

Ngày sinh:

VinaAcademy
Nền tảng học trực tuyến hàng đầu Việt Nam
Giới thiệu Khóa học Liên hệ Điều khoản Bảo mật
© 2025 VinaAcademy

Hình 34. Giao diện chỉnh sửa Hồ sơ cá nhân

4.4.8. Giỏ hàng

Vina Academy Khám phá

Blog Giảng dạy Khóa học của tôi

Giỏ hàng

Tự do tài chính cho người dưới 30 tuổi Xóa **44.000đ**
Giảng viên: Linh Phan Lưu lại sau 66.000đ
0
0 học viên Thêm vào yêu thích
0.00 giờ học • 2 bài giảng • Cơ bản

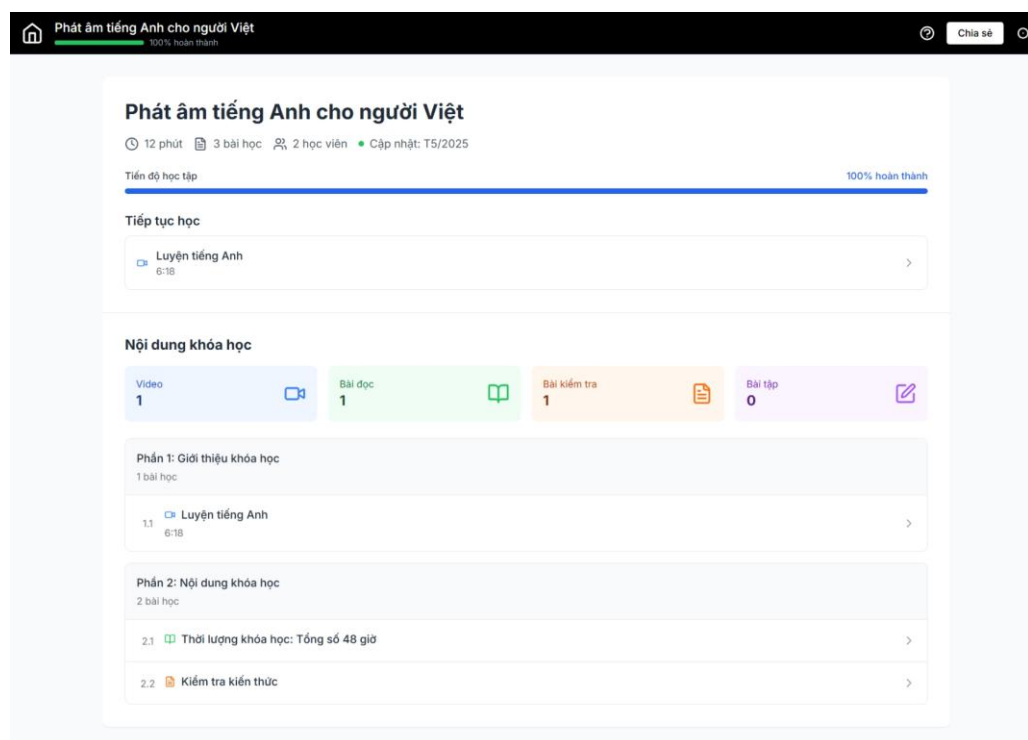
Tổng cộng:
Giá gốc: 66.000đ
Giá ưu đãi: 44.000đ

Đảm bảo hoàn tiền trong 30 ngày

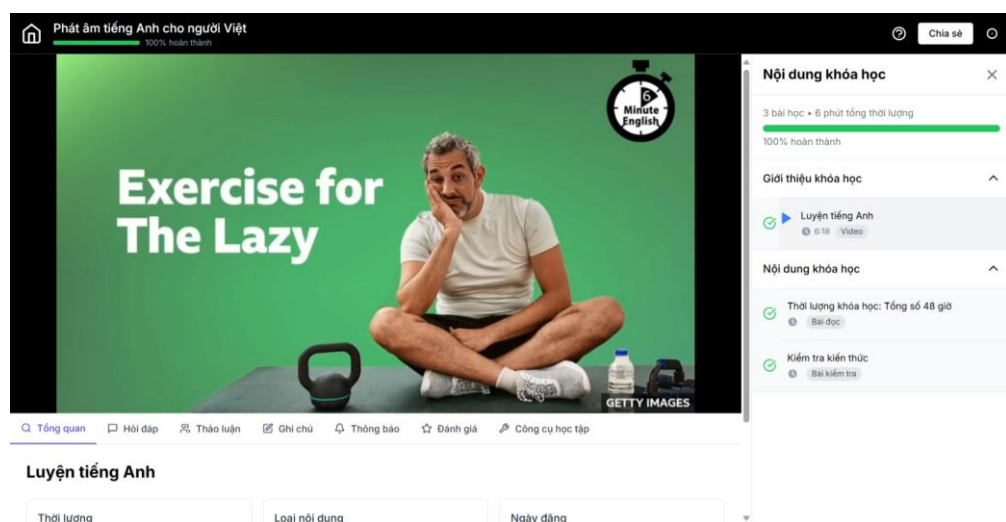
VinaAcademy
Nền tảng học trực tuyến hàng đầu Việt Nam
Giới thiệu Khóa học Liên hệ Điều khoản Bảo mật
© 2025 VinaAcademy

Hình 35. Giao diện giỏ hàng

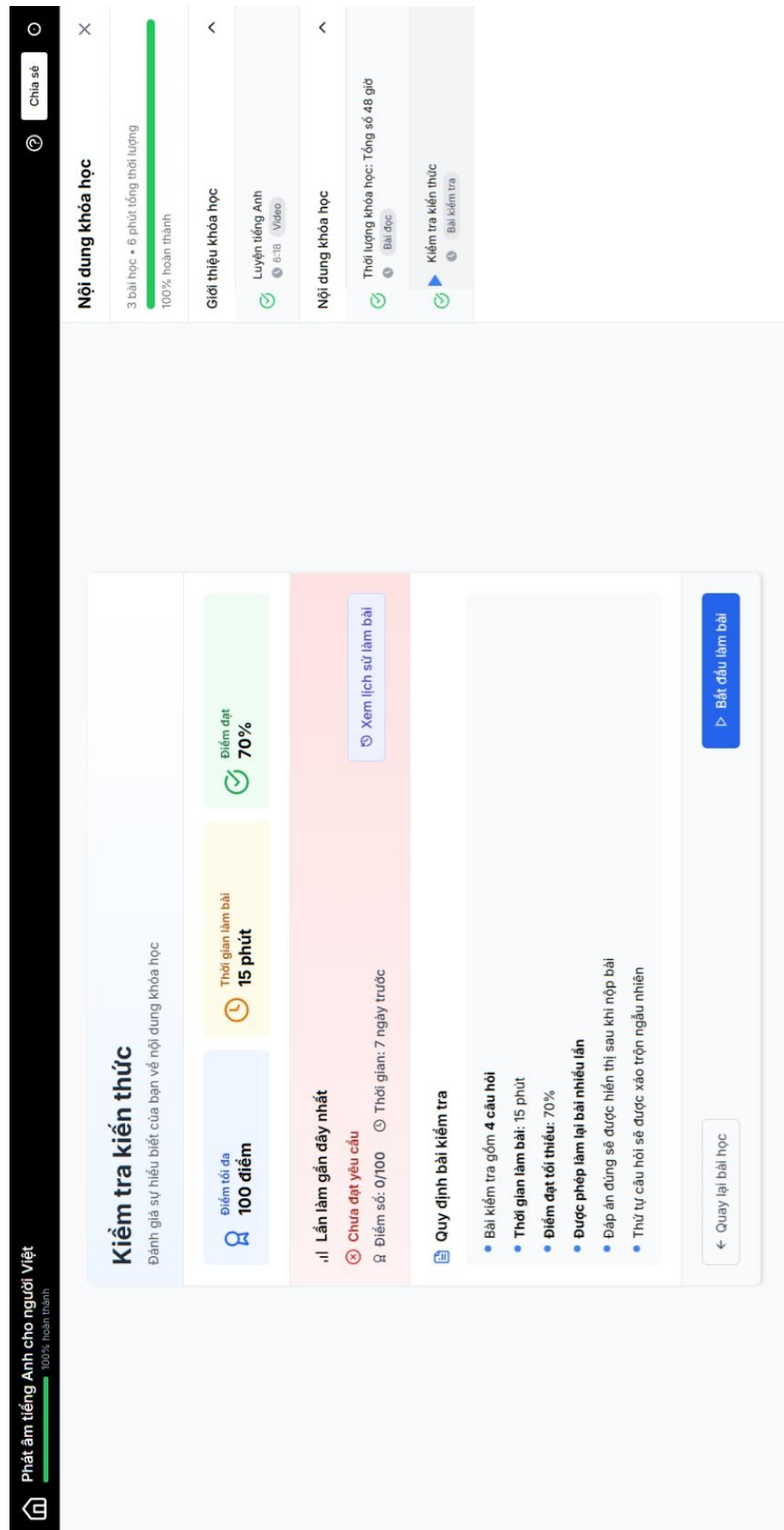
4.4.9. Học



Hình 36. Giao diện Xem nội dung khóa học



Hình 37. Giao diện Học - Video



Hình 38. Giao diện Học – Làm Trắc nghiệm

CHƯƠNG 5. CÀI ĐẶT VÀ KIỂM THỬ

5.1. LẬP TRÌNH CHỨC NĂNG

5.1.1. Tìm kiếm Ngữ nghĩa (Semantic Search Implementation)

Nhóm đã triển khai quy trình tìm kiếm đa tầng (Multi-stage Retrieval Pipeline) kết hợp giữa Vector Search, thuật toán Hybrid Scoring tùy chỉnh và Semantic Reranking để tối ưu hóa độ chính xác (Precision) và độ phủ (Recall).

a. Quy trình Tạo và Lưu trữ Vector (Embedding Generation)

Sử dụng mô hình text-embedding-3-small của OpenAI để chuyển đổi thông tin khóa học (Tên, Mô tả, Giảng viên...) thành vector 1536 chiều. Hệ thống tích hợp cơ chế Caching để giảm chi phí gọi API và tăng tốc độ xử lý.

```
@Override
public List<Float> createEmbedding(String text) {
    String normalized = text.trim().toLowerCase();
    // 1. Kiểm tra Cache (In-memory) để tránh gọi API lặp lại
    List<Float> cached = embeddingCache.get(normalized);
    if (cached != null) return cached;
    // 2. Gọi OpenAI API tạo Embedding (1536 chiều)
    EmbeddingCreateParams params = EmbeddingCreateParams.builder()
        .model(EmbeddingModel.TEXT_EMBEDDING_3_SMALL)
        .input(normalized)
        .build()
    CreateEmbeddingResponse response = client.embeddings().create(params);
    List<Float> vector = response.data().get(0).embedding();
    // 3. Lưu vào Cache
    embeddingCache.put(normalized, vector);
    return vector;
}

// Lưu Vector vào PostgreSQL (pgvector)
public void updateCourseEmbedding(CourseTransfer course) {
    // Kết hợp nhiều trường thông tin để tạo ngữ cảnh đầy đủ
    String text = course.getCourseName() + " | " + cleanHtml(course.getDescription())
        + " | " + course.getCategoryName();
    float[] vector = toFloatArray(createEmbedding(text));

    CourseEmbedding embedding = CourseEmbedding.builder()
        .courseId(course.getCourseId())
        .embedding(vector)
        .build();
    courseEmbeddingRepository.save(embedding);
}
```

Mã nguồn 1. Tạo Vector Embeddings với Caching (EmbeddingServiceImpl.java)

b. Giai đoạn 1: Truy xuất Ứng viên (Candidate Retrieval)

Sử dụng pgvector để truy xuất nhanh top 100 khóa học có vector gần nhất với truy vấn của người dùng. Câu lệnh SQL sử dụng toán tử khoảng cách <-> kết hợp với JOIN để lấy đầy đủ metadata trong một lần truy vấn.

```
private List<CourseDto> retrieveCandidates(CourseSearchRequest req, String keyword) {
    // Tạo embedding cho từ khóa tìm kiếm
    float[] embedding = embeddingService.toFloatArray(
        embeddingService.createEmbedding(keyword)
    );

    // SQL truy xuất ứng viên (Candidates)
    String sql = """
        SELECT c.*, ce.embedding, cate.slug as category_slug, u.full_name as
instructor_name,
            (ce.embedding <-> (:vector)::vector) AS distance -- Tính khoảng cách vector
        FROM courses c
        INNER JOIN course_embedding ce ON c.id = ce.course_id
        INNER JOIN categories cate ON c.category_id = cate.id
        INNER JOIN course_instructor ci ON c.id = ci.course_id
        INNER JOIN users u ON ci.user_id = u.id
        """ + CourseQueryBuilder.buildWhere(req, params) + """
        ORDER BY ce.embedding <-> (:vector)::vector ASC
        LIMIT :limit OFFSET :offset
        """;

    return jdbcTemplate.query(sql, params, new CourseRowMapper());
}
```

Mã nguồn 2. Truy vấn Vector SQL (CourseSearchServiceImpl.java)

c. Giai đoạn 2: Thuật toán Lai (Hybrid Scoring)

Vector Search đôi khi bỏ qua các từ khóa chính xác (Exact match) hoặc các khóa học chất lượng cao. Hệ thống khắc phục bằng cách tính lại điểm phù hợp (RelevanceScore) dựa trên công thức trọng số hỗn hợp.

```
private List<CourseDto> hybridSearch(List<CourseDto> candidates, String originalQuery)
{
    String kw = normalizeText(originalQuery);

    for (CourseDto course : candidates) {
        // 1. Điểm ngữ nghĩa (Vector Score): Dựa trên khoảng cách cosine (0.0 - 1.0)
        double distance = course.getDistance();
        float vectorScore = (float) Math.max(0.0, Math.min(1.0, 1.0 - distance));

        // 2. Điểm từ vựng (Lexical Score): Tự cài đặt thuật toán BM25 đơn giản
        // Ưu tiên khớp chính xác tiêu đề, danh mục
        float bm25Score = calculateBM25Score(kw, course);

        // 3. Điểm chất lượng (Quality Score): Dựa trên Rating và Số lượng học viên
        float quality = calculateQuality(course);

        // CÔNG THỨC TỔNG HỢP: 50% Vector + 35% Từ khóa + 15% Chất lượng
        float finalScore = (vectorScore * 0.50f) + (bm25Score * 0.35f) + (quality * 0.15f);

        course.setRelevanceScore(Math.max(0f, Math.min(1f, finalScore)));
    }

    // Sắp xếp lại danh sách theo điểm mới
    candidates.sort((c1, c2) -> Float.compare(c2.getRelevanceScore(),
c1.getRelevanceScore()));
    return candidates;
}
```

Mã nguồn 3. Logic tính điểm Hybrid (Hybrid Search Logic)

d. Giai đoạn 3: Tái xếp hạng ngữ nghĩa (Semantic Reranking)

Re-ranking: lấy Top 10 khoá học có mức điểm tương đồng lớn nhất, thông qua mô hình Cross-Encoder của Jina AI, để đánh giá lại từng khoá học theo ngữ nghĩa sát nhất với keyword.

Reranker có độ chính xác cao hơn Vector Search nhưng tốc độ chậm với các tập dữ liệu lớn, nên nhóm áp dụng sau khi đã thực hiện lọc dữ liệu để mang lại kết quả tối ưu nhất.

```
public List<ScoredDocument> batchRerank(String query, List<String> docs) {  
    // Kiểm tra Cache Rerank  
    String cacheKey = generateCacheKey(query, docs);  
    if (rerankCache.containsKey(cacheKey)) return rerankCache.get(cacheKey);  
  
    // Gọi API Jina Reranker  
    ObjectNode requestBody = objectMapper.createObjectNode();  
    requestBody.put("model", "jina-reranker-v3");  
    requestBody.put("query", query);  
    // ... (Add documents to payload)  
  
    HttpResponse<String> response = httpClient.send(request,  
        HttpResponse.BodyHandlers.ofString());  
  
    // Phân tích kết quả và cập nhật lại điểm số  
    // Kết quả Rerank giúp đưa những khóa học thực sự liên quan lên vị trí số 1  
    // ...  
  
    return result;  
}
```

Việc áp dụng mô hình Hybrid Search với 3 giai đoạn giúp chức năng tìm kiếm được cải thiện hiệu suất với retrieval - pgvector giúp lọc nhanh trong hàng nghìn bản ghi, Hybrid Scoring - kết hợp BM25 giúp đảm bảo người dùng tìm chính xác tên khóa học vẫn ra kết quả đúng và Reranking - Jina AI giúp hiểu sâu ngữ cảnh phức tạp mà Embedding model nhỏ (text-embedding-3-small) có thể bỏ sót, nâng cao trải nghiệm người dùng tối đa."

5.1.2. Chatbot Thông minh với Agentic RAG

Khác với chức năng tìm kiếm khóa học (sử dụng Java), dịch vụ Chatbot được xây dựng trên nền tảng Python (FastAPI) để tận dụng hệ sinh thái AI mạnh mẽ. Hệ thống áp dụng kiến trúc Agentic RAG với LangGraph, cho phép AI tự động quyết định công cụ (Tools) cần sử dụng dựa trên ý định của người dùng.

a. Luồng xử lý hội thoại (Conversation Flow)

Controller đóng vai trò tiếp nhận câu hỏi và chuyển tiếp đến Agent xử lý.


```
@router.post("/chat/stream")
async def chat_stream(
    request: ChatRequest, # {"message": "Gợi ý khóa Python", "lesson_id": null}
    chatbot_service: ChatbotService = Depends(get_chatbot_service),
    user_info: dict = Depends(AuthService.get_user_info)
) -> StreamingResponse:
    """
    Stream chat responses với Server-Sent Events (SSE).

    Response format:
    {"type": "tool_call", "text": "Đang tìm kiếm..."}
    {"type": "text", "text": "Tôi đã tìm thấy..."}
    """

    # Tạo context từ request
    context = ChatContext(
        user_id=user_info["user_id"],
        lesson_id=request.lesson_id,
        course_id=request.course_id
    )

    async def event_stream():
        async for event in chatbot_service.stream_chat(
            user_message=request.message,
            context=context
        ):
            yield f"data: {json.dumps(event, ensure_ascii=False)}\n\n"

    return StreamingResponse(event_stream(), media_type="text/event-stream")
```

Mã nguồn 4. API Controller tiếp nhận hội thoại (chatbot_controller.py)

b. Xây dựng Agent thông minh (LangGraph Agent)

Đây là "bộ não" của Chatbot. Thay vì code cứng nhắc (if-else), Agent sử dụng LLM để phân tích ngữ nghĩa và quyết định gọi Tool nào: tìm khóa học hay giải thích bài học.

```
@router.post("/chat/stream")
async def chat_stream(
    request: ChatRequest, # {"message": "Gợi ý khóa Python", "lesson_id": null}
    chatbot_service: ChatbotService = Depends(get_chatbot_service),
    user_info: dict = Depends(AuthService.get_user_info)
) -> StreamingResponse:
    """
    Stream chat responses với Server-Sent Events (SSE).

    Response format:
    {"type": "tool_call", "text": "Đang tìm kiếm..."}
    {"type": "text", "text": "Tôi đã tìm thấy..."}
    """

    # Tạo context từ request
    context = ChatContext(
        user_id=user_info["user_id"],
        lesson_id=request.lesson_id,
        course_id=request.course_id
    )

    async def event_stream():
        async for event in chatbot_service.stream_chat(
            user_message=request.message,
            context=context
        ):
            yield f"data: {json.dumps(event, ensure_ascii=False)}\n\n"

    return StreamingResponse(event_stream(), media_type="text/event-stream")
```

Mã nguồn 5. Cấu hình LangGraph Agent (*chatbot_service.py*)

c. Công cụ tích hợp (Agent Tools Implementation)

Các hàm này được "gắn" vào Agent để thực hiện các tác vụ cụ thể như truy vấn Vector Database.

```
@tool
async def search_courses(
    query: str, # "Khóa học Python cho người mới"
    course_level: Optional[str] = None,
    min_price: Optional[float] = None,
    max_price: Optional[float] = None,
    min_rating: Optional[float] = None
) -> str:
    """
    Tìm kiếm khóa học bằng Vector Search (pgvector).

    Quy trình RAG:
    1. Nhận query từ LLM
    2. Gọi Vector Search Service
    3. Trả về kết quả format đẹp
    """

    # 1. Try keyword search first (fast)
    result = await search_courses_keyword(
        keyword=query,
        size=5,
        filters={
            "courseLevel": course_level,
            "minPrice": min_price,
            "maxPrice": max_price,
            "minRating": min_rating
        }
    )

    if result['status'] == 'SUCCESS' and result['data']['content']:
        courses = result['data']['content']
        return PromptService.get_courses_recommend_prompt(courses)

    # 2. Fallback to semantic search (vector search)
    result = await search_courses_semantic(
        query=query,
        filters=filters,
        size=5
    )

    if result['status'] != 'SUCCESS':
        return "❌ Không tìm thấy khóa học phù hợp."

    courses = result['data']['content']
    return PromptService.get_courses_recommend_prompt(courses)
```

Mã nguồn 6. Định nghĩa công cụ RAG (*agent_tools_service.py*) - Tìm kiếm khóa học (Vector Search)

```
@tool
async def get_lesson_context(lesson_id: str, runtime: ToolRuntime) -> str:
    """
    Truy xuất nội dung bài học từ PostgreSQL.

    Dùng khi:
    - User hỏi về bài học đang học
    - Cần giải thích nội dung bài
    """
    context = runtime.context

    if not context.lesson_id:
        return "❗ Không có bài học cụ thể đang được học."

    lesson_uuid = UUID(lesson_id)

    # Query database với JOIN
    lesson_context = await service.get_single_lesson_context(lesson_uuid)

    if not lesson_context:
        return "❌ Không tìm thấy nội dung bài học."

    # Format context cho LLM
    context_text = [
        f"📖 **Khóa học:** {lesson_context['course_name']}",
        f"📖 **Bài học hiện tại:** {lesson_context['lesson_title']}",
        f"📖 **Mô tả:** {lesson_context['lesson_description']}",
    ]

    # Nếu là bài đọc, thêm full content
    if lesson_context['lesson_type'] == 'READING':
        context_text.append(
            f"\n📖 **Nội dung:**\n{lesson_context['reading_content']}"
        )

    return "\n".join(context_text)
```

Mã nguồn 7. Định nghĩa công cụ RAG (*agent_tools_service.py*) - Lấy context bài học từ DB

d. Chiến lược lai (Hybrid Retrieval)

Để tăng độ chính xác khi tìm kiếm trong tài liệu bài giảng, hệ thống sử dụng thuật toán Reciprocal Rank Fusion (RRF) để kết hợp kết quả từ tìm kiếm từ khóa (BM25) và tìm kiếm Vector.

```
class RRFFusion:
    """Reciprocal Rank Fusion - Kết hợp BM25 + Dense"""

    def __init__(self, bm25, dense, rrf_k: int = 60):
        self.bm25 = bm25
        self.dense = dense
        self.rrf_k = rrf_k

    def fuse(self, query: str, top_k: int = 10):
        """
        Hybrid Search = BM25 (keyword) + Dense (vector)

        RRF score = 1/(k + rank_bm25) + 1/(k + rank_dense)
        """
        bm25_results = self.bm25.search(query, top_k)
        dense_results = self.dense.search(query, top_k)

        scores = {}

        # BM25 scores
        for rank, (idx, _, _) in enumerate(bm25_results):
            scores[idx] = scores.get(idx, 0) + 1 / (self.rrf_k + rank + 1)

        # Dense scores
        for rank, (idx, _, _) in enumerate(dense_results):
            scores[idx] = scores.get(idx, 0) + 1 / (self.rrf_k + rank + 1)

        # Rerank
        reranked = sorted(scores.items(), key=lambda x: x[1], reverse=True)[:top_k]
        return [(idx, self.bm25.texts[idx]) for idx, _ in reranked]
```

Mã nguồn 8. Thuật toán tổng hợp kết quả tìm kiếm (fusion.py)

5.1.3. Cơ chế Hướng sự kiện và Xử lý Bất đồng bộ (Event-Driven & Async Implementation)

Để giải quyết bài toán hiệu năng khi tích hợp nhiều dịch vụ bên thứ 3 (như AI Service, Email Service), hệ thống áp dụng triệt để cơ chế xử lý bất đồng bộ kết hợp với Apache Kafka. Kiến trúc này giúp tách rời (decouple) các luồng nghiệp vụ chính khỏi các tác vụ phụ trợ, giảm độ trễ phản hồi cho người dùng cuối.

a. Cấu hình Thread Pool cho Tác vụ Nền (Async Configuration)

Thay vì để hệ thống tự quản lý luồng một cách không kiểm soát, nhóm thiết lập một Thread Pool riêng biệt cho các tác vụ phân tích AI. Cấu hình này đảm bảo tài nguyên CPU không bị cạn kiệt khi tải cao.

```
@Configuration
@EnableAsync // ← Bật async processing
@Slf4j
public class SentimentAnalysisConfig {

    /**
     * Async executor riêng cho sentiment analysis
     * Tránh blocking main application thread
     */
    @Bean(name = "sentimentAnalysisExecutor")
    public Executor sentimentAnalysisExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();

        // Core pool: 5 threads luôn chạy
        executor.setCorePoolSize(5);

        // Max pool: tối đa 10 threads
        executor.setMaxPoolSize(10);

        // Queue: 100 tasks chờ trong hàng đợi
        executor.setQueueCapacity(100);

        // Thread name prefix để debug
        executor.setThreadNamePrefix("sentiment-async-");

        // Graceful shutdown
        executor.setWaitForTasksToCompleteOnShutdown(true);
        executor.setAwaitTerminationSeconds(60);

        executor.initialize();

        log.info("✅ Initialized sentiment analysis async executor: core={}, max={},
queue={}",
            executor.getCorePoolSize(),
            executor.getMaxPoolSize(),
            executor.getQueueCapacity()
        );

        return executor;
    }
}
```

Mã nguồn 9. Cấu hình Async Executor (SentimentAnalysisConfig.java)

b. Producer gửi sự kiện Kafka (Event Publisher)

Lớp Producer chịu trách nhiệm đẩy các sự kiện nghiệp vụ (như "Cần gửi thông báo", "Cần gửi Email") vào Kafka Topic. Việc sử dụng `CompletableFuture` (callback) giúp ứng dụng nắm bắt được trạng thái gửi tin mà không bị block.

```
@Slf4j
@Service
@RequiredArgsConstructor
public class NotificationProducer {
    private final KafkaTemplate<String, Object> kafkaTemplate;

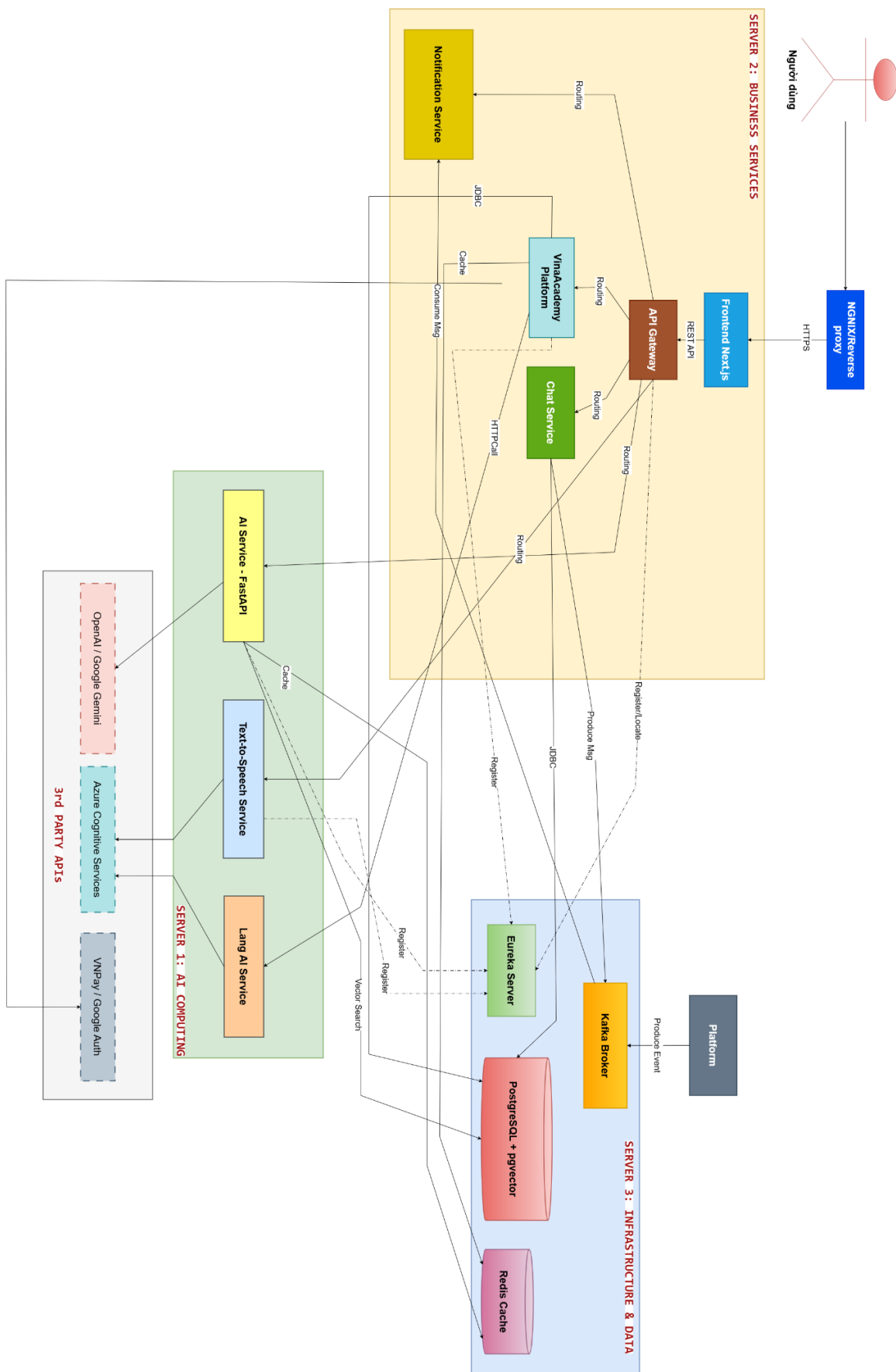
    /**
     * Gửi notification event tới Kafka topic
     * Sử dụng CompletableFuture để xử lý bất đồng bộ
     */
    public void sendNotification(NotificationCreateEvent event) {
        kafkaTemplate
            .send(KafkaTopic.NOTIFICATION_TOPIC, event)
            .whenComplete((res, ex) -> {
                if (ex != null) {
                    log.error("❌ Failed to send notification event to Kafka", ex);
                } else {
                    log.info(
                        "✅ Sent notification event to Kafka topic={} partition={} offset=",
                        res.getRecordMetadata().topic(),
                        res.getRecordMetadata().partition(),
                        res.getRecordMetadata().offset()
                    );
                }
            });
    }
}
```

Mã nguồn 10. Gửi sự kiện thông báo bất đồng bộ (*NotificationProducer.java*)

5.2. TỔ CHỨC MÃ NGUỒN VÀ CHIẾN LƯỢC TRIỂN KHAI

5.2.1. Chiến lược và Mô hình Triển khai

Do đặc thù của hệ thống Vina Academy được xây dựng dựa trên kiến trúc Microservices kết hợp với các mô hình phức tạp như Event-Driven Architecture (EDA) và AI Integration, việc triển khai thủ công từng dịch vụ là không khả thi và tiềm ẩn nhiều rủi ro về tính không đồng nhất môi trường.



Hình 39. Kiến trúc Triển khai Vật lý (Physical Deployment Architecture)

- Do đó, nhóm thực hiện áp dụng chiến lược Containerization (Container hóa) toàn diện với các nguyên tắc cốt lõi:
- Tính cô lập (Isolation): Mỗi dịch vụ (Java, Python, Node.js, Go) chạy trong một container riêng biệt, loại bỏ hoàn toàn xung đột thư viện (Dependency Conflict).
- Tính nhất quán (Consistency): Đảm bảo môi trường phát triển (Development) đồng nhất tuyệt đối với môi trường triển khai (Production).
- Khả năng điều phối (Orchestration): Sử dụng Docker Compose để quản lý vòng đời, thứ tự khởi động và mạng kết nối giữa 10 dịch vụ thành phần. Để đảm bảo hiệu năng và khả năng chịu lỗi, hệ thống được triển khai trên cụm 3 máy chủ vật lý (hoặc VPS Droplets) riêng biệt, kết nối qua mạng nội bộ ảo (Private Network/VPC):
 - Server 1 (AI Cluster): Được trang bị cấu hình tối ưu cho tính toán (Compute Optimized). Chuyên trách các tác vụ nặng như Vector Embedding, RAG Processing và Text-to-Speech.
 - Server 2 (Business Cluster): Chứa các dịch vụ nghiệp vụ cốt lõi (Core, Chat, Frontend). Đây là nơi xử lý phần lớn lưu lượng người dùng cuối.
 - Server 3 (Infrastructure Hub): Đóng vai trò là "trái tim" của hệ thống, lưu trữ toàn bộ CSDL (PostgreSQL, Redis) và điều phối luồng sự kiện (Kafka, Zookeeper, Eureka).

Việc tách biệt hạ tầng (Server 3) khỏi các dịch vụ ứng dụng (Server 1 & 2) giúp đảm bảo an toàn dữ liệu: ngay cả khi các service nghiệp vụ bị quá tải hoặc khởi động lại, tầng dữ liệu vẫn hoạt động ổn định.

5.2.2. Cấu trúc Repository

Mã nguồn được phân chia thành các nhóm chức năng logic:

- Nhóm Dịch vụ Nghiệp vụ (Business Domain Services):
 - *course-service (Java)*: Quản lý nội dung khóa học, bài giảng và lộ trình học tập.

- *chat-service (Java) & notification-service (Java)*: Xử lý tương tác thời gian thực và thông báo bất đồng bộ.
- Nhóm Dịch vụ AI & Xử lý thông minh (AI & Intelligence Services): Hệ thống áp dụng chiến lược Polyglot Programming (Đa ngôn ngữ) để tận dụng thế mạnh của từng công nghệ:
 - *ai-service (Python)*: Tận dụng hệ sinh thái thư viện AI mạnh mẽ của Python để xử lý RAG và LLM.
 - *vector-search-service (Java)*: Tối ưu hóa việc truy vấn vector tốc độ cao.
 - *ai-lang-service (Go)*: Xử lý các tác vụ ngôn ngữ cần hiệu năng cao và độ trễ thấp (Low latency).
 - *text-to-speech-service (Java)*: Tích hợp chuyển đổi văn bản sang giọng nói.
- Nhóm Hạ tầng & Cấu hình (Infrastructure & Configuration):
 - *api-gateway (Java)*: Cổng vào duy nhất của hệ thống.
 - *eureka-server*: Máy chủ khám phá dịch vụ.
 - *vinaacademy-parent & common-library*: Quản lý các dependencies chung và các đối tượng DTO/Utils chia sẻ giữa các microservices, đảm bảo tính nhất quán dữ liệu.
- Nhóm Tự động hóa (Automation):
 - *ms-ci-cd (PowerShell)*: Chứa các kịch bản tự động hóa cho quy trình tích hợp và triển khai liên tục (CI/CD).

5.2.3. Kiến trúc Container (Container Architecture)

Hệ thống áp dụng mô hình ảo hóa ở cấp độ hệ điều hành (OS-level virtualization) thông qua Docker. Thay vì triển khai trực tiếp lên nền tảng máy chủ, các dịch vụ được đóng gói thành các đơn vị chuẩn hóa gọi là Container. Kiến trúc này được thiết kế theo nguyên tắc:

1. Tính Nhất quán (Consistency):
 - Mọi dịch vụ đều sử dụng các Base Image tối ưu (như `eclipse-temurin:17-jre-alpine` cho Java, `python:3.10-slim` cho Python).

- Điều này đảm bảo mã nguồn hoạt động đồng nhất từ môi trường phát triển (Local) đến môi trường sản xuất (Production), loại bỏ hoàn toàn lỗi "It works on my machine".
2. Phân tầng Dịch vụ (Layered Services): Các container được tổ chức thành 3 tầng logic trong mạng nội bộ Docker:
- Tầng Dữ liệu (Data Persistence Layer): Các container chứa trạng thái (Stateful) như PostgreSQL, Redis, Kafka. Được cấu hình Volume Mapping để đảm bảo dữ liệu bền vững.
 - Tầng Nghiệp vụ (Business Layer): Các container phi trạng thái (Stateless) như Core Service, Chat Service. Dễ dàng mở rộng (Scale-out) bằng cách nhân bản container.
 - Tầng Cổng (Gateway Layer): API Gateway đóng vai trò cửa ngõ, điều hướng giao thông mạng vào các container bên trong.
3. Tối ưu hóa Kích thước (Image Optimization): Sử dụng kỹ thuật Multi-stage Build để loại bỏ mã nguồn gốc và công cụ biên dịch (Maven, GCC) khỏi Image cuối cùng, giúp giảm kích thước trung bình từ 600MB xuống dưới 150MB, tăng tốc độ khởi động và triển khai.

5.3. QUY TRÌNH BUILD VÀ ĐÓNG GÓI (BUILD PIPELINE)

5.3.1. Quy trình tích hợp và triển khai liên tục (CI/CD pipeline)

Do hệ thống được tổ chức theo mô hình Poly-repo với sự đa dạng về ngôn ngữ lập trình (Java, Python, Go, TypeScript) và triển khai trên hạ tầng phân tán (3 Server vật lý), quy trình CI/CD được chuẩn hóa qua 5 giai đoạn tuần tự.

Quy trình này được điều phối bởi bộ công cụ tự động hóa ms-ci-cd (viết bằng PowerShell), đảm bảo mã nguồn từ máy phát triển được đưa lên môi trường sản phẩm (Production) một cách an toàn.

Giai đoạn 1: Giải quyết phụ thuộc (Dependency Resolution)

Đây là bước tiên quyết đối với các dịch vụ Java Spring Boot trong hệ thống Vina Academy.

- Vấn đề: Các microservice (course-service) phụ thuộc vào các module dùng chung. Nếu build service trước khi build thư viện, quá trình sẽ thất bại.
- Thực hiện:
 1. Build Parent POM: Cấu hình vinaacademy-parent được cài đặt vào Maven Local Repository để đồng bộ phiên bản thư viện.
 2. Build Common Library: Repository common-library (chứa common DTO, Utils, Security Config) được biên dịch và đóng gói thành file .jar.
 3. Cung cấp Artifact: Các file .jar này sẵn sàng để các service nghiệp vụ sử dụng.

Giai đoạn 2: Biên dịch đa ngôn ngữ (Polyglot Build & Test)

Hệ thống áp dụng chiến lược build chuyên biệt cho từng loại dịch vụ:

- Java Microservices (Core, Chat):
 - Sử dụng Maven để chạy Unit Test: `mvn test`.
 - Đóng gói mã nguồn thành tệp thực thi: `mvn package -DskipTests` (tạo file .jar tối ưu).
- Python AI Services (AI RAG):
 - Thiết lập môi trường ảo và cài đặt dependencies từ requirements.txt.
 - Kiểm tra cú pháp (Linting) để đảm bảo chuẩn code Python.
- Go Lang Services (AI Lang):
 - file: `go build -o app`.
- Frontend (Next.js):
 - Thực hiện `npm run build` để tạo bản dựng production, tối ưu hóa Static Assets và SSR.

Giai đoạn 3: Đóng gói Container (Dockerization)

Áp dụng kỹ thuật Multi-stage Build để tối ưu hóa kích thước và bảo mật cho Docker Image:

- Stage 1 (Builder): Sử dụng image gốc chứa đầy đủ công cụ biên dịch (Maven, JDK, GCC, Node.js). Thực hiện việc build artifact.

- Stage 2 (Runner): Sử dụng image siêu nhẹ (Alpine Linux hoặc Distrosless Java). Chỉ sao chép (COPY) file thực thi từ Stage 1 sang.
- Kết quả:
 - Image Java: Giảm từ ~900MB xuống ~500MB.
 - Image Python: Loại bỏ các file cache và công cụ dev, giảm kích thước đáng kể.

Giai đoạn 4: Phân phối Image (Registry Push)

- Gán nhãn (Tagging): Image được gán 2 tag:
 - *latest*: Phiên bản mới nhất (dùng cho Dev/Test).
 - *vX.Y.Z*: Phiên bản ổn định (Semantic Versioning) dùng cho Rollback.
- Đẩy lên kho chứa: Image được đẩy lên Docker Hub (Private/Public Repository).

```
docker push vinaacademy/ai-service:latest
```

```
docker push vinaacademy/frontend:v1.2
```

Giai đoạn 5: Triển khai Đa máy chủ (Multi-Server Deployment)

Đây là bước phức tạp nhất do hệ thống chạy trên 3 server vật lý khác nhau. Script tự động hóa (ms-ci-cd) sẽ xác định service nào thuộc server nào để thực hiện kết nối SSH tương ứng.

- Cơ chế định tuyến triển khai:
 - Nếu là AI Service / TTS / LangAI $\rightarrow$ SSH vào Server 1 (AI Computing).
 - Nếu là Frontend / Gateway / Core $\rightarrow$ SSH vào Server 2 (Business Services).
 - Nếu là Kafka / Postgres / Eureka $\rightarrow$ SSH vào Server 3 (Infrastructure).
- Thao tác trên Server đích:
 1. Secure SSH: Kết nối an toàn sử dụng RSA Key.
 2. Config Sync: Cập nhật file docker-compose.yml mới nhất lên server.
 3. Pull & Update:

```
docker-compose pull <service_name>
```

```
docker-compose up -d --no-deps --build <service_name>
```

4. Health Check: Gọi API /actuator/health hoặc /health để xác nhận service đã khởi động thành công.

5.3.2. Tự động hóa quy trình (CI/CD Automation)

Để đảm bảo tính liên tục và giảm thiểu lỗi con người, nhóm áp dụng chiến lược Hybrid CI/CD:

- CI (Continuous Integration): Sử dụng GitHub Actions để tự động kiểm thử và đóng gói Docker Image mỗi khi có code mới đẩy lên nhánh develop.
- CD (Continuous Deployment): Kết hợp giữa GitHub Actions (tự động deploy lên môi trường Production) và bộ công cụ Automation Script (PowerShell) cho các tác vụ quản trị thủ công.

5.3.3. Quy trình Tích hợp và Kiểm thử Tự động (CI Workflow)

File cấu hình .github/workflows/ci-cd.yml định nghĩa quy trình tích hợp liên tục, tập trung vào việc đảm bảo chất lượng mã nguồn thông qua kỹ thuật Service Container.

a. Chiến lược Kiểm thử Tích hợp (Integration Testing Strategy):

Khác với kiểm thử đơn vị (Unit Test) thông thường chỉ kiểm tra logic hàm bằng các đối tượng giả lập (Mock Objects), nhóm áp dụng kiểm thử tích hợp sâu với cơ sở dữ liệu thực tế ngay trong luồng CI.

- Vấn đề của Mocking: Việc giả lập Database (bằng H2 in-memory hoặc Mockito) thường không phản ánh chính xác các ràng buộc dữ liệu, trigger, hoặc các tính năng đặc thù của PostgreSQL (như pgvector), dẫn đến rủi ro mã nguồn chạy đúng khi test nhưng lỗi khi triển khai thật.
- Giải pháp - Service Containers: Sử dụng tính năng services của GitHub Actions để khởi tạo các Ephemeral Container (Container tạm thời) song song với quá trình build.

b. Cấu hình Kỹ thuật (Technical Configuration):

Trong Job test, hệ thống tự động thiết lập môi trường sạch cho mỗi lần chạy:

1. Khởi tạo Database tạm thời:

YAML

```
services:
```

```
  postgres:
```

```
    image: postgres:15 # Sử dụng đúng phiên bản
```

```
  Production
```

```
  env:
```

```
    POSTGRES_DB: backend_service_test
```

```
  ports:
```

```
    - 5432:5432
```

2. Thực thi Kiểm thử: Mã nguồn Spring Boot được cấu hình để kết nối vào database tạm thời này. Các test case sẽ thực hiện ghi/đọc dữ liệu thật để xác minh tính đúng đắn của các câu truy vấn SQL và logic nghiệp vụ.

3. Cơ chế Tự hủy (Self-Destruction): Ngay sau khi quá trình test hoàn tất (dù thành công hay thất bại), container PostgreSQL này sẽ tự động bị tiêu hủy. Điều này đảm bảo:

- Tính cô lập (Isolation): Không bị ảnh hưởng bởi dữ liệu rác từ các lần test trước.
- Tính nhất quán (Consistency): Môi trường test luôn ở trạng thái "sạch" (Clean State) ban đầu.

c. Tối ưu hóa Hiệu năng và Bảo mật trong Đóng gói (Build Performance & Security):

Giai đoạn đóng gói (*build-and-push*) chỉ được kích hoạt khi Job Test thành công (*needs: test*). Quy trình này được thiết kế để giải quyết bài toán thời gian build lâu và rủi ro lộ lọt thông tin:

- Cơ chế Caching Đa tầng (Multi-level Caching): Sử dụng *actions/cache@v4* để lưu trữ thư mục *~/.m2* (Maven Repository). Kỹ thuật này giúp hệ thống không phải tải lại hàng trăm thư viện phụ thuộc (Dependencies) mỗi lần chạy, giúp giảm thời gian build trung bình từ 5-10 phút xuống còn dưới 30 giây.

- Tăng tốc với Docker Buildx: Hệ thống sử dụng công cụ build hiện đại *docker/setup-buildx-action*. Tính năng này cho phép Docker phân tích và tái sử dụng các lớp (layers) không thay đổi của Image cũ (*cache-from: type=gha*), giúp quá trình đóng gói diễn ra gần như tức thì nếu chỉ có thay đổi nhỏ trong mã nguồn.
- Quản lý Bí mật (Secrets Management): Tuyệt đối tuân thủ nguyên tắc không lưu trữ thông tin nhạy cảm trong mã nguồn (Hard-coding). Thông tin đăng nhập Docker Hub (*DOCKERHUB_USERNAME*, *DOCKERHUB_TOKEN*) và thông tin kết nối VPS (*VPS_PASSWORD*) được mã hóa và lưu trữ an toàn trong GitHub Secrets, chỉ được đưa vào (inject) môi trường thực thi khi Workflow chạy.

5.3.4. Quy trình Triển khai (CD Workflow)

Sau khi quá trình đóng gói hoàn tất thành công, Job deploy trong GitHub Actions sẽ được kích hoạt để đưa ứng dụng lên môi trường Production. Quy trình này sử dụng chiến lược Remote SSH Execution (Thực thi từ xa qua SSH) để đảm bảo tính bảo mật và kiểm soát.

a. Cơ chế Kết nối Bảo mật (Secure Handshake) Hệ thống không mở API Docker ra internet (tránh rủi ro bảo mật), thay vào đó sử dụng *appleboy/ssh-action* để thiết lập kênh kết nối an toàn tới máy chủ VPS.

- Xác thực: Sử dụng cặp khóa SSH (SSH Key Pair). Private Key được lưu trong GitHub Secrets, đảm bảo chỉ có GitHub Actions mới có quyền truy cập vào máy chủ.
- Cấu hình:

YAML

```
name: Deploy to VPS
uses: appleboy/ssh-action@v1.0.0
with:
  host: ${ secrets.VPS_HOST } # IP của Server
  username: ${ secrets.VPS_USER }
```

Production


```
key: ${ secrets.VPS_SSH_KEY }}
```

b. Chiến lược Cập nhật Không gián đoạn (Rolling Update Strategy) Script triển khai (script block trong YAML) thực hiện chuỗi thao tác tuần tự để cập nhật dịch vụ mà giảm thiểu tối đa thời gian chết (Downtime):

1. Đồng bộ Cấu hình:

Bash

```
cd /root/ms-ci-cd
git pull origin develop
```

Lệnh này đảm bảo file docker-compose.yml trên server luôn đồng bộ với mã nguồn mới nhất, cập nhật các biến môi trường hoặc cấu hình mạng nếu có thay đổi.

2. Tải và Tái khởi tạo Container:

Bash

```
sudo docker compose pull vinaacademy-platform
sudo docker compose up -d vinaacademy-platform
```

- *pull*: Chỉ tải các layer thay đổi của image mới từ Docker Hub.
- *up -d*: Docker Compose thông minh sẽ phát hiện container đang chạy sử dụng image cũ, dừng nó lại và khởi tạo container mới từ image vừa tải. Các container khác (Database, Kafka) không bị ảnh hưởng.

3. Dọn dẹp Tài nguyên (Resource Cleanup):

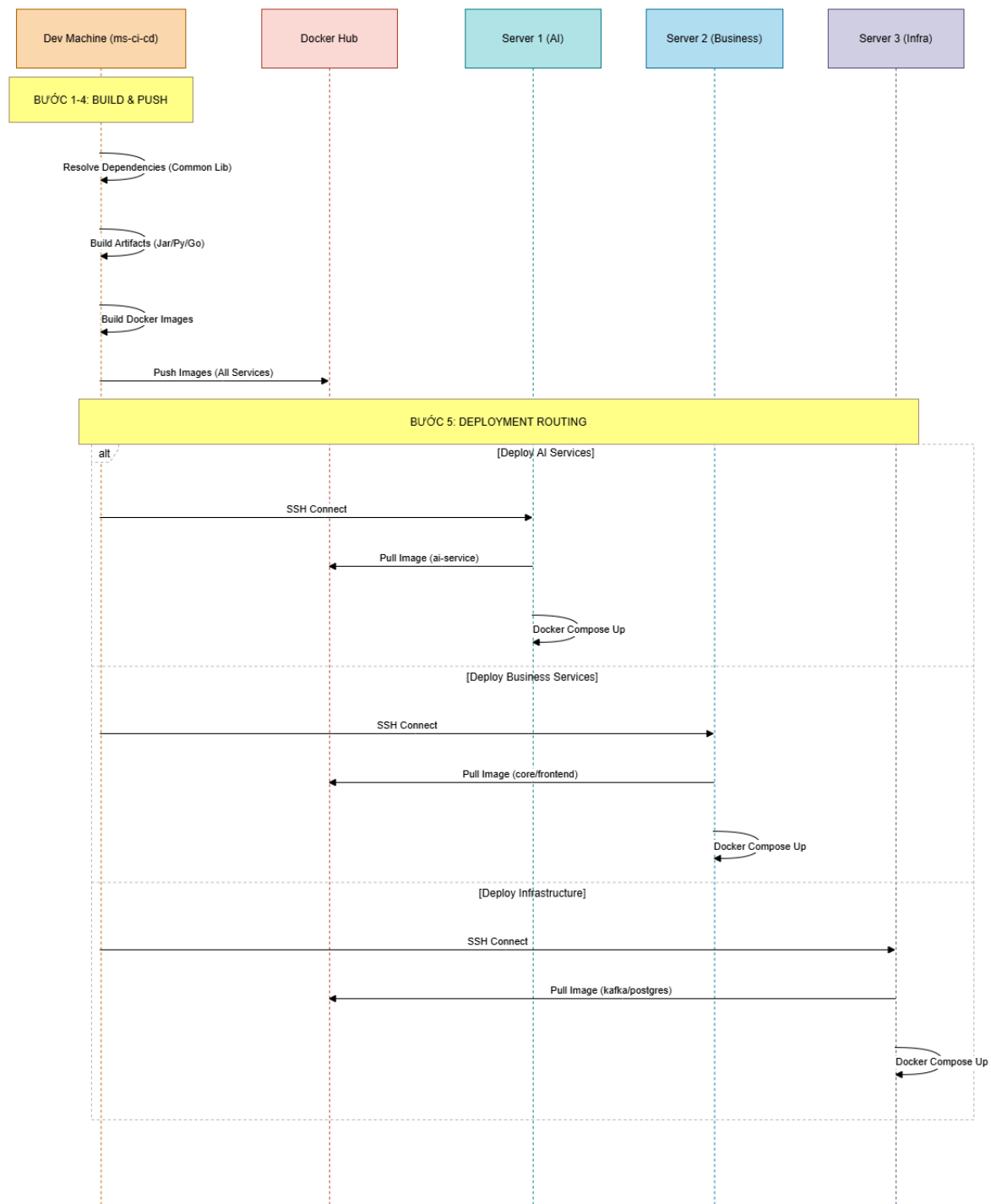
Bash

```
sudo docker image prune -f
```

Tự động xóa các "dangling images" (image cũ không còn dùng) để giải phóng dung lượng ổ cứng cho VPS, ngăn chặn sự cố đầy bộ nhớ sau nhiều lần deploy.

5.3.5. Công cụ Quản trị Tác vụ (Automation Scripting)

Bên cạnh luồng CD tự động từ GitHub, nhóm phát triển bộ công cụ ms-ci-cd (viết bằng PowerShell Core) để phục vụ các tác vụ quản trị vận hành (DevOps Tasks) một cách linh hoạt.



Hình 40. Deployment Pipeline Diagram

a. Mục đích sử dụng Trong khi GitHub Actions phù hợp cho quy trình chuẩn (Commit -> Deploy), công cụ ms-ci-cd giải quyết các bài toán thực tế phát sinh:

- Hotfix Deployment: Cần sửa gấp một lỗi nhỏ và deploy ngay lập tức mà không muốn chờ hàng chờ của GitHub Actions.

- Bulk Operations: Cần khởi động lại toàn bộ cụm Microservices hoặc cập nhật cấu hình hàng loạt.
- Local Simulation: Chạy thử quy trình deploy ngay trên máy local để debug trước khi đẩy code.

b. Kiến trúc Script Script được thiết kế theo mô hình Pipeline-as-Code, tham số hóa các biến đầu vào để tái sử dụng cho mọi service:

PowerShell

```
# Mã giả mô tả logic của script deploy.ps1
param (
    [string]$ServiceName, # Tên dịch vụ: core-service, chat-
service...
    [string]$Tag = "latest"
)

# Bước 1: Build & Push (Chạy tại Local)
Write-Host "Building $ServiceName..."
docker build -t "vinaacademy/$ServiceName:$Tag"
"../backend/$ServiceName"
docker push "vinaacademy/$ServiceName:$Tag"
# Bước 2: Remote Trigger (Gửi lệnh sang VPS)
$RemoteCommand = "
    cd /app/vina-academy
    docker-compose pull $ServiceName
    docker-compose up -d --no-deps $ServiceName
"
ssh -i key.pem user@vps-ip $RemoteCommand
```

c. Hiệu quả Vận hành Việc kết hợp giữa GitHub Actions và Automation Script mang lại sự linh hoạt tối đa:

- Tự động hóa 100%: Cho luồng phát triển thông thường (Feature Development).
- Kiểm soát thủ công: Cho các tình huống khẩn cấp hoặc bảo trì hệ thống.

- Thống nhất quy trình: Dù deploy tự động hay thủ công, hệ thống đều tuân thủ cùng một chuẩn Docker Image và Docker Compose, loại bỏ rủi ro sai lệch cấu hình.

5.4. KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

Quy trình kiểm thử được xây dựng dựa trên mô hình Tháp kiểm thử (Testing Pyramid), kết hợp giữa kiểm thử tự động (Automation) trong quy trình CI/CD và kiểm thử thủ công (Manual) cho các kịch bản phức tạp.

5.4.1. Chiến lược và Phạm vi kiểm thử

Mục tiêu là đảm bảo chất lượng phần mềm toàn diện trên 4 khía cạnh:

- Chức năng (Functional): Đảm bảo logic nghiệp vụ chính xác (Đăng ký, Thanh toán, Học tập).
- Tích hợp (Integration): Đảm bảo sự tương tác trơn tru giữa các Microservices, Database và Message Broker (Kafka).
- Hiệu năng (Performance): Đảm bảo hệ thống chịu tải được 1000 người dùng đồng thời (CCU).
- Chất lượng AI (AI Accuracy): Đánh giá độ chính xác của kết quả tìm kiếm Vector và câu trả lời của Chatbot.

5.4.2. Công cụ và Môi trường

Loại kiểm thử	Công cụ	Mục đích
Unit Testing	JUnit 5, Mockito	Kiểm thử đơn vị cho Java Backend, mock các dependencies.
Frontend Testing	Jest, React Testing Library	Kiểm thử logic giao diện Next.js.
Integration Testing	TestContainers, Postman	Kiểm thử API và kết nối Database thực tế (PostgreSQL, Redis).
Security Testing	OWASP ZAP	Quét các lỗ hổng bảo mật phổ biến (SQL Injection, XSS).

5.4.3. Quy trình thực thi

a. Kiểm thử Đơn vị (Unit Testing - Tự động hóa) Được tích hợp vào pipeline GitHub Actions. Mỗi khi có code mới, hệ thống tự động chạy toàn bộ Test Suite.

- Phạm vi: Các lớp Service, Utility, Calculation.
- Tiêu chuẩn: Code Coverage (Độ phủ mã nguồn) phải đạt tối thiểu 80%.

b. Kiểm thử Tích hợp (Integration Testing)

- Kịch bản API (Postman Collection): Nhóm đã xây dựng bộ Collection gồm hơn 50 requests bao phủ các module: Authentication, Course Management, Enrollment, Payment.
- Kiểm thử luồng sự kiện (Event-Driven): Xác minh việc Chat Service gửi tin nhắn vào Kafka và Notification Service nhận được tin nhắn đó trong vòng $< 500\text{ms}$.

c. Đánh giá mô hình AI (AI Evaluation)

- Semantic Search: Kiểm thử với bộ từ khóa tiếng Việt không dấu và từ đồng nghĩa. Kết quả trả về độ tương đồng (Cosine Similarity) đạt trung bình > 0.85 cho các nội dung liên quan.
- RAG Chatbot: Đánh giá khả năng "ảo giác" (Hallucination). Chatbot chỉ trả lời dựa trên context khóa học, từ chối trả lời các câu hỏi ngoài phạm vi.

5.4.4. Kết quả và xử lý vấn đề

Vấn đề phát hiện	Nguyên nhân	Giải pháp khắc phục
Lỗi 500 khi cập nhật khóa học	Thiếu validation dữ liệu đầu vào (null pointer).	Bổ sung lớp Validation (DTO) và trả về lỗi 400 Bad Request rõ ràng.
Tìm kiếm tiếng Việt không chính xác	pgvector mặc định so sánh vector không xử lý tốt dấu tiếng Việt.	Tích hợp thư viện xử lý ngôn ngữ VnCoreNLP để tiền xử lý văn bản trước khi embedding.

Giao dịch thanh toán treo	User tắt trình duyệt khi đang thanh toán VNPay.	Thêm Cron Job quét các giao dịch "Pending" quá 15 phút để hủy tự động hoặc hoàn tiền.
Chatbot trả lời chậm (>5s)	Quá trình truy vấn Vector trên CPU bị nghẽn.	Tối ưu hóa Indexing (HNSW) trong PostgreSQL và tăng Cache Redis cho các câu hỏi phổ biến.

PHẦN KẾT LUẬN

1. KẾT QUẢ ĐẠT ĐƯỢC

1.1. Về lý thuyết

- Kiến trúc Microservices & Event-Driven: Vận dụng thành công mô hình vi dịch vụ kết hợp với kiến trúc hướng sự kiện sử dụng Apache Kafka. Giải quyết được bài toán phân rã hệ thống, đảm bảo tính nhất quán dữ liệu và giao tiếp bất đồng bộ hiệu quả.
- Trí tuệ nhân tạo (GenAI & RAG): Nghiên cứu về kỹ thuật Retrieval-Augmented Generation (RAG) kết hợp với Cơ sở dữ liệu Vector (pgvector) để xây dựng giải pháp tìm kiếm theo ngữ nghĩa.
- Quy trình DevOps: Chuẩn hóa quy trình vận hành thông qua công nghệ Container hóa (Docker) và thiết lập và triển khai liên tục (CI/CD) tự động hóa.

1.2. Về ứng dụng

- WebApp Vina Academy đã được hoàn thiện và triển khai thực tế.
- Hạ tầng triển khai: Hệ thống vận hành trên 7 cụm vi dịch vụ độc lập (Core, User, Payment, Chatbot, AI Search...) được phân tải trên 3 máy chủ vật lý.
- Trải nghiệm người dùng thông minh:
- Tích hợp Semantic Search giúp tìm kiếm khóa học theo ngữ cảnh và ý định thay vì từ khóa cứng nhắc.
- Chatbot AI hỗ trợ giải đáp thắc mắc của học viên 24/7 dựa trên nội dung bài giảng.
- AI hỗ trợ giảng viên tạo Quiz theo yêu cầu
- Hiệu quả vận hành: Xây dựng thành công bộ công cụ ms-ci-cd tự phát triển, giúp giảm thời gian triển khai (Deploy) từ 15 phút xuống dưới 2 phút.

2. ƯU ĐIỂM

- Tính linh hoạt và khả năng mở rộng (Scalability): Nhờ thiết kế Microservices, từng module nghiệp vụ có thể được cấp phát tài nguyên và mở rộng độc lập theo nhu cầu tải thực tế.

- Độ tin cậy và khả năng tự phục hồi (Resilience): Áp dụng cơ chế ngắt mạch (Circuit Breaker) và điều phối sự kiện qua Kafka giúp hệ thống duy trì ổn định cục bộ ngay cả khi một số thành phần gặp sự cố.
- Tính tiên phong trong công nghệ: Ứng dụng thành công RAG và Vector Search mang lại trải nghiệm cá nhân hóa; mô hình bảo mật đa lớp (Defense in Depth) đảm bảo an toàn dữ liệu.

3. NHƯỢC ĐIỂM

- Chi phí tài nguyên phần cứng: Việc vận hành đồng thời nhiều Container (Java, Python, Kafka) yêu cầu cấu hình RAM nền tảng cao (>8GB), gây khó khăn khi triển khai trên các hạ tầng hosting chi phí thấp.
- Độ phức tạp trong vận hành: Việc giám sát (Monitoring) và truy vết lỗi (Distributed Tracing) giữa các vi dịch vụ đòi hỏi kiến thức chuyên sâu để xử lý khi có sự cố liên hoàn.
- Độ trễ tính năng AI: Công cụ tìm kiếm AI đôi khi gặp độ trễ xử lý (Latency) cao đối với các tập dữ liệu Vector lớn do chưa áp dụng triệt để các kỹ thuật tối ưu hóa mô hình.
- Hạn chế trong việc ngăn chặn các hành vi sao chép trái phép (quay màn hình, tải video thông qua tool).

4. HƯỚNG PHÁT TRIỂN

4.1. Nâng cấp Trí tuệ nhân tạo

Trong giai đoạn tiếp theo, nhóm định hướng phát triển hệ thống gợi ý lộ trình học tập (Learning Path Recommendation) sử dụng các mô hình học sâu (Deep Learning) để phân tích hành vi và kết quả học tập, từ đó đề xuất nội dung cá nhân hóa chính xác hơn cho từng học viên. Bên cạnh đó, tính năng tương tác giọng nói (Voice AI) cũng sẽ được nghiên cứu tích hợp, cho phép học viên luyện tập ngoại ngữ hoặc phỏng vấn giả định trực tiếp với trợ lý ảo với độ trễ thấp.

4.2. Tối ưu hóa Hạ tầng Kỹ thuật

Về mặt hạ tầng, nhóm dự kiến chuyển đổi mô hình điều phối từ Docker Compose sang Kubernetes (K8s) để tận dụng khả năng tự động mở rộng (Auto-scaling) và tự phục hồi (Self-healing) chuyên nghiệp. Đồng thời, hệ thống sẽ

được tích hợp bộ công cụ giám sát tập trung (như ELK Stack hoặc Prometheus/Grafana) để quản lý log và cảnh báo sức khỏe hệ thống theo thời gian thực, cũng như triển khai mạng phân phối nội dung (CDN) để tối ưu hóa tốc độ tải video bài giảng cho người dùng ở các khu vực địa lý khác nhau.

4.3. Tối ưu chi phí

Nghiên cứu áp dụng Serverless (AWS Lambda) cho các tác vụ không thường xuyên để giảm chi phí duy trì Server vật lý.

TÀI LIỆU THAM KHẢO

- [1] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional.
- [2] Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- [3] Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional.
- [4] Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley Professional.
- [5] Richardson, C. (2018). Microservices Patterns: With examples in Java. Manning Publications.
- [6] Walls, C. (2020). Spring in Action, 6th Edition. Manning Publications.
- [7] Banks, A., & Porcello, E. (2020). Learning React: Modern Patterns for Developing React Apps. O'Reilly Media.
- [8] Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python. O'Reilly Media.
- [9] Oracle Corporation. (2023). Java Platform, Standard Edition Documentation. Truy cập từ: <https://docs.oracle.com/en/java/javase/>
- [10] Pivotal Software. (2023). Spring Framework Reference Documentation. Truy cập từ: <https://docs.spring.io/spring-framework/docs/current/reference/html/>
- [11] Vercel Inc. (2023). Next.js Documentation. Truy cập từ: <https://nextjs.org/docs>
- [12] Facebook Inc. (2023). React Documentation. Truy cập từ: <https://react.dev/>
- [13] PostgreSQL Global Development Group. (2023). PostgreSQL Documentation. Truy cập từ: <https://www.postgresql.org/docs/>
- [14] Redis Ltd. (2023). Redis Documentation. Truy cập từ: <https://redis.io/documentation>

[15] Áp dụng Specification Design Pattern trong Laravel. (2022). Viblo.
Truy cập từ: <https://viblo.asia/p/ap-dung-specification-design-pattern-trong-laravel-0gdJzQke4z5>

PHỤ LỤC

Link website:

- <https://vnacademy.io.vn>

Account Test:

- locn562836@gmail.com admin123
- huuloc2155@gmail.com staff123
- linhph263@outlook.com.vn instructor123
- trihung987@gmail.com student123

Thông tin thanh toán:

- Ngân hàng: NCB
- Số thẻ: 9704198526191432198
- Tên chủ thẻ: NGUYEN VAN A
- Ngày phát hành: 07/15
- Mật khẩu OTP: 123456

Drive + Video:

https://drive.google.com/drive/folders/10gSIeOWz0yjAlG9nzHJ06xj98QOENryG?usp=drive_link

Link Repo: github.com/VinaAcademy

Link figma:

<https://www.figma.com/design/RNKIesvrnmYWV39uWSTnRn/Untitled?node-id=0-1&m=dev&t=P5PUncjhW7Y9135g-1>

PPT: https://www.canva.com/design/DAG9_pI4tKE/FqQunELNEgxVDMG-SEVIpA/edit?utm_content=DAG9_pI4tKE&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton